Parte 3 — Periféricos do STM32G070RB com HAL

Esta parte consolida a transição entre a base de linguagem C, o entendimento do núcleo ARM e o uso prático dos periféricos do STM32G070RB. O foco está em compreender o hardware, configurar corretamente cada bloco no STM32CubeMX/STM32CubeIDE e aplicar os padrões de uso mais úteis com HAL.

Ao longo dos capítulos, a progressão segue do controle de pinos e eventos externos até aquisição, comunicação, temporização, persistência e diagnóstico. A ideia é que o leitor consiga não apenas "fazer funcionar", mas também depurar, validar e estruturar firmware de forma mais profissional.

Visão Geral da Parte 3

- **Capítulo 1**: GPIO, EXTI, múltiplos botões e debounce.
- **Capítulo 2**: DMA, buffers, half transfer, prioridades e diagnóstico.
- **Capítulo 3**: ADC, polling, interrupção, DMA, sampling time e filtro RC.
- **Capítulo 4**: USART/UART, recepção robusta e detecção de IDLE.
- **Capítulo 5**: SPI, transações atômicas, CS, timeout e reentrância.
- **Capítulo 6**: I^2C, leitura/escrita de registradores, retries e timeout.

- **Capítulo 7**: Timers, PWM, input capture, encoder e exemplos de aquisição.
- **Capítulo 8**: RTC, wakeup, alarmes, tamper e backup registers.
- **Capítulo 9**: Independent Watchdog (IWDG) e estratégias de robustez.
- **Capítulo 10**: Flash interna, gravação, integridade e partição com bootloader.
- **Capítulo 11**: Sistema de clocks, PLL, prescalers e clocks de periféricos.
- **Capítulo 12**: Interrupções, NVIC, EXTI, pendência, flags e limpeza correta.

Capítulo 1 — GPIO no STM32G070RB (Visão de Hardware + Uso)

1. O que é o periférico GPIO e para que ele serve

GPIO (General Purpose Input/Output) é o bloco de hardware do microcontrolador responsável por interagir com sinais digitais externos. Em termos práticos, ele permite:

Ler níveis lógicos vindos de botões, sensores digitais, chaves, sinais de presença, etc. (**Inputs**).

Gerar níveis lógicos para acionar LEDs, relés (via transistor), habilitar módulos, selecionar modos, etc. (**Outputs**).

Conectar um pino a um periférico interno (UART, SPI, I²C, TIM/PWM, ADC trigger, etc.) usando função alternativa (**alternate function**).

Em alguns casos, usar o pino em modo analógico (por exemplo, como entrada do ADC), onde a lógica digital é desligada para reduzir consumo/ruído.

Em um STM32, "GPIO" não é apenas "um fio que liga e desliga". Cada pino tem circuitos internos configuráveis (modo, resistores, driver de saída, velocidade, interrupções, etc.). Entender essa parte de hardware evita bugs clássicos: leitura flutuando, LED fraco, ruído, consumo elevado, bordas com EMI, curto por erro de configuração, etc.

---

2. Como o hardware de um pino GPIO é organizado

Cada pino (ex.: PA5, PB3…) pode ser visto como um conjunto de blocos internos:

Multiplexador de função (MUX)

Escolhe "quem controla o pino":

GPIO (controle por software) ou Periférico (UART/SPI/TIM etc.).

Quando você seleciona Alternate Function, o pino deixa de ser “GPIO puro” e passa a ser controlado pelo periférico associado.

Caminho de entrada (Input Buffer)

Um comparador digital interpreta a tensão no pino como 0 ou 1.

Pode ser habilitado/desabilitado dependendo do modo (em modo analógico, normalmente é desligado).

Caminho de saída (Output Driver)

Circuito que “empurra” o pino para nível alto/baixo quando em modo de saída.

Possui duas formas típicas de condução:

Push-Pull: dirige alto e baixo ativamente (mais comum para LED, sinais digitais).

Open-Drain: só força nível baixo; nível alto depende de resistor de pull-up (comum em I^2C e “wired-AND”).

Resistores internos de Pull-up / Pull-down

Resistores internos fracos que “puxam” o pino para 1 (pull-up) ou 0 (pull-down).

Evitam entrada “flutuando” quando o pino não tem nível definido externamente (ex.: botão solto).

Controle de velocidade (slew-rate)

Ajusta quão rápidas são as bordas (subida/descida) do sinal.

Bordas muito rápidas podem aumentar EMI e ringing em cabos/placas; bordas mais lentas podem ser melhores para integridade de sinal em sinais simples.

Lógica de interrupção por pino (via EXTI)

O STM32 permite gerar interrupções por borda de subida, borda de descida ou ambas.

Isso é feito por linhas EXTI mapeadas ao número do pino (0 a 15), com seleção do port (A/B/C…).

---

3. Modos principais de um GPIO

Em STM32, cada pino normalmente assume um dos modos abaixo:

Input (Entrada)

Lê o nível lógico presente no pino.

Recomendado usar pull-up/pull-down (interno ou externo) para não flutuar.

Output (Saída)

Controla o pino em 0/1 por software.

Pode ser Push-Pull ou Open-Drain.

Alternate Function (Função Alternativa)

O pino é conectado a um periférico interno.

Ex.: PA2/PA3 como UART; PB6/PB7 como I^2C; PA8 como PWM de timer.

Analog

Desliga a parte digital para reduzir consumo e ruído.

Usado com ADC, comparadores analógicos, ou pino "desconectado" para baixa potência.

---

4. Conceitos elétricos e "pegadinhas" comuns

4.1 Entrada flutuante

Se um pino em modo Input não tem pull-up/pull-down e nada o força externamente, ele fica flutuando: pode ler 0 e 1 aleatoriamente, especialmente com ruído eletromagnético.

Solução: usar pull-up/pull-down interno (HAL) ou resistor externo.

4.2 Push-Pull vs Open-Drain

Push-Pull: melhor para sinais comuns e LEDs.

Open-Drain: obrigatório em barramentos compartilhados como I^2C (com pull-up).

Erro típico: configurar I^2C como Push-Pull → risco de conflito elétrico e mau funcionamento.

4.3 Corrente por pino e por porta

Pinos têm limites de corrente (fonte e dreno). LEDs devem usar resistor série e, dependendo do consumo, um transistor/driver. Um GPIO não é feito para alimentar cargas diretamente.

4.4 Velocidade excessiva

Configurar pinos em velocidade muito alta sem necessidade pode aumentar EMI e causar ruído. Para LED e botões, velocidades baixas geralmente são melhores.

---

5. Usando GPIO com HAL (STM32CubeIDE / STM32CubeMX)

Embora o foco desta etapa seja hardware, o livro usará HAL. A forma típica de uso no HAL é:

Habilitar clock da porta GPIO (normalmente gerado pelo CubeMX).

Configurar o pino com GPIO_InitTypeDef.

Ler e escrever com funções HAL.

5.1 Exemplo 1 — Saída (LED)

Objetivo: configurar um pino como saída e alternar o estado.

```
// Exemplo genérico (pino e porta são apenas ilustrativos)
GPIO_InitTypeDef GPIO_InitStruct = {0};

// 1) Habilita o clock da porta (exemplo: GPIOA)
__HAL_RCC_GPIOA_CLK_ENABLE();

// 2) Configura PA5 como saída push-pull, sem pull, baixa velocidade
GPIO_InitStruct.Pin = GPIO_PIN_5;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

// 3) Liga o pino
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);

// 4) Alterna (toggle)
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
```

Observações de hardware:

Push-pull é ideal para LED (com resistor série).

Velocidade baixa reduz ruído e é suficiente.

5.2 Exemplo 2 — Entrada com Pull-up (Botão)

Objetivo: ler um botão ligado ao GND (ativo em nível baixo).

```
GPIO_InitTypeDef GPIO_InitStruct = {0};

__HAL_RCC_GPIOC_CLK_ENABLE();

GPIO_InitStruct.Pin = GPIO_PIN_13;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLUP;  // garante nível alto quando solto
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

// Leitura
GPIO_PinState st = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
if (st == GPIO_PIN_RESET)
{
   // botão pressionado (pino foi ao GND)
}
```

Observações de hardware:

O pull-up interno evita flutuação.

Se o botão estiver longe (cabos longos), pode ser necessário filtro RC ou debounce por software.

5.3 Exemplo 3 — Interrupção por borda (EXTI)

Objetivo: gerar interrupção ao pressionar um botão.

```
GPIO_InitTypeDef GPIO_InitStruct = {0};

__HAL_RCC_GPIOC_CLK_ENABLE();

GPIO_InitStruct.Pin  = GPIO_PIN_13;
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING; // borda de descida: com pull-up, pressionar gera 1->0
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

// Configura prioridade e habilita no NVIC (IRQ depende do pino/linha EXTI)
HAL_NVIC_SetPriority(EXTI4_15_IRQn, 2, 0);
HAL_NVIC_EnableIRQ(EXTI4_15_IRQn);
```

Callback típico do HAL:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
   if (GPIO_Pin == GPIO_PIN_13)
   {
      // tratamento do evento do botão
   }
}
```

E a ISR:

```
void EXTI4_15_IRQHandler(void)
{
   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
}
```

Observações importantes:

Interrupção resolve “polling constante”, mas debounce continua necessário em botão mecânico.

EXTI é por linha 0..15; vários ports podem “competir” por uma mesma linha, dependendo do mapeamento.

---

6. Boas práticas para projetos reais

Para botões: use pull-up/pull-down e implemente debounce (timer/estado).

Para LEDs: sempre resistor série; evite corrente alta direto do GPIO.

Para barramentos (I^2C): open-drain + pull-up.

Para sinais rápidos (SPI/PWM): ajuste velocidade conforme necessidade e layout.

Para baixo consumo: use modo analógico em pinos não usados e evite pull-ups desnecessários.

---

7. Encerramento e ponte para os próximos tópicos

Compreender GPIO como hardware configurável (e não apenas “ler/escrever pino”) é a base para todos os periféricos do STM32, porque quase todos dependem de pinos: UART, SPI, I^2C, timers, PWM, ADC, etc. No próximo capítulo, a configuração de GPIO será reaproveitada para explicar Alternate Functions na prática e como o roteamento de pinos impacta a configuração de cada periférico.

Capítulo 1.1 — GPIO no CubeMX/CubeIDE e estrutura do MX_GPIO_Init()

1. Por que o CubeMX gera um MX_GPIO_Init()

No ecossistema STM32Cube, o CubeMX (integrado ao STM32CubeIDE) funciona como um “gerador de configuração”. Você seleciona pinos e periféricos (ex.: LED em PA5, botão em PC13, UART em PA2/PA3) e ele gera:

habilitação de clocks (RCC),

inicialização de pinos (GPIO),

inicialização de periféricos (USART, SPI, I^2C, TIM etc.),

arquivo .ioc (a “fonte da verdade” do projeto).

O objetivo é padronizar e reduzir erros na configuração de registradores, deixando o programador focar na aplicação.

---

2. Onde fica a inicialização de GPIO no projeto HAL

No projeto gerado pelo CubeIDE, é comum encontrar:

main.c chamando MX_GPIO_Init();

gpio.c e gpio.h contendo a implementação (ou parte dela), dependendo do template escolhido.

seções protegidas por comentários do tipo:

/* USER CODE BEGIN ... */

/* USER CODE END ... */

Essas seções são cruciais: tudo fora delas pode ser sobrescrito quando você "regenera" o código pelo CubeMX.

---

3. Estrutura típica do MX_GPIO_Init()

A função gerada costuma seguir uma ordem lógica (e muito importante do ponto de vista elétrico):

Habilitar clocks das portas GPIO

Sem clock, o bloco GPIO não responde.

Definir nível inicial de saídas antes de configurar o modo

Boa prática para evitar "glitches" (pulsos indesejados) ao trocar o modo do pino para saída.

Configurar pinos de saída

Modo, pull, velocidade, tipo de saída.

Configurar pinos de entrada

Modo input e pull-up/pull-down.

Configurar pinos com interrupção (EXTI)

Modo IT_FALLING/IT_RISING e configuração do NVIC.

Exemplo bem próximo do padrão gerado (ilustrativo):

```
void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
```

```
  /* 1) Clocks */
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOC_CLK_ENABLE();

  /* 2) Níveis iniciais */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

  /* 3) Saída: PA5 */
  GPIO_InitStruct.Pin = GPIO_PIN_5;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /* 4) Entrada: PC13 (botão) */
  GPIO_InitStruct.Pin = GPIO_PIN_13;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
}
```

Nota importante (de engenharia): colocar o HAL_GPIO_WritePin() antes do HAL_GPIO_Init() para saídas é uma forma simples de reduzir transientes durante a partida do firmware.

---

4. Convenções práticas para o livro e projetos

Para facilitar manutenção e leitura, é comum adotar no gpio.h:

#define LED_GPIO_Port GPIOA e #define LED_Pin GPIO_PIN_5

#define BTN_GPIO_Port GPIOC e #define BTN_Pin GPIO_PIN_13

Isso torna o código de aplicação mais expressivo:

```
HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);


if (HAL_GPIO_ReadPin(BTN_GPIO_Port, BTN_Pin) == GPIO_PIN_RESET) {
   /* botão pressionado */
}
```

Capítulo 1.2 — Alternate Function: como o pino "vira" UART, SPI, I²C, PWM…

1. O que é Alternate Function (AF)

No STM32, a maioria dos pinos é compartilhada com vários periféricos. O pino possui um multiplexador interno que pode conectá-lo a:

GPIO "normal" (entrada/saída),

periféricos internos (USART, SPI, I²C, TIM, etc.).

Quando você seleciona Alternate Function, o pino deixa de ser controlado pelo "GPIO genérico" e passa a ser controlado pelo periférico escolhido.

No STM32G0 (incluindo o STM32G070RB), isso é descrito nas tabelas de pinagem/funções alternativas do datasheet e no controle de registradores do GPIO no reference manual.

---

2. Como descobrir "qual pino serve para qual periférico"

Você tem dois caminhos corretos (e recomendados):

A) Pelo CubeMX (mais prático)

Abra a aba de pinos/periféricos.

Ative o periférico (USART2, SPI1, I2C1 etc.).

O CubeMX sugere automaticamente pinos compatíveis.

Ao escolher o pino, ele configura automaticamente o AF correto.

B) Pelo Datasheet (mais "raiz" e indispensável no entendimento)

Vá à seção "Pinouts, pin description and alternate functions".

Encontre o pino (ex.: PA2) e veja a lista de funções possíveis.

---

3. Exemplos reais e típicos no STM32G070RB

Abaixo estão mapeamentos comuns (e muito usados em placas como a NUCLEO-G070RB):

3.1 USART2 em PA2/PA3

PA2 = USART2_TX

PA3 = USART2_RX

Isso é um mapeamento clássico (inclusive usado como VCP em várias Nucleo).

3.2 I²C1 (variações por placa/projeto)

O datasheet indica I²C disponível em combinações como:

PB6/PB7 ou PB10/PB11 (dependendo do pacote/roteamento).

Em algumas configurações de placa (ex.: headers Arduino/board definitions), também é comum ver I²C em PB8/PB9 como escolha de mapeamento de projeto/placa.

Moral prática: o periférico pode ter mais de um conjunto de pinos possível. A decisão depende do layout da placa e de conflitos com outros periféricos.

3.3 SPI1 em PA5/PA6/PA7 (e NSS em outro pino)

Para SPI, o datasheet cita opções típicas incluindo PA4/PA5/PA6/PA7 (variante de mapeamento).

Em muitas placas, é comum usar:

PA5 = SCK

PA6 = MISO

PA7 = MOSI

e o NSS/CS como GPIO comum (por exemplo PB0 em alguns roteamentos de placa).

---

4. O que muda no código HAL quando o pino é Alternate Function

Você vai ver o GPIO_InitStruct.Mode como AF:

GPIO_MODE_AF_PP (Alternate Function Push-Pull)

GPIO_MODE_AF_OD (Alternate Function Open-Drain, típico de I²C)

E aparece a seleção do AF:

```
GPIO_InitStruct.Alternate = GPIO_AFx_XXXX;
```

Esse GPIO_AFx_... é exatamente a escolha do multiplexador interno (AF number). O CubeMX define isso automaticamente quando você escolhe o periférico/pino.

Exemplo ilustrativo (USART em AF push-pull):

```
GPIO_InitStruct.Pin = GPIO_PIN_2 | GPIO_PIN_3;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_PULLUP;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF1_USART2;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

Exemplo ilustrativo (I^2C em AF open-drain):

```
GPIO_InitStruct.Pin = GPIO_PIN_6 | GPIO_PIN_7;
GPIO_InitStruct.Mode = GPIO_MODE_AF_OD;
GPIO_InitStruct.Pull = GPIO_PULLUP;       // muitas vezes necessário
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
GPIO_InitStruct.Alternate = GPIO_AFx_I2C1;  // AF depende do pino
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

Observação: para I^2C, Open-Drain + Pull-up não é "opcional": é parte do funcionamento elétrico do barramento.

Capítulo 1.3 — GPIO + EXTI no STM32G070RB

1. Objetivo do EXTI

EXTI (External Interrupt/Event Controller) é o bloco de hardware que permite ao microcontrolador reagir automaticamente a mudanças em pinos sem precisar ficar lendo o estado em loop (polling). Com EXTI é possível:

gerar interrupções quando um pino muda (borda de subida, descida ou ambas);

gerar eventos internos (muito usados para acordar o sistema ou sincronizar blocos, dependendo do modo de baixo consumo).

Neste capítulo o foco é interrupção por pino (GPIO + EXTI), que é o caso típico para botões, chaves, sensores digitais com pulso, "fault pins", etc.

---

2. Visão de hardware: como o caminho GPIO → EXTI funciona

2.1 Linhas EXTI (0 a 15)

O STM32 organiza interrupções externas em linhas numeradas. Para GPIO, as linhas mais importantes são:

EXTI0 … EXTI15: associadas ao número do pino (0 a 15)

Isso significa que:

PA0, PB0, PC0… competem pela mesma linha EXTI0

PA13, PB13, PC13… competem pela mesma linha EXTI13

Em outras palavras: o número do pino define a linha EXTI, e o "port" (A/B/C…) é escolhido por um multiplexador.

2.2 Seleção do port (roteamento para a linha EXTI)

Existe uma lógica de roteamento (um multiplexador interno) que define qual porta alimenta aquela linha EXTI.

Exemplo:

Você quer interrupção no PC13 → você precisa rotear Port C para a linha 13.

O CubeMX faz isso automaticamente. Do ponto de vista de hardware, essa seleção evita que dois pinos com mesmo número (ex.: PA13 e PC13) disparem a mesma linha ao mesmo tempo. Você escolhe um deles.

2.3 Detecção de borda

Cada linha EXTI pode ser configurada para:

borda de subida (0 → 1)

borda de descida (1 → 0)

ambas

O EXTI não “mede nível por tempo”; ele detecta transições.

2.4 Flags pendentes (por que a interrupção “fica voltando”)

Quando a transição é detectada, o EXTI seta uma flag pendente (pending flag) para aquela linha.

A CPU entra na ISR.

O handler precisa limpar a flag pendente, senão a interrupção pode voltar imediatamente.

No HAL, essa limpeza é feita dentro do fluxo HAL_GPIO_EXTI_IRQHandler(pin).

---

3. Quando usar EXTI em vez de polling

EXTI é indicado quando:

o evento é esporádico (botão pressionado, falha, sensor de pulso);

você quer economizar CPU (principalmente se houver RTOS);

você quer reação mais rápida e determinística.

Polling é aceitável quando:

o estado muda com frequência e você já precisa de amostragem constante;

sua arquitetura já tem um loop de tempo fixo (ex.: 1 ms) e o custo é baixo;

você precisa de “nível estável” e não apenas borda.

---

4. Botões reais: bouncing e ruído (o problema número 1)

Botões mecânicos não geram uma transição limpa. Ao pressionar/soltar, o contato “quica” por alguns milissegundos, produzindo várias bordas falsas.

Sem debounce:

um único clique pode gerar 5, 10, 20 interrupções.

Logo, GPIO + EXTI quase sempre exige debounce para botões e chaves mecânicas.

---

5. Configuração com CubeMX (fluxo prático)

Para configurar um botão com interrupção:

selecione o pino como GPIO_EXTI

escolha o gatilho: Rising, Falling ou Rising/Falling

configure Pull-up ou Pull-down conforme o circuito

no NVIC, habilite a interrupção correspondente ao grupo EXTI

O CubeMX então gera:

MX_GPIO_Init() com o pino em GPIO_MODE_IT_...

configuração do NVIC (HAL_NVIC_SetPriority e HAL_NVIC_EnableIRQ)

o nome correto da IRQ handler para aquela faixa de linhas (ex.: EXTI4_15)

---

6. Exemplo testável: botão em PC13 com pull-up e borda de descida

6.1 Hardware recomendado (conceito)

Um lado do botão no GND

Outro lado no PC13

Pull-up interno habilitado (ou resistor externo)

Assim:

solto → lê 1

pressionado → lê 0 (borda de descida ao pressionar)

6.2 MX_GPIO_Init() (HAL)

Exemplo do que normalmente aparece (ilustrativo, mas fiel ao padrão):

```
void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};


  __HAL_RCC_GPIOC_CLK_ENABLE();
```

```
   GPIO_InitStruct.Pin  = GPIO_PIN_13;
   GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
   GPIO_InitStruct.Pull = GPIO_PULLUP;
   HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

   HAL_NVIC_SetPriority(EXTI4_15_IRQn, 2, 0);
   HAL_NVIC_EnableIRQ(EXTI4_15_IRQn);
}
```

6.3 ISR e callback

O fluxo padrão HAL é:

a IRQ chama o handler do HAL

o HAL limpa a flag pendente e chama o callback de usuário

```
void EXTI4_15_IRQHandler(void)
{
   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
   if (GPIO_Pin == GPIO_PIN_13)
   {
      // evento do botão
   }
}
```

Esse padrão é importante porque:

garante limpeza correta das flags;

unifica o tratamento para diferentes pinos EXTI.

---

7. Debounce por software

7.1 Requisitos do debounce correto

Um debounce bom precisa:

ignorar repetição de interrupções em janela curta (ex.: 30–50 ms);

confirmar o estado final do pino (evitar falsos por ruído);

ser previsível e testável.

A estratégia abaixo é simples, eficiente e muito usada:

Estratégia:

na primeira interrupção, bloqueia novas por X ms (debounce time)

ao final do tempo, lê o pino e confirma se o botão realmente está pressionado/solto.

Isso evita contar "quicadas" internas do botão.

7.2 Implementação (HAL + SysTick) — exemplo testável

A ideia é manter um timestamp por pino e aceitar evento apenas se passou do tempo.

Definições:

```
#define BTN_DEBOUNCE_MS  50


static volatile uint32_t btn_last_tick = 0;

static volatile uint8_t  btn_event = 0;   // usado para teste no main
```

Callback com debounce por tempo:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

{

   if (GPIO_Pin == GPIO_PIN_13)

   {

      uint32_t now = HAL_GetTick();
```

```
        if ((now - btn_last_tick) < BTN_DEBOUNCE_MS)
            return; // ignora bouncing

        btn_last_tick = now;

        // Confirma nível (opcional, mas recomendado)
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET)
        {
            btn_event = 1; // botão confirmado pressionado
        }
    }
}
```

Teste no while(1)

```
while (1)
{
    if (btn_event)
    {
        btn_event = 0;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); // LED
    }
}
```

Esse exemplo é “testável” porque:

- cada clique (mesmo com bouncing) gera um toggle;
- o comportamento é reproduzível;
- não exige periféricos extras.

---

8. Melhorias e cuidados importantes

8.1 Evite trabalho pesado dentro da interrupção

Na interrupção (callback) faça:

validação rápida

set de flags

acordar tarefa / sinalizar evento (se houver RTOS)

Evite:

printf pesado

delays

loops longos

chamadas de drivers demoradas

O EXTI deve ser rápido para não aumentar latência de outras interrupções.

8.2 Interrupção em ambas as bordas

Usar RISING_FALLING é útil quando você quer detectar:

pressionar e soltar

medir largura de pulso (com cuidado)

Mas para botões, aumenta muito a chance de ruído. Use apenas se necessário.

8.3 Conflito de linhas EXTI

Lembrete fundamental:

pinos com mesmo número compartilham a mesma linha EXTI (ex.: PA13 e PC13).

escolha apenas um como fonte daquela linha.

---

9. Encerramento

GPIO + EXTI é o passo natural para sair do “loop lendo botão” e passar para um sistema reativo e eficiente. O ponto crítico, em aplicações reais, é debounce e tratamento rápido na interrupção.

Capítulo 1.4 — GPIO + EXTI com múltiplos botões (arquitetura organizada e exemplo testável)

1. Motivação

Em projetos reais, raramente existe apenas um botão. O mais comum é ter:

2 a 6 teclas de navegação (OK, ESC, UP/DOWN, etc.)

entradas digitais de falha (fault), sensores de porta, ignição, presença

sinais de "evento" (pulso curto) onde polling é inadequado

O desafio não é apenas "habilitar EXTI", e sim organizar o código para que:

seja fácil adicionar/remover botões,

cada botão tenha seu próprio debounce,

o tratamento seja rápido na interrupção,

o programa principal (ou tarefas) processe eventos de forma limpa.

Este capítulo apresenta um modelo robusto e didático.

---

2. Relembrando a regra de hardware mais importante

As linhas EXTI para GPIO são EXTI0..EXTI15, e cada uma corresponde ao número do pino.

Isso implica:

PC13 usa EXTI13

PA0 usa EXTI0

PB7 usa EXTI7

E também:

PA13, PB13, PC13… compartilham a mesma linha EXTI13

(você escolhe qual porta alimenta a linha).

Essa regra influencia o projeto: evite escolher, por exemplo, dois botões em PA13 e PC13, pois ambos tentariam usar EXTI13.

---

3. Estrutura recomendada: "driver" simples de botões (somente GPIO+EXTI)

A proposta é dividir em duas camadas:

Camada de interrupção (rápida)

faz debounce por tempo

confirma o nível (opcional, mas recomendado)

registra “evento pendente” (flag)

Camada de processamento (fora da interrupção)

consulta quais eventos ocorreram

executa ações (trocar tela, togglar LED, etc.)

Isso mantém a ISR curta, previsível e escalável.

---

4. Exemplo testável (3 botões) com debounce independente

4.1 Convenção elétrica (padrão recomendado)

Para simplificar e padronizar:

botões ligados ao GND

pino com pull-up

botão pressionado = nível 0 (ativo em baixo)

interrupção na borda de descida (press)

Isso reduz ruído e torna o comportamento uniforme.

4.2 Definição de um “mapa” de botões

Você define uma tabela com:

porta

pino

ID do botão

tempo de debounce

estado/flags

Exemplo (em buttons.c, ou no próprio main.c para fins didáticos):

```
#include "stm32g0xx_hal.h"
```

```
typedef enum
{
    BTN_OK = 0,
    BTN_ESC,
    BTN_OUT,
    BTN_COUNT
} ButtonId;

typedef struct
{
    GPIO_TypeDef *port;
    uint16_t      pin;
    uint32_t      debounce_ms;
    uint32_t      last_tick;
    volatile uint8_t pending;   // evento pendente (pressionado confirmado)
} Button;

#define DEFAULT_DEBOUNCE_MS  50

static Button g_buttons[BTN_COUNT] =
{
    [BTN_OK]  = { .port = GPIOC, .pin = GPIO_PIN_13, .debounce_ms =
DEFAULT_DEBOUNCE_MS, .last_tick = 0, .pending = 0 },
    [BTN_ESC] = { .port = GPIOB, .pin = GPIO_PIN_7,  .debounce_ms =
DEFAULT_DEBOUNCE_MS, .last_tick = 0, .pending = 0 },
    [BTN_OUT] = { .port = GPIOA, .pin = GPIO_PIN_0,  .debounce_ms =
DEFAULT_DEBOUNCE_MS, .last_tick = 0, .pending = 0 },
};
```

Observação: os pinos acima são apenas exemplo "de livro". No seu hardware real, você substitui pelos pinos corretos e garante que não existe conflito de linha EXTI (mesmo número de pino repetido em ports diferentes).

4.3 Inicialização GPIO/EXTI (CubeMX)

No CubeMX, para cada botão:

Mode: GPIO_EXTI (por exemplo Falling edge trigger)

Pull: Pull-up

NVIC: habilitar IRQ correspondente ao grupo do pino

O CubeMX vai gerar o MX_GPIO_Init() com algo equivalente a:

```
// Exemplo ilustrativo para um pino:
GPIO_InitStruct.Pin  = GPIO_PIN_13;
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
```

E habilitar IRQs do tipo:

EXTI0_1_IRQn, EXTI2_3_IRQn, EXTI4_15_IRQn (varia por família/linhas)

4.4 ISRs (handlers) chamando o HAL

O HAL exige que a ISR chame HAL_GPIO_EXTI_IRQHandler(pin) para cada pino que aquela IRQ pode atender.

Exemplo típico quando vários pinos estão em EXTI4..15:

```
void EXTI4_15_IRQHandler(void)
{
   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_7);
}
```

E se houver um botão em EXTI0:

```
void EXTI0_1_IRQHandler(void)
```

```
{
   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}
```

Nota de projeto: o CubeMX pode gerar parte disso; em alguns templates você precisa completar manualmente. Sempre mantenha isso dentro de /* USER CODE BEGIN */.

4.5 Callback único para todos os botões (com debounce e confirmação)

Aqui está o núcleo do modelo:

```
static inline Button* button_find_by_pin(uint16_t gpio_pin)
{
   for (int i = 0; i < BTN_COUNT; i++)
   {
      if (g_buttons[i].pin == gpio_pin)
         return &g_buttons[i];
   }
   return NULL;
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
   Button *b = button_find_by_pin(GPIO_Pin);
   if (b == NULL)
      return;

   uint32_t now = HAL_GetTick();

   // 1) Debounce por tempo
   if ((now - b->last_tick) < b->debounce_ms)
```

```
        return;

    b->last_tick = now;

    // 2) Confirma nível (ativo em baixo: pressionado = RESET)
    if (HAL_GPIO_ReadPin(b->port, b->pin) == GPIO_PIN_RESET)
    {
        b->pending = 1; // registra evento: "pressionado confirmado"
    }
}
```

Características importantes:

- O callback continua rápido.
- Debounce é independente por botão.
- Há confirmação de nível, reduzindo falsos positivos por ruído.

4.6 “Leitura” dos eventos fora da interrupção

A aplicação não deve depender de executar lógica dentro do callback. Em vez disso, ela “consome” eventos:

```
static uint8_t button_pop_event(ButtonId id)
{
    if (g_buttons[id].pending)
    {
        g_buttons[id].pending = 0;
        return 1;
    }
    return 0;
}
```

4.7 Exemplo testável no while(1)

Agora um exemplo simples e verificável:

BTN_OK: alterna LED

BTN_ESC: alterna LED (ou outro)

BTN_OUT: alterna LED

```
while (1)
{
  if (button_pop_event(BTN_OK))
  {
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
  }

  if (button_pop_event(BTN_ESC))
  {
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
  }

  if (button_pop_event(BTN_OUT))
  {
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
  }
}
```

Você pode substituir o toggle por:

alterar variável de estado (menu, tela, modo),

incrementar contador,

sinalizar uma tarefa (se houver RTOS, mais adiante no livro).

---

5. Extensões úteis (ainda apenas GPIO + EXTI)

5.1 Detectar “soltar o botão” (release)

Para detectar eventos de pressionar e soltar:

configure EXTI em RISING_FALLING

no callback, verifique o nível e gere eventos diferentes (PRESS/RELEASE)

Mas para botões mecânicos, isso exige cuidado maior com debounce (idealmente debounce com máquina de estados).

5.2 Pressão longa (long press)

Sem entrar em timers periféricos, dá para fazer com HAL_GetTick():

no PRESS, salva press_tick

no loop principal, se o botão continua pressionado e passou X ms, gera evento de “long press”

Isso mantém o EXTI apenas como “gatilho” e o restante como lógica de aplicação.

---

6. Boas práticas finais

Uma ação = um evento: debounce garante que um clique não vira vários eventos.

Não faça trabalho pesado na ISR: apenas registre evento.

Padronize a elétrica (pull-up + GND): reduz problemas e simplifica software.

Evite conflitos de linha EXTI: não repita o mesmo número de pino em ports diferentes para sinais que precisam de EXTI.

Capítulo 1.5 — Debounce avançado com máquina de estados (PRESS, RELEASE, LONG PRESS, DOUBLE CLICK)

1. Objetivo

O modelo anterior (debounce por janela de tempo) resolve a maioria dos casos de "um clique = um evento". Porém, interfaces reais frequentemente precisam de:

PRESS (pressionou)

RELEASE (soltou)

LONG PRESS (segurou por X ms)

DOUBLE CLICK (dois cliques em uma janela)

Fazer isso de forma confiável requer algo mais estruturado: uma máquina de estados por botão.

A regra principal continua a mesma:

A interrupção (EXTI) deve ser mínima.

Ela apenas "acorda" o botão e registra a hora do evento.

A validação e a interpretação (press/long/double) acontece fora da ISR.

---

2. Conceito de "estado estável" vs "transição ruidosa"

O botão físico tem dois estados reais:

solto (nível 1, pull-up)

pressionado (nível 0, ligado ao GND)

Mas durante a troca de estado existem alguns milissegundos de instabilidade (bouncing).

A máquina de estados separa claramente:

momento de transição (instável → precisa esperar estabilizar)

momento estável (pode gerar eventos confiáveis)

---

3. Estratégia adotada (robusta e didática)

Vamos usar três ideias simples:

EXTI em ambas as bordas (RISING_FALLING)

Para “acordar” a lógica em press e release.

Tempo de debounce (ex.: 30–50 ms)

Após uma borda, não confiamos imediatamente no nível. Esperamos estabilizar.

Processamento periódico (poll controlado)

Uma função buttons_process() é chamada no while(1) (ou em uma tarefa) e decide:

quando o estado estabilizou,

quando emitir PRESS/RELEASE,

quando emitir LONG PRESS,

quando fechar janela de DOUBLE CLICK.

Isso mantém o sistema determinístico e “testável”.

---

4. Definições e eventos

4.1 Eventos possíveis

```
typedef enum
{
   BTN_EVT_NONE = 0,
   BTN_EVT_PRESS,
   BTN_EVT_RELEASE,
   BTN_EVT_LONG_PRESS,
   BTN_EVT_DOUBLE_CLICK
} ButtonEvent;
```

4.2 Parâmetros típicos

DEBOUNCE_MS: 40 a 60 ms (botão mecânico comum)

LONG_PRESS_MS: 600 a 1200 ms (depende da UX)

DOUBLE_CLICK_MS: 250 a 450 ms (depende da UX)

Para o livro, usaremos valores bem “padrão”:

```
#define BTN_DEBOUNCE_MS      50
#define BTN_LONG_PRESS_MS    800
#define BTN_DBLCLICK_MS      350
```

---

5. Máquina de estados por botão

5.1 Estados

```
typedef enum
{
   BS_UP_STABLE = 0,      // solto estável
   BS_DEBOUNCE_DOWN,      // esperando estabilizar após borda para baixo
   BS_DOWN_STABLE,        // pressionado estável
   BS_DEBOUNCE_UP,        // esperando estabilizar após borda para cima
   BS_WAIT_SECOND_CLICK   // após 1º clique, aguardando 2º clique
} ButtonState;
```

5.2 Estrutura do botão

```
typedef struct
{
   GPIO_TypeDef *port;
   uint16_t      pin;

   ButtonState   st;

   uint32_t      t_edge;       // quando ocorreu a última borda (EXTI)
   uint32_t      t_press;      // quando confirmou PRESS estável
   uint32_t      t_release;    // quando confirmou RELEASE estável
   uint32_t      t_first_click; // usado para double-click

   uint8_t       long_sent;    // evita repetir LONG_PRESS
```

```
    volatile uint8_t edge_seen;  // marcado pela ISR quando há borda

    // fila simples de eventos (1 slot por simplicidade didática)
    volatile ButtonEvent pending_evt;
} ButtonEx;
```

Por que edge_seen?

Porque a ISR apenas sinaliza “houve borda”, e a lógica decide o que fazer.

---

6. Configuração CubeMX (por botão)

Para cada botão:

- Mode: GPIO_EXTI
- Trigger: Rising/Falling
- Pull: Pull-up (padrão do nosso exemplo)
- NVIC: habilitar IRQ do grupo correspondente

---

7. ISR mínima + Callback

A ISR chama o HAL e o HAL chama o callback. No callback:

- localizar o botão pelo GPIO_Pin
- salvar timestamp
- marcar edge_seen = 1

```
static ButtonEx *button_find_by_pin(uint16_t gpio_pin);

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    ButtonEx *b = button_find_by_pin(GPIO_Pin);
    if (!b) return;

    b->t_edge = HAL_GetTick();
```

```
    b->edge_seen = 1; // apenas sinaliza
}
```

Importante: não fazemos debounce nem geramos PRESS/RELEASE aqui.

---

8. Função de processamento: buttons_process()

Essa função deve ser chamada frequentemente (ex.: a cada 1–5 ms no loop, ou com HAL_Delay(1)), e é ela que emite eventos.

8.1 Leitura "ativa em baixo"

```
static inline uint8_t button_is_down(ButtonEx *b)
{
    return (HAL_GPIO_ReadPin(b->port, b->pin) == GPIO_PIN_RESET);
}
```

8.2 Emissão de evento (1 slot)

```
static inline void button_emit(ButtonEx *b, ButtonEvent evt)
{
    if (b->pending_evt == BTN_EVT_NONE)
        b->pending_evt = evt;
}
```

8.3 Processamento do estado

```
static void button_process_one(ButtonEx *b, uint32_t now)
{
    switch (b->st)
    {
        case BS_UP_STABLE:
        {
            if (b->edge_seen)
            {
                b->edge_seen = 0;
```

```
        b->st = BS_DEBOUNCE_DOWN;
    }
} break;

case BS_DEBOUNCE_DOWN:
{
    if ((now - b->t_edge) >= BTN_DEBOUNCE_MS)
    {
        if (button_is_down(b))
        {
            b->st = BS_DOWN_STABLE;
            b->t_press = now;
            b->long_sent = 0;
            button_emit(b, BTN_EVT_PRESS);
        }
        else
        {
            // borda falsa / ruído
            b->st = BS_UP_STABLE;
        }
    }
} break;

case BS_DOWN_STABLE:
{
    // LONG PRESS
    if (!b->long_sent && (now - b->t_press) >= BTN_LONG_PRESS_MS)
    {
```

```
        b->long_sent = 1;
        button_emit(b, BTN_EVT_LONG_PRESS);
    }

    if (b->edge_seen)
    {
        b->edge_seen = 0;
        b->st = BS_DEBOUNCE_UP;
    }
} break;

case BS_DEBOUNCE_UP:
{
    if ((now - b->t_edge) >= BTN_DEBOUNCE_MS)
    {
        if (!button_is_down(b))
        {
            b->st = BS_WAIT_SECOND_CLICK;
            b->t_release = now;
            b->t_first_click = now;
            button_emit(b, BTN_EVT_RELEASE);
        }
        else
        {
            // ainda está pressionado (bouncing / contato ruim)
            b->st = BS_DOWN_STABLE;
        }
    }
```

```
        } break;

        case BS_WAIT_SECOND_CLICK:
        {
            // Se houver uma nova borda dentro da janela, tentamos confirmar 2º clique
            if (b->edge_seen)
            {
                b->edge_seen = 0;
                b->st = BS_DEBOUNCE_DOWN;
                // O double-click será confirmado quando o PRESS estabilizar,
                // porém precisamos marcar que estamos na janela do 2º clique.
                // Para simplificar, confirmamos no momento do 2º PRESS estável:
                // se (now - t_first_click) <= BTN_DBLCLICK_MS => DOUBLE_CLICK.
            }

            // Se o tempo estourou, volta ao estado solto normal
            if ((now - b->t_first_click) > BTN_DBLCLICK_MS)
            {
                b->st = BS_UP_STABLE;
            }
        } break;

        default:
            b->st = BS_UP_STABLE;
            break;
    }
}
```

8.4 Ajuste para confirmar DOUBLE CLICK (no PRESS do 2º clique)

No estado BS_DEBOUNCE_DOWN, quando confirmar o PRESS, adicionamos:

se estamos vindo de BS_WAIT_SECOND_CLICK (ou se now - t_first_click ainda está na janela), emitimos DOUBLE_CLICK.

Para manter simples sem variável extra, usamos t_first_click != 0 e janela:

Substitua o trecho do PRESS confirmado em BS_DEBOUNCE_DOWN por:

```
if (button_is_down(b))
{
   b->st = BS_DOWN_STABLE;
   b->t_press = now;
   b->long_sent = 0;
   // Se está dentro da janela desde o último release, considera double click
   if (b->t_first_click != 0 && (now - b->t_first_click) <= BTN_DBLCLICK_MS)
   {
      button_emit(b, BTN_EVT_DOUBLE_CLICK);
      b->t_first_click = 0; // consome a janela
   }
   else
   {
      button_emit(b, BTN_EVT_PRESS);
   }
}
```

E no final de BS_WAIT_SECOND_CLICK, quando estourar o tempo:

```
if ((now - b->t_first_click) > BTN_DBLCLICK_MS)
{
   b->t_first_click = 0;
   b->st = BS_UP_STABLE;
}
```

9. API de consumo de eventos (fora da ISR)

A aplicação “puxa” o evento quando quiser:

```
static ButtonEvent button_pop(ButtonEx *b)
{
   ButtonEvent e = b->pending_evt;
   b->pending_evt = BTN_EVT_NONE;
   return e;
}
```

---

10. Exemplo testável completo (conceito)

10.1 No while(1)

chama buttons_process() continuamente

consome eventos e executa ações simples (LED, contadores)

```
static void buttons_process_all(void)
{
   uint32_t now = HAL_GetTick();

   for (int i = 0; i < BTN_COUNT; i++)
      button_process_one(&g_buttons[i], now);
}

while (1)
{
   buttons_process_all();

   ButtonEvent e;

   e = button_pop(&g_buttons[BTN_OK]);
```

```
  if (e == BTN_EVT_PRESS)        { /* ação OK press */ }
  if (e == BTN_EVT_LONG_PRESS)    { /* ação OK long */ }
  if (e == BTN_EVT_DOUBLE_CLICK)  { /* ação OK double */ }

  e = button_pop(&g_buttons[BTN_ESC]);
  if (e == BTN_EVT_PRESS)        { /* ação ESC press */ }

  e = button_pop(&g_buttons[BTN_OUT]);
  if (e == BTN_EVT_PRESS)        { /* ação OUT press */ }

  // pequena espera opcional para reduzir CPU (depende do projeto)
  // HAL_Delay(1);
}
```

10.2 Como testar (procedimento)

Pressione e solte rapidamente: deve gerar PRESS e RELEASE.

Segure por ~0,8 s: deve gerar PRESS e depois LONG_PRESS.

Dê dois cliques rápidos: deve gerar DOUBLE_CLICK (ao confirmar o 2º press).

---

11. Considerações importantes (qualidade e confiabilidade)

11.1 Por que "RISING_FALLING" não explode de bouncing aqui?

Porque:

bordas disparam edge_seen,

mas a máquina de estados só aceita mudança após BTN_DEBOUNCE_MS e com confirmação do nível.

Ou seja: bordas extras viram "sinais", mas não viram evento final.

11.2 Fila de eventos

Para fins didáticos usamos "1 slot". Em firmware real, você pode:

usar bitmask de eventos,

ou uma fila circular de eventos por botão,

ou enviar para uma fila do RTOS.

A arquitetura permanece a mesma.

11.3 Uso com RTOS

Mesmo com RTOS, a ideia é idêntica:

ISR sinaliza (ex.: edge_seen)

uma tarefa periódica executa buttons_process_all()

eventos vão para fila/dispatcher

---

12. Encerramento

Com esta máquina de estados, o GPIO + EXTI deixa de ser apenas “interrupção de botão” e passa a fornecer uma camada confiável de eventos de interface (press, release, long, double), mantendo:

ISR curta,

comportamento determinístico,

código organizado e escalável.

Capítulo 2 — DMA no STM32G070RB (conceito, hardware, vantagens e inicialização básica)

1. O que é DMA

DMA (Direct Memory Access) é um periférico do microcontrolador projetado para mover dados entre memória e periféricos (ou memória↔memória) sem que a CPU precise copiar byte a byte.

Em vez da CPU executar um loop do tipo "ler registrador → escrever RAM → repetir", o DMA faz isso sozinho, em hardware, enquanto a CPU fica livre para:

processar dados,

atualizar interface,

executar controle,

dormir (reduzir consumo),

atender outras interrupções.

Ideia central:

DMA é um "motor de cópia" configurável, acionado por eventos, que transfere blocos de dados com baixa sobrecarga de CPU.

---

2. Para que o DMA foi feito (motivação)

Sem DMA, transferências rápidas e contínuas viram um gargalo:

ADC amostrando centenas/milhares de amostras por segundo

UART recebendo streams

SPI lendo sensores com bursts

I^2C recebendo blocos

TIM capturando/gerando padrões

Nesses casos, a CPU seria "escrava" do fluxo de dados, gastando tempo com cópia e perdendo prazos.

O DMA resolve exatamente isso: tirar a CPU do caminho da transferência.

---

3. Como o DMA se encaixa na arquitetura (visão de hardware)

3.1 Elementos fundamentais

Um DMA típico no STM32 possui:

Controlador DMA: o bloco que gerencia transferências.

Canais (channels): “linhas” de transferência configuráveis.

Cada canal pode ser associado a uma fonte de requisição (request).

Requests (requisições): sinais gerados por periféricos (ADC, USART RX/TX, SPI RX/TX etc.) pedindo “mais uma transferência”.

Arbitragem: quando vários canais pedem ao mesmo tempo, o DMA decide quem atende primeiro (prioridade).

No STM32G0, o mapeamento “periférico → canal/request” é definido pela família (CubeMX cuida disso na prática), mas o conceito é fixo: um periférico gera um evento, esse evento dispara um canal DMA, que move 1 unidade (byte/halfword/word) por evento.

3.2 Unidades de transferência

Cada “passo” do DMA move uma unidade configurada:

8 bits (byte)

16 bits (halfword)

32 bits (word)

Isso precisa bater com:

o registrador do periférico (ex.: ADC geralmente 16 bits)

o buffer em memória

3.3 Incremento e modos

O DMA pode:

incrementar endereço de memória (para preencher um buffer)

não incrementar endereço do periférico (registrador fixo)

operar em normal mode (para após N transferências)

operar em circular mode (reinicia e continua, ideal para streaming)

---

4. Modos de uso mais comuns (sem amarrar a um periférico ainda)

Periférico → Memória

Ex.: ADC para buffer; UART RX para ring buffer.

Memória → Periférico

Ex.: UART TX enviando um bloco; SPI TX enviando uma trama.

Memória ↔ Memória (quando suportado/permitido pela família/config)

Útil para cópia de blocos, mas muitas vezes o ganho real depende do barramento e do custo de setup.

---

5. Vantagens (por que usar DMA)

Reduz carga da CPU drasticamente em transferências repetitivas.

Maior throughput em streaming (especialmente com circular).

Menos jitter: CPU não fica interrompendo processamento para copiar dados.

Melhor consumo: CPU pode dormir enquanto DMA trabalha (dependendo do modo).

Arquitetura mais limpa: "quem move dados" (DMA) separado de "quem processa" (CPU).

---

6. Desvantagens e armadilhas (o lado que dá bugs)

Complexidade: tem mais estados e sinais envolvidos.

Debug mais chato: o dado "aparece" no buffer sem passar por um loop visível.

Condições de corrida: CPU e DMA podem acessar o mesmo buffer.

Gerenciamento de buffer: principalmente em modo circular (metade cheia, cheia, overrun).

Eventos/IRQs: você precisa decidir se vai usar interrupção de "transfer complete", "half transfer", erros, etc.

Latência: DMA é excelente para throughput, mas a reação a cada byte pode ser pior do que uma ISR se você precisa tratar byte a byte (raramente é o caso ideal).

---

7. Conceitos que o leitor precisa dominar antes de usar DMA com periféricos

Para virar "especialista" e não só copiar exemplo, o leitor precisa ter clareza sobre:

7.1 Quem "marca o ritmo" da transferência?

Em transferências com periférico, o periférico gera a requisição.

Ex.: UART RX gera request quando chega dado; ADC gera request quando conversão termina.

O DMA não “inventa dados”; ele responde a eventos.

7.2 Onde o dado “vive”?

O DMA move para um buffer em RAM.

A CPU processa esse buffer — mas deve saber:

quando ele está válido (half/full transfer, flags)

o que acontece se o DMA continuar escrevendo (circular)

7.3 Normal vs Circular

Normal: melhor para “transação com tamanho conhecido” (ex.: ler 32 bytes).

Circular: melhor para “stream contínuo” (ex.: ADC contínuo, UART contínua).

7.4 Interrupções do DMA

O DMA pode gerar interrupções por:

Transfer Complete (TC): terminou N itens.

Half Transfer (HT): atingiu metade do buffer (muito útil em circular).

Error (TE): erro de transferência (config, barramento, etc.).

Essas interrupções são a “ponte” entre o mundo DMA e a lógica da aplicação.

---

8. Inicialização do DMA no HAL (sem pendurar em periférico)

8.1 O que significa “inicializar DMA” isoladamente

No HAL, “inicializar DMA” sem periférico significa:

habilitar clock do controlador DMA,

configurar NVIC para as interrupções do DMA (se você pretende usá-las),

(opcional) preparar handles DMA_HandleTypeDef (normalmente feito quando você inicializa um periférico específico).

Ou seja: a configuração completa do canal (direção, endereços, tamanho, request) normalmente só faz sentido quando existe um periférico associado. Mas a base (clock + NVIC + infraestrutura) pode e deve ser introduzida antes.

8.2 Exemplo mínimo: habilitar clock + NVIC do DMA

Observação: o nome exato das IRQs e do clock depende do STM32G0 e do template do CubeMX, mas o padrão é sempre "habilitar DMA e habilitar as IRQs correspondentes".

Exemplo típico:

```
static void MX_DMA_Init(void)
{
  /* 1) Habilita clock do DMA */
  __HAL_RCC_DMA1_CLK_ENABLE();

  /* 2) Configura interrupções do DMA (opcional, mas recomendado para exemplos futuros) */
  HAL_NVIC_SetPriority(DMA1_Channel1_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(DMA1_Channel1_IRQn);

  HAL_NVIC_SetPriority(DMA1_Channel2_3_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(DMA1_Channel2_3_IRQn);

  HAL_NVIC_SetPriority(DMA1_Channel4_5_6_7_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(DMA1_Channel4_5_6_7_IRQn);
}
```

E no main():

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
```

```
    MX_DMA_Init();  // inicialização “base” do DMA

    while (1)
    {
    }
}
```

Por que já configurar NVIC aqui?

Porque nos próximos capítulos (ADC, UART, SPI…) o DMA vai gerar eventos (TC/HT/TE). Deixar o NVIC pronto facilita a progressão didática.

8.3 Handlers (ISRs) do DMA — “casca” pronta para o futuro

Sem amarrar a um periférico, ainda não existe DMA_HandleTypeDef conectado. Mesmo assim, é útil mostrar ao leitor a estrutura que será usada:

```
void DMA1_Channel1_IRQHandler(void)
{
    // no futuro: HAL_DMA_IRQHandler(&hdma_xxx);
}

void DMA1_Channel2_3_IRQHandler(void)
{
    // no futuro: HAL_DMA_IRQHandler(&hdma_xxx);
}

void DMA1_Channel4_5_6_7_IRQHandler(void)
{
    // no futuro: HAL_DMA_IRQHandler(&hdma_xxx);
}
```

Nos próximos capítulos, quando um periférico for inicializado com DMA, você verá algo como:

um handle hdma_adc1, hdma_usart2_rx, etc.

HAL_DMA_IRQHandler(&hdma_...) dentro dessas ISRs

callbacks do tipo HAL_ADC_ConvCpltCallback, HAL_UART_RxCpltCallback ou callbacks DMA dependendo do fluxo.

---

9. Encerramento (ponte para os próximos periféricos)

DMA é o “trilho” que vai sustentar exemplos modernos de alto desempenho:

ADC + DMA para aquisição contínua de sinais,

UART + DMA para receber streams sem travar CPU,

SPI + DMA para transferências rápidas em bursts,

e até TIM + DMA em aplicações avançadas (padrões, atualização de duty/CCR).

A partir daqui, o leitor já deve reconhecer:

o papel do DMA (motor de transferência),

por que ele existe,

como ele se integra aos periféricos,

quais são os erros clássicos (buffer/circular/concorrência),

e como o HAL “prepara o terreno” com MX_DMA_Init().

Capítulo 2.1 — DMA “por dentro”: Normal vs Circular, Half Transfer e projeto de buffers (sem depender de ADC/UART/SPI)

1. O que realmente acontece em uma transferência DMA

Uma transferência DMA não é “copiar um bloco de uma vez”, necessariamente. Na maioria dos usos com periféricos, ela funciona assim:

Um periférico gera uma requisição (request) quando:

um dado chegou (RX), ou

um dado precisa ser enviado (TX), ou

uma conversão terminou (ADC), etc.

O DMA atende essa requisição e move uma unidade (8/16/32 bits) entre:

registrador do periférico ↔ memória

O DMA atualiza internamente:

contador restante

ponteiro de memória (se incremento habilitado)

Quando o contador chega a zero:

em Normal mode: a transferência para e gera “Transfer Complete”

em Circular mode: o contador recarrega e o ponteiro volta ao início do buffer

O ponto essencial:

O periférico “marca o ritmo” e o DMA executa o movimento de dados com baixíssima sobrecarga.

---

2. Normal Mode (modo normal)

2.1 Definição

No modo normal, você configura um tamanho N e o DMA executa exatamente N transferências. Ao terminar:

seta flag de Transfer Complete (TC)

opcionalmente dispara interrupção de TC

para (não escreve mais)

2.2 Quando usar

pacotes com tamanho conhecido (ex.: comando de 32 bytes)

leituras esporádicas

envio de uma mensagem fixa

capturas “one-shot” (ex.: adquirir 128 amostras e parar)

2.3 Prós e contras

Prós

mais simples de raciocinar

“fim” de transferência bem definido

Contras

para streaming contínuo, você precisará reiniciar o DMA toda hora

reinicializações frequentes aumentam overhead e risco de perder dados

---

3. Circular Mode (modo circular)

3.1 Definição

No modo circular, o DMA é configurado com tamanho N, mas ao chegar ao fim:

ele recarrega automaticamente o contador

volta o endereço de memória ao início do buffer

continua transferindo indefinidamente

Isso implementa naturalmente um buffer circular em memória.

3.2 Quando usar

aquisição contínua (ADC contínuo)

recepção contínua (UART RX “stream”)

qualquer fluxo de dados que não tem “fim” bem definido

3.3 Prós e contras

Prós

ideal para streaming (estável e eficiente)

CPU quase não participa da transferência

reduz risco de perda por “setup repetido”

Contras

exige organização de buffer e sincronização CPU↔DMA

se a CPU não consumir rápido o suficiente, o DMA sobrescreve dados (overrun lógico do aplicativo)

---

4. Half Transfer (HT) e Transfer Complete (TC)

Em DMA, duas interrupções/flags são extremamente importantes para “streaming”:

HT (Half Transfer): o DMA atingiu a metade do buffer

TC (Transfer Complete): o DMA atingiu o final do buffer

No modo circular:

HT e TC acontecem repetidamente, em loop:

preenche metade → HT

preenche outra metade → TC

reinicia → HT → TC → ...

Isso permite uma técnica muito poderosa:

Processar dados em blocos enquanto o DMA continua preenchendo o restante do buffer.

---

5. Três modelos clássicos de buffer (e quando usar cada um)

5.1 Buffer “single-shot” (Normal)

Ideia: DMA preenche N itens e para. CPU processa depois.

simples

ótimo para capturas pontuais

Risco: nenhum streaming; se você precisar de contínuo, vai reiniciar manualmente.

5.2 Ping-Pong (Double Buffer) conceitual

Mesmo sem um recurso de “double buffer” automático no DMA específico, o conceito é:

enquanto DMA escreve no Buffer A

CPU processa o Buffer B

depois trocam

Em muitos STM32, o ping-pong é implementado na prática usando:

dois buffers e reinicialização rápida (em normal),

ou usando HT/TC em um único buffer (modelo abaixo), que costuma ser mais simples no HAL.

5.3 “Buffer único com duas metades” (Circular + HT/TC) — o mais usado

Você cria um buffer de tamanho N e pensa nele em duas metades:

primeira metade: índices [0 .. N/2 - 1]

segunda metade: índices [N/2 .. N - 1]

No modo circular:

quando HT ocorre → primeira metade está pronta (estável)

quando TC ocorre → segunda metade está pronta (estável)

Resultado: a CPU sempre processa “metades” prontas, e o DMA sempre escreve na outra metade.

Esse modelo é extremamente popular porque:

não precisa parar o DMA

não precisa trocar ponteiros manualmente

a sincronização é simples (evento HT/TC)

---

6. Overrun do ponto de vista do software (o “perigo invisível”)

Em circular mode, o DMA nunca para. Portanto existe um limite real:

se a CPU demora mais do que “um ciclo de buffer” para processar,

o DMA volta ao início e sobrescreve dados ainda não consumidos

Isso não é “erro de hardware” necessariamente, é um erro de dimensionamento e arquitetura.

6.1 Como evitar

aumentar o buffer N

processar mais rápido (otimizar algoritmo)

reduzir a taxa de dados (ex.: sampling rate)

processar em blocos menores (usar HT/TC em vez de esperar buffer inteiro)

mover processamento pesado para momentos de menor carga

em RTOS: priorizar a tarefa de consumo / usar filas e sinalizações eficientes

---

7. Concorrência CPU ↔ DMA (boas práticas que evitam bugs)

DMA escreve na memória ao mesmo tempo em que a CPU pode ler. Então a regra de ouro é:

A CPU só deve ler regiões do buffer que estejam “finalizadas” (HT/TC) ou protegidas por uma lógica de ponteiros/estado.

Modelos típicos:

HT/TC definem regiões seguras

flags “metade pronta” e “metade consumida”

índices de leitura/escrita (ring buffer lógico) em casos avançados

Mesmo sem cache em muitos Cortex-M0+, a concorrência ainda existe (não é problema de cache; é problema de acesso simultâneo e consistência de “o que está pronto”).

---

8. Mini-exemplo didático (sem periférico real): como pensar HT/TC

Imagine um buffer de 8 amostras:

buf[0 1 2 3 | 4 5 6 7]

HT TC

Quando HT dispara: {0..3} está completo e não será mais modificado até o DMA avançar para a segunda metade.

Quando TC dispara: {4..7} está completo e o DMA vai reiniciar no índice 0.

O processamento ideal:

em HT → processa 0..3

em TC → processa 4..7

Assim a CPU nunca “briga” com o DMA pela mesma metade.

---

9. Como o HAL normalmente expõe isso (visão para os próximos capítulos)

Sem amarrar em um periférico agora, o que importa é o padrão que você verá depois:

interrupções do DMA chamam HAL_DMA_IRQHandler(&hdma_xxx)

callbacks do periférico ou do DMA disparam:

Half complete

Complete

Error

E você terá pontos de gancho claros para:

“metade pronta”

“buffer cheio”

“erro”

O leitor deve enxergar desde já que:

DMA é transferência

HT/TC são sincronização

Buffer é arquitetura

---

10. Encerramento e ponte direta para ADC/UART/SPI com DMA

A partir daqui, o leitor já está preparado para entender qualquer exemplo com DMA, porque sabe:

diferença entre Normal e Circular

para que servem HT e TC

como organizar buffers de forma segura

como e por que ocorre overrun

como pensar em throughput e consumo de dados

No próximo periférico que usar DMA (por exemplo, ADC com buffer de amostras), nós vamos apenas “plugar” esses conceitos em:

taxa de amostragem,

tamanho do buffer,

HT/TC como gatilhos de processamento,

e estratégia de consumo dos dados.

Capítulo 2.2 — DMA: prioridades, erros e diagnóstico rápido

1. Prioridades no DMA (por que existem)

Quando mais de um canal DMA está ativo, pode acontecer de dois (ou mais) pedirem acesso ao barramento ao mesmo tempo. O controlador DMA então usa prioridades para decidir qual canal é atendido primeiro.

Na prática, prioridade serve para:

garantir que fluxos “críticos” (ex.: RX contínuo) não sejam atrasados por fluxos menos críticos (ex.: TX ocasional);

reduzir risco de perda de dados quando o sistema está carregado.

Regra prática para o livro:

RX contínuo (stream) costuma ter prioridade maior que TX.

Aquisição (ADC) costuma ter prioridade alta quando a taxa é elevada.

---

2. Tipos de “erro” que aparecem com DMA

Em DMA, os erros mais relevantes (conceitualmente) são:

TE (Transfer Error): falha de transferência (configuração inválida, endereço, acesso, etc.).

Erro de periférico (não é do DMA em si): overrun/underrun, framing, etc.

O DMA “só” move dados; o periférico pode sinalizar que dados foram perdidos.

Além disso, existe um “erro silencioso” muito comum:

Overrun lógico do aplicativo em modo circular: CPU não consumiu a tempo e o DMA sobrescreveu dados.

Isso não necessariamente gera TE; é um problema de dimensionamento/arquitetura.

---

3. Checklist de diagnóstico (o que checar quando "não funciona")

A) Nada é transferido

clock do DMA habilitado?

clock do periférico habilitado?

canal/request correto (CubeMX geralmente acerta)

direção correta (P→M ou M→P)

tamanho de dado (8/16/32) compatível com periférico e buffer

incremento de memória habilitado (para preencher buffer)

B) Transfere, mas dados parecem "lixo"

alinhamento e tamanho (ex.: ADC 16-bit em buffer de 8-bit)

buffer pequeno demais e sendo sobrescrito

leitura da CPU pegando região ainda "em escrita" (não usar HT/TC)

C) Funciona por um tempo e "para"

em Normal mode, você esqueceu de reiniciar a transferência (isso é esperado).

em Circular mode, pode ser:

CPU não consumindo (overrun lógico),

interrupções HT/TC não habilitadas ou não tratadas,

prioridade/latência de ISR impedindo consumo no tempo.

---

4. Diagnóstico prático com HT/TC (regra simples)

Para streaming com circular:

habilite HT e TC

use HT/TC para marcar "metade pronta"

processe a metade pronta rápido

Se você não processa por HT/TC e espera “buffer inteiro”, a chance de atraso e sobreposição aumenta.

---

5. Erros e callbacks no HAL (o padrão que você verá depois)

Sem prender em periférico agora, o padrão será sempre este:

a IRQ do DMA chama HAL_DMA_IRQHandler(&hdma_xxx);

o HAL sinaliza:

Half complete

Complete

Error

No seu código, o diagnóstico típico é:

contar quantas vezes HT/TC chegam (contadores)

acender um LED em erro

registrar flags/estados

Isso dá visibilidade rápida sem precisar de printf dentro de interrupção.

---

6. Fechamento

DMA raramente “falha sozinho”: quando algo dá errado, quase sempre é:

configuração (direção, tamanho, request),

sincronização (CPU lendo enquanto DMA escreve),

ou capacidade (buffer pequeno / consumo lento).

Com este checklist, o leitor já consegue depurar 90% dos problemas de DMA que aparecem quando começarmos ADC/UART/SPI com DMA.

Capítulo 3 — ADC no STM32G070RB (hardware, configuração e leituras com HAL: polling, interrupção e DMA)

1. O que é o ADC e para que serve

O ADC (Analog-to-Digital Converter) é o periférico responsável por converter uma tensão analógica em um número digital, permitindo que o firmware “enxergue” sinais como sensores, divisores resistivos, shunts amplificados, potenciômetros, etc.

No STM32G070RB, o ADC:

suporta múltiplos canais externos e também canais internos (como sensor de temperatura, referência interna de tensão e VBAT), conforme o datasheet da família STM32G0.

pode operar em conversão única (single-shot) ou em varredura (scan) de um grupo de canais, e pode ser atendido por DMA.

possui clock próprio e consegue taxas altas de amostragem (ordem de MSps), além de recursos de economia (auto-shutdown).

---

2. Como o hardware do ADC funciona

Apesar da implementação interna ser complexa, o ADC pode ser entendido com três blocos:

Multiplexador de entrada (MUX)

Seleciona qual canal (pino analógico) será conectado ao conversor naquele instante.

Sample-and-Hold (S/H)

Um capacitor interno “amostra” a tensão do canal durante um tempo configurável (sampling time).

Esse tempo é crítico para precisão, principalmente quando a fonte do sinal tem alta impedância.

Conversor + registrador de dados

Após a amostragem, o ADC converte e deposita o resultado em um registrador (de onde o HAL lê).

Conceito-chave: o sampling time precisa ser suficiente para o capacitor interno carregar até a tensão real do sinal. Se não, o valor fica “puxado” e instável, principalmente em sinais com divisor alto, sensores fracos e filtros RC.

---

3. Referência de tensão (VREF) e alimentação analógica (VDDA)

A conversão do ADC é sempre relativa a uma referência: em termos práticos:

"ADC_counts"≈V_in/V_ref ·(2^N-1)

VDDA/VSSA alimentam o domínio analógico; ruído em VDDA vira ruído de medição.

Alguns STM32G0 suportam VREFBUF (buffer de referência interna), com requisitos e limitações (ex.: tensão mínima em VDDA para certos níveis e corrente máxima disponível no pino VREF+ em certas condições).

Recomendação prática (manual de projeto):

desacoplar VDDA com capacitor dedicado (bem próximo ao pino);

se usar VREFBUF, seguir o treinamento/datasheet da ST;

para medições “sérias”, medir/estimar Vref via VREFINT (canal interno) e compensar em software (quando aplicável).

---

4. Calibração do ADC (por que existe e quando fazer)

A calibração ajusta erros internos do ADC (offset/ganho internos) e normalmente deve ser feita:

no boot (após inicializar clocks e antes de iniciar conversões contínuas),

e sempre que o projeto exigir melhor repetibilidade.

No STM32G0, o fluxo e detalhes de calibração e estados do ADC são documentados no reference manual (RM0444) e são expostos no HAL por funções “Ex” (extension).

Além disso, há discussões públicas de comportamento/edge cases da calibração na família G0 em issues do pacote STM32CubeG0, o que reforça a importância de seguir o fluxo recomendado e testar.

---

5. Configuração do ADC no CubeMX (o que realmente importa)

No CubeMX, os campos mais importantes para o leitor dominar são:

Resolution: geralmente 12 bits (trade-off: ruído vs. velocidade).

Data Alignment: right (mais comum).

Sampling Time: ajuste conforme impedância da fonte e ruído.

Single / Continuous Conversion:

single-shot: converte quando você manda

continuous: converte continuamente

Scan Conversion Mode: para múltiplos canais (sequência).

EOC Selection: comportamento do “end of conversion” (impacta interrupção/leituras).

DMA Continuous Requests (quando usar DMA em fluxo contínuo).

Regra didática: primeiro garanta um canal funcionando em polling, depois adicione interrupção, e só então DMA.

---

6. Leituras com HAL (do simples ao avançado)

6.1 Leitura simples (Polling) — “começando certo”

Quando usar: leituras esporádicas, debug, baixa taxa.

```
uint16_t adc_read_polling(ADC_HandleTypeDef *hadc)
{
  HAL_ADC_Start(hadc);
  HAL_ADC_PollForConversion(hadc, 10);        // timeout em ms
  uint16_t v = (uint16_t)HAL_ADC_GetValue(hadc);
  HAL_ADC_Stop(hadc);
  return v;
}
```

Pontos de atenção:

em single-shot, cada Start() gera uma conversão;

use timeout para evitar travamentos se algo estiver mal configurado.

6.2 Conversão contínua com polling (útil para testes rápidos)

Quando usar: validar ruído/estabilidade sem DMA ainda.

habilite Continuous Conversion no CubeMX

faça HAL_ADC_Start() uma vez

faça polling repetido:

```
HAL_ADC_Start(&hadc1);

while (1)
{
   if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
   {
      uint16_t v = (uint16_t)HAL_ADC_GetValue(&hadc1);
      // processa v
   }
}
```

6.3 Interrupção (IT) — leitura "event-driven"

Quando usar: quando você quer reagir ao fim da conversão sem ficar travando em polling.

```
volatile uint16_t adc_last = 0;
volatile uint8_t  adc_ready = 0;

void adc_start_it(void)
{
   adc_ready = 0;
   HAL_ADC_Start_IT(&hadc1);
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
   if (hadc->Instance == ADC1)
   {
      adc_last = (uint16_t)HAL_ADC_GetValue(hadc);
```

```
    adc_ready = 1;
  }
}
```

No main():

```
adc_start_it();


while (1)
{
  if (adc_ready)
  {
    adc_ready = 0;
    // usa adc_last
    adc_start_it(); // se estiver em single-shot
  }
}
```

Observação: se o ADC estiver em modo contínuo, o callback será chamado repetidamente; nesse caso, a lógica deve ser enxuta (evitar trabalho pesado dentro do callback).

As funções e callbacks do HAL são documentadas no manual de HAL da família (UM2319) e seguem esse padrão de API.

6.4 Múltiplos canais (Scan) — buffer "por software"

Quando usar: poucas amostras por canal, sem DMA (ex.: ler 3 sensores a cada 10 ms).

No CubeMX:

- habilite Scan Conversion Mode
- configure a sequência (Rank 1, Rank 2, …)

Leitura típica:

```
uint16_t ch[3];
```

```
HAL_ADC_Start(&hadc1);


for (int i = 0; i < 3; i++)
{
   HAL_ADC_PollForConversion(&hadc1, 10);
   ch[i] = (uint16_t)HAL_ADC_GetValue(&hadc1);
}


HAL_ADC_Stop(&hadc1);
```

Importante: aqui, “buffer” é apenas um array preenchido pela CPU. Funciona bem para baixa taxa.

6.5 ADC + DMA — o modo “profissional” para aquisição

Quando usar: streaming, alta taxa, baixa carga de CPU, buffers de amostras.

6.5.1 Configuração (conceito)

No CubeMX:

- habilite DMA para ADC
- escolha Circular se for contínuo
- defina o tamanho do buffer

No código:

```
#define ADC_BUF_LEN  256
uint16_t adc_buf[ADC_BUF_LEN];


void adc_start_dma(void)
{
   HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buf, ADC_BUF_LEN);
}
```

6.5.2 Processamento por metade (HT) e completo (TC)

A técnica recomendada para circular é processar metade por vez:

```
volatile uint8_t adc_half_ready = 0;
volatile uint8_t adc_full_ready = 0;

void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
{
   if (hadc->Instance == ADC1)
      adc_half_ready = 1;
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
   if (hadc->Instance == ADC1)
      adc_full_ready = 1;
}
```

No loop:

```
while (1)
{
   if (adc_half_ready)
   {
      adc_half_ready = 0;
      // processa adc_buf[0 .. ADC_BUF_LEN/2 - 1]
   }

   if (adc_full_ready)
   {
      adc_full_ready = 0;
      // processa adc_buf[ADC_BUF_LEN/2 .. ADC_BUF_LEN - 1]
```

```
    }
}
```

Isso aplica exatamente os conceitos de DMA (HT/TC) vistos no capítulo anterior.

---

7. Conversão para tensão (do “número” para engenharia)

Depois de ler o ADC, você normalmente quer volts:

V_in="ADC" /(2^N-1)·V_ref

Exemplo (12-bit, Vref conhecido):

```
float adc_to_volts(uint16_t adc, float vref, uint16_t bits)
{
    const float max = (float)((1u << bits) - 1u);
    return ((float)adc * vref) / max;
}
```

Em aplicações críticas, use VREFINT para estimar Vref real (quando aplicável) e reduzir erro por variação de VDDA — isso é especialmente útil em sistemas alimentados por bateria.

---

8. Checklist de “especialista” (erros clássicos e como evitar)

Leitura instável/flutuante: pino não está em modo analógico ou sinal sem referência/filtragem.

Valor mais baixo que o esperado: sampling time curto para fonte de alta impedância (capacitor S/H não carrega).

Muito ruído: VDDA ruim, layout ruim, ausência de RC, ou taxa/sampling inadequados.

DMA sobrescrevendo dados: circular sem consumo por HT/TC (overrun lógico do software).

Calibração ignorada: pior repetibilidade/offset; sempre calibre no boot (e teste).

Encerramento

Com este capítulo, o leitor deve ser capaz de:

entender o ADC como MUX + sample-and-hold + conversão;

configurar single, continuous e scan no CubeMX;

ler por polling, interrupção e DMA no HAL;

projetar buffers com HT/TC de forma segura;

converter contagens em tensão e depurar erros típicos.

O próximo passo natural, ainda dentro de ADC, é um subcapítulo curto e prático sobre:

impedância da fonte + sampling time (como escolher) e

filtro RC e anti-ruído (boas práticas de hardware para medições estáveis).

Capítulo 3.1 — Impedância da fonte, sampling time e filtro RC

1. Por que o sampling time importa de verdade

O ADC do STM32 usa sample-and-hold: ele conecta o canal a um capacitor interno por um tempo configurável (sampling time) e precisa de tempo para esse capacitor carregar até a tensão real do sinal. Se a fonte for "fraca" (alta impedância), o capacitor não carrega totalmente e a leitura fica:

mais baixa do que deveria (erro sistemático),

instável (varia conforme a sequência de canais e ruídos).

A própria ST trata isso como "erro adicional" em fontes de alta impedância e recomenda aumentar o sampling time, usando a constante de tempo $\tau = R_{in} \times C_{sh}$ como referência.

---

2. Como escolher o sampling time no STM32G070

A forma mais "de engenharia" (e específica do G070) é usar a tabela do datasheet Maximum ADC RAIN. Ela informa o máximo RAIN permitido para cada sampling time (em ciclos) e resolução.

Exemplo (12 bits, ADC clock 35 MHz, tabela do datasheet):

12.5 ciclos → RAIN máx ≈ 4.7 kΩ

39.5 ciclos → RAIN máx ≈ 15 kΩ

79.5 ciclos → RAIN máx ≈ 33 kΩ

160.5 ciclos → RAIN máx ≈ 50 kΩ

Interpretação prática:

Se sua fonte (sensor/divisor) tem impedância alta, você precisa de sampling time maior.

Se você quer 12 bits "bons", evite fontes tipo 100 kΩ direto no pino sem ajustes; normalmente vai exigir sampling time máximo e/ou capacitor no pino.

Nota do datasheet: quando VDDA < 2,4 V, pode ser necessário habilitar o analog switch voltage booster (BOOSTEN) para manter as características.

---

3. Onde isso aparece no HAL (o "botão" que resolve)

No HAL, isso é o campo SamplingTime na configuração do canal (geralmente gerado pelo CubeMX):

```
sConfig.SamplingTime = ADC_SAMPLINGTIME_160CYCLES_5;  // exemplo

HAL_ADC_ConfigChannel(&hadc1, &sConfig);
```

Regra de bolso para o leitor:

Se a leitura está "baixa" ou varia com o tempo/sequência, aumente o SamplingTime primeiro.

---

4. Filtro RC: quando colocar e como não atrapalhar o ADC

A ST recomenda adicionar filtro RC externo para remover componentes de alta frequência e reduzir ruído no sinal antes do ADC.

Topologia típica:

resistor série pequeno (ex.: 100 Ω a 1 kΩ)

capacitor para GND no pino ADC (ex.: 1 nF a 100 nF, dependendo da aplicação)

Cuidados:

RC grande demais pode atrasar o sinal (importante se você mede sinais rápidos).

RC + fonte de alta impedância pode piorar tempo de assentamento se você amostra muito rápido.

---

5. “Fonte muito alta” (divisores grandes): o truque do capacitor no pino

Quando a impedância é muito alta e o sampling time máximo ainda não resolve, a ST recomenda uma solução combinada:

software: aumentar sampling time

hardware: adicionar um capacitor externo (Cext) no pino para servir como “reservatório” de carga, reduzindo o erro de amostragem.

A aplicação nota também alerta que, com Cext, você precisa respeitar o tempo entre conversões para não ir carregando esse capacitor com erros acumulados (especialmente em taxas altas).

---

6. Checklist rápido de estabilidade (o que testar, nessa ordem)

Pino em modo analógico (CubeMX/IO).

Aumentar SamplingTime (comece alto e reduza depois).

Verificar se a fonte atende o RAIN máximo do datasheet.

Se ainda houver ruído: adicionar RC.

Se a fonte for muito alta: adicionar Cext no pino e ajustar taxa/espera.

Com isso, você consegue dimensionar sampling time e resolver grande parte dos problemas clássicos de ADC, como leitura baixa, instável ou dependente da sequência de canais, antes mesmo de partir para filtros digitais ou oversampling.

Capítulo 4 — USART/UART no STM32G070RB (hardware, configuração e uso com HAL: polling, interrupção e DMA)

1. O que é UART e o que é USART

UART (Universal Asynchronous Receiver/Transmitter) é um periférico de comunicação serial assíncrona: transmite e recebe bytes usando duas linhas principais:

TX (transmissão)

RX (recepção)

USART (Universal Synchronous/Asynchronous Receiver/Transmitter) é uma UART “mais completa”, porque além do modo assíncrono (UART), pode suportar modo síncrono (com clock de transmissão), dependendo da família/instância.

No uso mais comum do STM32, quando se fala em “USART”, normalmente trata-se do modo UART assíncrono: console serial, debug, comunicação com módulos GNSS, 4G, BLE e protocolos simples em geral.

---

2. Para que serve

Log/Debug (printf, console, telemetria)

Comunicação com módulos (GNSS, modem 4G, Bluetooth)

Protocolos industriais simples (ASCII/binary)

Interface com PCs (via USB-UART), displays e sensores seriais

---

3. Como o hardware funciona

3.1 Enquadramento (frame)

Em UART assíncrona, cada byte é transmitido como um quadro:

Start bit (sempre 0)

Bits de dados (7, 8 ou 9)

Paridade (opcional)

Stop bit(s) (1 ou 2)

A linha fica em nível alto quando ociosa (idle = 1). O start bit leva a linha para 0 e “avisa” o receptor para amostrar no tempo correto.

3.2 Baud rate (taxa)

A taxa em baud define o tempo de cada bit. Ex.: 115200 bps → ~8,68 µs por bit.

No STM32, o baud rate é gerado por um divisor interno (baseado no clock do periférico). A precisão depende do clock e da configuração; quanto melhor o clock, menor erro.

3.3 Oversampling e amostragem

O receptor não “lê um bit de uma vez”. Ele faz amostragem com oversampling (por exemplo 16x ou 8x) para posicionar melhor o ponto de amostragem e tolerar pequenas diferenças de baud entre TX e RX.

3.4 Buffers internos e flags

A USART tem registradores/buffers para:

TX: você escreve o dado e o hardware serializa

RX: o hardware desserializa e entrega o byte recebido

E flags típicas (conceito):

TX “pronto para novo byte”

RX “tem byte disponível”

erros: framing, parity, noise, overrun

Overrun (OVR) é clássico: chegou byte novo antes do anterior ser lido.

---

4. Configurações essenciais

4.1 Parâmetros básicos

BaudRate (ex.: 9600, 115200)

Word length (8/9 bits)

Parity (none/even/odd)

Stop bits (1/2)

Mode (TX, RX, TX_RX)

HW Flow control (RTS/CTS) quando necessário (modems, alta taxa)

4.2 Regra prática de compatibilidade

Os dois lados precisam concordar em:

baud rate (com tolerância limitada),

tamanho do quadro (bits/paridade/stop),

níveis elétricos corretos (TTL 3.3V vs RS-232 etc.).

---

5. CubeMX e pinos (AF)

Para UART funcionar você precisa:

configurar a instância USARTx no CubeMX,

escolher pinos TX/RX (Alternate Function),

habilitar clock e gerar MX_USARTx_UART_Init().

No código HAL isso vira o handle UART_HandleTypeDef huartX;.

---

6. Uso com HAL (do simples ao avançado)

6.1 Polling — transmissão e recepção "diretas"

Quando usar: debug simples, testes rápidos, baixo tráfego.

6.1.1 Transmit (bloqueante)

```
const char msg[] = "Hello\r\n";

HAL_UART_Transmit(&huart2, (uint8_t*)msg, sizeof(msg)-1, 100);
```

6.1.2 Receive (bloqueante)

```
uint8_t b;

if (HAL_UART_Receive(&huart2, &b, 1, 100) == HAL_OK)

{

   // recebeu 1 byte

}
```

Observação importante: polling bloqueia a CPU. Para recepção contínua, polling costuma ser a pior opção.

6.2 Interrupção (IT) — "event-driven" e leve

Quando usar: tráfego moderado, mensagens pequenas, sem streaming pesado.

6.2.1 RX byte-a-byte (modelo didático)

```
static uint8_t rx_byte;

static volatile uint8_t rx_ready;


void uart_rx_it_start(void)

{

   rx_ready = 0;
```

```
   HAL_UART_Receive_IT(&huart2, &rx_byte, 1);
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
   if (huart->Instance == USART2)
   {
      rx_ready = 1;

      // rearmar para receber o próximo byte
      HAL_UART_Receive_IT(&huart2, &rx_byte, 1);
   }
}
No while(1):
if (rx_ready)
{
   rx_ready = 0;
   // processa rx_byte
}
```

6.2.2 TX por interrupção

HAL_UART_Transmit_IT(&huart2, (uint8_t*)msg, len);

Vantagem: CPU não fica “presa” esperando o periférico.

Limite: byte-a-byte em IT não escala tão bem para streams longos; para isso, DMA é o ideal.

6.3 DMA — o modo “profissional” (alto desempenho)

Quando usar: recepção contínua, bursts longos, modems/GNSS, alto throughput.

6.3.1 TX com DMA (simples e muito eficiente)

```
HAL_UART_Transmit_DMA(&huart2, (uint8_t*)buf, len);
```

Callback de fim:

```
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART2)
  {
    // transmissão terminou
  }
}
```

6.3.2 RX com DMA: normal vs circular

Normal DMA: recebe N bytes e para (bom para pacotes de tamanho conhecido).

Circular DMA: recebe continuamente em um buffer circular (bom para streams).

A) RX DMA “tamanho fixo”

```
uint8_t rx_buf[64];
HAL_UART_Receive_DMA(&huart2, rx_buf, sizeof(rx_buf));
```

Quando completar, o HAL chama:

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART2)
  {
    // rx_buf cheio com 64 bytes
  }
}
```

B) RX DMA contínuo com “metade cheia” (HT/TC)

Quando em circular, use:

HalfCplt → primeira metade pronta

Cplt → segunda metade pronta

```
#define RX_DMA_LEN 256
uint8_t rx_dma[RX_DMA_LEN];

HAL_UART_Receive_DMA(&huart2, rx_dma, RX_DMA_LEN);
```

Callbacks:

```
void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART2)
  {
    // processa rx_dma[0 .. RX_DMA_LEN/2 - 1]
  }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART2)
  {
    // processa rx_dma[RX_DMA_LEN/2 .. RX_DMA_LEN - 1]
  }
}
```

6.3.3 “Como eu sei onde termina a mensagem?” (IDLE line)

Em UART real, muitas vezes você não sabe o tamanho exato do pacote. Um método clássico é usar a detecção de IDLE (linha ociosa por 1 frame), e então considerar que “a mensagem acabou”.

No HAL, existe um padrão muito usado:

RX em DMA circular

usar IDLE para “cortar” frames variáveis

(Implementação completa depende da versão HAL e do template do CubeMX; a ideia do livro é: DMA enche o buffer; IDLE define fronteira de mensagem; CPU consome do buffer com índice.)

---

7. Bufferização correta (para não perder dados)

7.1 Por que precisa de buffer

Se o firmware processar mais devagar do que os bytes chegam, você perde dados.

Soluções:

DMA circular + consumo por índice

fila/ring buffer em software

prioridades de interrupção bem definidas

7.2 Erros típicos

Overrun: RX chegou e ninguém leu a tempo (ou DMA/parâmetros errados).

Concorrência: CPU lendo regiões que DMA ainda está escrevendo (resolvido com HT/TC ou índices).

Bloqueio por polling: HAL_UART_Receive() com timeout alto "congela" o sistema.

---

8. Erros e diagnóstico (o que o especialista checa)

No HAL, quando algo falha:

verifique HAL_UART_ErrorCallback()

valide: baud/word/parity/stop

valide o nível elétrico (3.3V TTL vs RS-232)

confirme que RX/TX não estão invertidos

confira se o pino está em AF correto e sem conflito

Exemplo de callback:

```
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART2)
```

```
  {
    // aqui você pode registrar erro, reiniciar RX, etc.
  }
}
```

---

9. Checklist final do capítulo (o leitor "especialista")

Ao final, o leitor deve conseguir:

- explicar frame UART (start, dados, paridade, stop) e o papel do baud rate;
- configurar UART no CubeMX (pinos AF, parâmetros, clock);
- transmitir e receber por polling (e saber quando não usar);
- receber por interrupção (RX rearmado e callbacks);
- transmitir e receber por DMA (normal e circular);
- entender como construir recepção robusta (HT/TC e/ou IDLE + ring buffer);
- depurar problemas clássicos (overrun, framing, mismatch de config, níveis elétricos).

Capítulo 4.1 — Recepção robusta de mensagens de tamanho variável (UART + DMA + detecção de IDLE)

1. Problema típico

Em módulos como GNSS e modems AT, você raramente sabe o tamanho exato do que vai chegar (linhas, frames, respostas variáveis). Se você usar HAL_UART_Receive() (polling) ou RX por interrupção byte-a-byte, o risco de travar CPU ou perder dados aumenta.

A solução mais usada em STM32 é:

- RX com DMA (buffer circular ou "grande")
- usar IDLE line para detectar "fim de burst" (linha ficou ociosa por ~1 frame)
- processar somente o que chegou desde o último evento

O HAL oferece isso diretamente via ReceiveToIdle: HAL_UARTEx_ReceiveToIdle_DMA() + callback de evento.

---

2. Conceito: “DMA enche, IDLE delimita”

O DMA coloca bytes no buffer automaticamente.

O IDLE funciona como um delimitador natural quando há pausas entre mensagens.

A aplicação processa o delta: “quantos bytes novos chegaram desde a última vez”.

---

3. Exemplo prático com HAL: ReceiveToIdle + DMA

3.1 Variáveis e buffer

```
#define UART_RX_BUF_LEN  256


static uint8_t  uart_rx_dma[UART_RX_BUF_LEN];

static volatile uint16_t uart_rx_size = 0;

static volatile uint8_t  uart_rx_event = 0;
```

3.2 Iniciar recepção (no init do app)

```
void uart_rx_start_to_idle_dma(void)

{

   uart_rx_event = 0;

   uart_rx_size  = 0;


   HAL_UARTEx_ReceiveToIdle_DMA(&huart2, uart_rx_dma, UART_RX_BUF_LEN);


   // Boa prática: desabilitar IRQ de Half Transfer do DMA se você não for usar,

   // para reduzir interrupções (depende do projeto/geração CubeMX).

   // __HAL_DMA_DISABLE_IT(huart2.hdmarx, DMA_IT_HT);

}
```

3.3 Callback de evento (quando IDLE ocorre ou buffer enche)

O HAL chama este callback informando o Size (quantos bytes válidos existem no buffer desde o start/rearm):

```
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size)
{
  if (huart->Instance == USART2)
  {
    uart_rx_size  = Size;
    uart_rx_event = 1;

    // Rearma para o próximo burst (padrão de uso do ReceiveToIdle)
    HAL_UARTEx_ReceiveToIdle_DMA(&huart2, uart_rx_dma, UART_RX_BUF_LEN);
    // __HAL_DMA_DISABLE_IT(huart2.hdmarx, DMA_IT_HT);
  }
}
```

3.4 Processar no loop principal (fora de interrupção)

```
void app_loop(void)
{
  if (uart_rx_event)
  {
    uart_rx_event = 0;

    // Processa uart_rx_dma[0 .. uart_rx_size-1]
    // Ex.: procurar "\r\n", montar linha AT, parser NMEA, etc.
  }
}
```

Por que isso é “robusto”?

Você não depende de receber “N bytes exatos”.

O IDLE cria um gatilho natural para processar um pacote variável.

A CPU não fica presa copiando byte a byte.

---

4. Observações importantes (para o leitor não cair em armadilhas)

Se não houver pausas (stream contínuo sem idle), o IDLE pode nunca ocorrer. Nesse caso, use HT/TC em circular ou um protocolo com delimitador/tamanho.

Buffer encheu: o callback também pode disparar por "Size == UART_RX_BUF_LEN". Trate isso como "pacote grande / overflow lógico" (ou aumente buffer).

Não processe pesado no callback: apenas marque evento/copie para uma fila. Processamento pesado deve ficar fora da ISR.

---

5. Quando preferir circular + índices (modelo "stream")

Se você precisa receber continuamente (sem pausas confiáveis), o modelo clássico é:

DMA circular em uart_rx_dma[]

processar por índice de leitura (consome o que chegou)

usar HT/TC (ou IDLE apenas como "aceleração" de processamento)

Capítulo 5 — SPI no STM32G070RB (hardware, configuração e uso com HAL: polling, interrupção e DMA)

1. O que é SPI e para que serve

SPI (Serial Peripheral Interface) é um barramento serial síncrono, de alta velocidade, usado para comunicação entre um mestre (master) e um ou mais escravos (slaves). É extremamente comum para:

displays (TFT/OLED SPI), controladores touch

memórias Flash/EEPROM (ex.: W25Qxx)

sensores (IMU, ADCs externos, expansores)

DACs, drivers de LED, conversores diversos

O SPI é simples, rápido e previsível, mas exige atenção elétrica e de protocolo (CS, modos, tamanho de frame).

---

2. Como o hardware funciona

SPI transmite dados em forma de bits sincronizados por um clock gerado pelo master.

Linhas típicas:

SCK (Serial Clock): clock do master

MOSI (Master Out, Slave In): dados do master para o slave

MISO (Master In, Slave Out): dados do slave para o master

NSS/CS (Chip Select): seleciona qual slave está ativo

2.1 Full-duplex por natureza

No SPI clássico, quando o master gera clock, sempre há deslocamento:

um bit sai em MOSI

um bit entra em MISO

ao mesmo tempo.

Isso é importante: mesmo que você "só queira ler", normalmente você precisa "enviar dummy" (0xFF, 0x00) para gerar clock e receber dados.

2.2 O papel do CS (Chip Select)

O CS define o início e o fim de uma transação. Muitos dispositivos:

exigem CS baixo durante toda a transferência,

"latcham" comandos no primeiro byte,

interpretam bytes seguintes como endereço/dados.

Regra prática:

O CS faz parte do protocolo do dispositivo, não do SPI genérico.

---

3. Modos do SPI (CPOL/CPHA) — o ponto que mais dá erro

SPI tem 4 modos, definidos por:

CPOL: polaridade do clock em idle (0 = idle baixo, 1 = idle alto)

CPHA: fase (em qual borda o dado é amostrado)

Modo	CPOL	CPHA	Clock idle	Amostra em
0	0	0	baixo	borda de subida
1	0	1	baixo	borda de descida
2	1	0	alto	borda de descida
3	1	1	alto	borda de subida

Regra de ouro: o modo é definido pelo slave (datasheet). Se o modo estiver errado, os dados ficam "trocados", deslocados ou lixo.

---

4. Outras configurações importantes

Bit order: MSB first (padrão) ou LSB first (raro)

Data size: 8 bits (comum) ou 16 bits

Prescaler: define a frequência de SCK

NSS:

hardware NSS (o periférico controla)

software NSS (CS por GPIO, mais comum em projetos práticos)

CRC: raro no dia a dia, depende do dispositivo

---

5. Prós e contras do SPI

5.1 Vantagens

alta velocidade (comparado a I^2C)

latência baixa e determinística

implementação simples no hardware

excelente para bursts e streaming curto

5.2 Desvantagens

usa mais pinos (SCK/MOSI/MISO/CS)

não tem endereçamento nativo (cada slave precisa de CS)

não tem confirmação/ack no protocolo básico (integridade depende do sistema)

em cabos longos é sensível a ruído/reflexão (clock rápido + bordas rápidas)

---

6. Configuração no CubeMX (HAL)

Para SPI funcionar:

habilite SPIx (ex.: SPI1)

escolha pinos AF para SCK/MOSI/MISO

escolha:

Master/Slave

CPOL/CPHA

Prescaler

Data Size

NSS (hardware ou software)

gere MX_SPIx_Init() e SPI_HandleTypeDef hspiX;

Recomendação: use NSS por software (CS como GPIO), pois é o modelo mais aplicável a memórias, displays e sensores.

---

7. Exemplos com HAL (polling → interrupção → DMA)

7.1 CS por GPIO (padrão recomendado)

```
static inline void spi_cs_low(void)
{
   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);
}


static inline void spi_cs_high(void)
```

```
{
   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
}
```

7.2 Polling (bloqueante) — simples e confiável

Quando usar: comandos curtos, debug, inicialização de dispositivo.

7.2.1 Transmit

```
spi_cs_low();
HAL_SPI_Transmit(&hspi1, tx, tx_len, 100);
spi_cs_high();
```

7.2.2 Receive (com dummy clocks)

Muitos HALs oferecem HAL_SPI_Receive(), mas por baixo ele precisa gerar clock. O uso mais universal é TransmitReceive:

```
spi_cs_low();
HAL_SPI_TransmitReceive(&hspi1, tx_dummy, rx, len, 100);
spi_cs_high();
```

7.2.3 Full-duplex (comando + resposta)

```
spi_cs_low();
HAL_SPI_Transmit(&hspi1, cmd, cmd_len, 100);
HAL_SPI_Receive(&hspi1, resp, resp_len, 100);
spi_cs_high();
```

Observação: alguns slaves exigem CS baixo sem “intervalo” entre comando e leitura.

7.3 Interrupção (IT) — sem travar CPU

Quando usar: transferências moderadas e quando você não quer bloquear.

7.3.1 Transmit IT

```
spi_cs_low();
HAL_SPI_Transmit_IT(&hspi1, tx, tx_len);
```

Callback:

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi)
{
   if (hspi->Instance == SPI1)
   {
      spi_cs_high();
      // sinaliza "tx terminou"
   }
}
```

7.3.2 Receive IT / TransmitReceive IT

```
spi_cs_low();
HAL_SPI_TransmitReceive_IT(&hspi1, tx, rx, len);
```

Callback:

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi)
{
   if (hspi->Instance == SPI1)
   {
      spi_cs_high();
      // rx pronto
   }
}
```

Cuidado importante: CS deve permanecer ativo até o fim da transferência (no callback).

7.4 DMA — alto desempenho (bursts grandes)

Quando usar: displays, memórias, sensores com bursts grandes, throughput alto.

7.4.1 Transmit DMA

```
spi_cs_low();
HAL_SPI_Transmit_DMA(&hspi1, tx, tx_len);
```

No callback:

```
void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi)
{
   if (hspi->Instance == SPI1)
   {
      spi_cs_high();
      // tx terminou
   }
}
```

7.4.2 TransmitReceive DMA (full duplex)

```
spi_cs_low();
HAL_SPI_TransmitReceive_DMA(&hspi1, tx, rx, len);
```

No callback:

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi)
{
   if (hspi->Instance == SPI1)
   {
      spi_cs_high();
      // rx pronto
   }
}
```

Nota: para “só ler” com DMA, você ainda precisa transmitir dummy para gerar clock.

---

8. “Tipos” de SPI na prática

8.1 SPI 2 fios (full duplex)

SCK + MOSI + MISO, padrão.

8.2 SPI 1 fio (half duplex)

Alguns dispositivos usam uma linha de dados bidirecional. O STM32 suporta modos half-duplex, mas isso é mais raro no dia a dia.

8.3 SPI com vários CS

O barramento (SCK/MOSI/MISO) é compartilhado e cada slave tem seu CS dedicado.

O software controla qual slave está ativo.

---

9. Problemas clássicos e diagnóstico

9.1 Dados incorretos

CPOL/CPHA errado (modo errado)

frequência alta demais (reduzir prescaler)

CS não respeita timing do slave

nível lógico incompatível (3.3V vs 5V)

fios longos/ruído/ringing

9.2 Comunicação “funciona às vezes”

clock muito alto + bordas rápidas + layout ruim

CS com glitches (GPIO configurado errado / interferência)

falta de pull-up/pull-down em MISO quando slave está em alta impedância

9.3 Leitura sempre 0xFF ou 0x00

MISO desconectado ou slave não respondendo

CS incorreto

“dummy clocks” não gerados (leitura mal feita)

---

10. Encerramento

Após este capítulo, o leitor deve ser capaz de:

entender SPI como barramento síncrono full-duplex;

dominar CPOL/CPHA e por que isso é o principal ponto de falha;

configurar SPI no CubeMX e usar CS por GPIO;

comunicar via HAL em polling, interrupção e DMA;

depurar erros típicos (modo, clock, CS, integridade de sinal).

Um complemento importante deste tema é um modelo de driver SPI com transações atômicas, controle correto de CS, timeout e proteção contra reentrância.

Capítulo 5.1 — Modelo de driver SPI: transações atômicas com CS, timeouts e reentrância

1. Por que criar um "driver de transação"

Em SPI, o que realmente importa para o dispositivo é a transação completa:

CS desce

troca de bytes acontece (TX/RX)

CS sobe

Se duas partes do firmware tentarem usar o mesmo SPI ao mesmo tempo, o erro é imediato: bytes misturam, CS "corta" no meio, e o slave interpreta comandos errados.

Por isso, um driver SPI bem feito oferece uma função de transação atômica, que garante:

CS controlado corretamente,

uma transferência por vez,

timeout previsível,

mesmo padrão para polling/IT/DMA.

---

2. Estrutura mínima recomendada

```
typedef struct
{
  SPI_HandleTypeDef *hspi;
  GPIO_TypeDef      *cs_port;
  uint16_t          cs_pin;
  volatile uint8_t   busy;
```

```
} spi_dev_t;
```

Funções auxiliares:

```
static inline void spi_cs_low(spi_dev_t *dev)
{
   HAL_GPIO_WritePin(dev->cs_port, dev->cs_pin, GPIO_PIN_RESET);
}

static inline void spi_cs_high(spi_dev_t *dev)
{
   HAL_GPIO_WritePin(dev->cs_port, dev->cs_pin, GPIO_PIN_SET);
}
```

---

3. “Lock” simples para reentrância (sem RTOS)

Sem RTOS, o suficiente é bloquear uso concorrente:

```
static uint8_t spi_try_lock(spi_dev_t *dev)
{
   if (dev->busy) return 0;
   dev->busy = 1;
   return 1;
}

static void spi_unlock(spi_dev_t *dev)
{
   dev->busy = 0;
}
```

Em RTOS, a ideia é a mesma, mas você troca isso por um mutex/semaphore.

---

4. Transação atômica em polling (a base mais segura)

Essa é a “versão ouro” para inicialização e comandos curtos:

```
HAL_StatusTypeDef spi_txrx_polling(spi_dev_t *dev,
                                   uint8_t *tx, uint8_t *rx,
                                   uint16_t len,
                                   uint32_t timeout_ms)
{
    if (!spi_try_lock(dev))
        return HAL_BUSY;

    spi_cs_low(dev);

    HAL_StatusTypeDef st = HAL_SPI_TransmitReceive(dev->hspi, tx, rx, len,
timeout_ms);

    spi_cs_high(dev);
    spi_unlock(dev);

    return st;
}
```

E para “só escrever”:

```
HAL_StatusTypeDef spi_tx_polling(spi_dev_t *dev,
                                 uint8_t *tx,
                                 uint16_t len,
                                 uint32_t timeout_ms)
{
    if (!spi_try_lock(dev))
        return HAL_BUSY;
```

```
    spi_cs_low(dev);
    HAL_StatusTypeDef st = HAL_SPI_Transmit(dev->hspi, tx, len, timeout_ms);
    spi_cs_high(dev);


    spi_unlock(dev);
    return st;
}
```

---

5. Transação atômica com DMA (padrão com callback)

Em DMA/IT, a transferência termina no callback, então o driver precisa:

- marcar busy
- baixar CS
- iniciar DMA
- no callback: subir CS, liberar busy, sinalizar “done”

5.1 Estrutura com flag de conclusão (sem RTOS)

```
typedef struct
{
    spi_dev_t dev;
    volatile uint8_t done;
    volatile HAL_StatusTypeDef last_status;
} spi_dev_dma_t;
```

5.2 Iniciar TX/RX DMA

```
HAL_StatusTypeDef spi_txrx_dma_start(spi_dev_dma_t *d,
                                     uint8_t *tx, uint8_t *rx,
                                     uint16_t len)
{
    if (!spi_try_lock(&d->dev))
        return HAL_BUSY;
```

```
    d->done = 0;
    d->last_status = HAL_OK;

    spi_cs_low(&d->dev);

    HAL_StatusTypeDef st = HAL_SPI_TransmitReceive_DMA(d->dev.hspi, tx, rx, len);
    if (st != HAL_OK)
    {
        spi_cs_high(&d->dev);
        spi_unlock(&d->dev);
        return st;
    }

    return HAL_OK;
}
```

5.3 Callback para finalizar a transação

No callback global do HAL, finalize apenas se for a SPI correta:

```
extern spi_dev_dma_t g_spi1_dev;

void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi)
{
    if (hspi->Instance == SPI1)
    {
        spi_cs_high(&g_spi1_dev.dev);
        spi_unlock(&g_spi1_dev.dev);
        g_spi1_dev.done = 1;
    }
```

```
}

void HAL_SPI_ErrorCallback(SPI_HandleTypeDef *hspi)
{
  if (hspi->Instance == SPI1)
  {
    spi_cs_high(&g_spi1_dev.dev);
    spi_unlock(&g_spi1_dev.dev);
    g_spi1_dev.last_status = HAL_ERROR;
    g_spi1_dev.done = 1;
  }
}
```

5.4 Espera “testável” no loop (opcional)

Se for desejada uma API bloqueante por cima do DMA, especialmente em exemplos didáticos, um modelo simples e testável e:

```
HAL_StatusTypeDef spi_wait_done(spi_dev_dma_t *d, uint32_t timeout_ms)
{
  uint32_t t0 = HAL_GetTick();
  while (!d->done)
  {
    if ((HAL_GetTick() - t0) > timeout_ms)
      return HAL_TIMEOUT;
  }
  return d->last_status;
}
```

---

6. Timeouts e segurança de CS (ponto crítico)

Um erro comum é deixar o CS baixo se algo der errado. O driver acima garante que:

se HAL_SPI_... falhar ao iniciar → CS volta alto imediatamente

se der erro no meio → ErrorCallback sobe CS e libera lock

se timeout no wait_done → você pode decidir:

abortar (HAL_SPI_Abort / HAL_SPI_Abort_IT dependendo do HAL)

forçar CS alto e liberar lock (estratégia defensiva)

No livro, a recomendação geral é:

sempre ter uma estratégia de "abort/timeout" para não travar o barramento.

---

7. Como usar esse modelo em drivers de dispositivos

Um driver de memória Flash, por exemplo, vira algo limpo:

flash_read_id() → monta comando → spi_txrx_polling()

flash_read(addr, buf, len) → comando+endereço + leitura

flash_write_enable() → comando curto

O SPI fica "genérico" e o dispositivo fica "específico".

Encerramento

Com esse modelo, o leitor aprende a forma correta de usar SPI em firmware real:

transação atômica com CS,

exclusão mútua (reentrância),

timeouts,

e fechamento limpo no callback (DMA/IT).

Isso evita os bugs mais caros de SPI: corrupção aleatória, travamentos e "funciona só às vezes".

Capítulo 6 — I²C no STM32G070RB (hardware, protocolo passo a passo e HAL: polling, interrupção e DMA)

1. O que é I²C e para que serve

I^2C (Inter-Integrated Circuit) é um barramento serial síncrono, de 2 fios, feito para conectar vários dispositivos na mesma linha:

sensores (temperatura, IMU, pressão)

RTCs

EEPROMs

expansores de GPIO

controladores touch, codecs simples, PMICs

Linhas do barramento:

SCL (clock)

SDA (dados)

I^2C é muito eficiente em pinos e suportado por quase todo sensor/módulo do mercado.

---

2. Como o hardware do I^2C funciona (o que torna ele diferente)

2.1 Open-drain e pull-ups (a base elétrica do I^2C)

I^2C não dirige nível alto ativamente. SDA e SCL são open-drain/open-collector:

para enviar 0: o dispositivo puxa a linha para GND (conduz)

para enviar 1: o dispositivo solta a linha (alta impedância) e o pull-up externo leva para alto

Conclusão:

I^2C exige resistores de pull-up em SDA e SCL (normalmente externos).

Sem pull-up adequado, o barramento falha (subida lenta, ruído, tempo fora de especificação).

2.2 Multi-master, arbitragem e "wired-AND"

Como tudo é open-drain, vários dispositivos podem "puxar para baixo" sem curto. Isso permite:

multi-master (mais de um master no barramento)

arbitragem: se dois masters tentarem enviar ao mesmo tempo, quem tentar enviar '1' mas ler '0' perde e para.

2.3 Clock stretching

O slave pode “segurar” SCL baixo para atrasar o master (ex.: está ocupado). Isso é chamado clock stretching.

Nem todo sistema gosta de clock stretching (principalmente em ambientes com timeouts agressivos), mas é parte do protocolo.

---

3. Prós e contras do I²C

Vantagens

- apenas 2 fios para muitos dispositivos
- endereçamento nativo (7-bit e 10-bit)
- bom para sensores e registradores
- hardware de suporte forte no STM32

Desvantagens

- mais sensível a layout, capacitância e pull-ups (subida lenta)
- taxa menor que SPI em muitos casos
- debug de “travou o barramento” é comum (SDA preso baixo, etc.)
- transações têm mais regras (ACK/NACK, repeated start, etc.)

---

4. Velocidades e dimensionamento básico do pull-up

Modos comuns:

- Standard: 100 kHz
- Fast: 400 kHz
- Fast Mode Plus: 1 MHz (depende do MCU e do barramento)

Pull-up na prática

Quanto maior a capacitância do barramento (cabos, trilhas longas, muitos devices), mais lenta a subida.

Pull-up menor (ex.: 2.2 kΩ) sobe mais rápido, mas consome mais corrente quando a linha está em 0.

Pull-up maior (ex.: 10 kΩ) consome menos, mas pode não subir a tempo em 400 kHz/1 MHz.

Regra de bolso típica:

100 kHz: 4.7 kΩ costuma funcionar bem em placas curtas

400 kHz: 2.2 kΩ a 4.7 kΩ dependendo da capacitância

---

5. I²C passo a passo: o que acontece em uma comunicação

Vamos assumir master STM32 conversando com um slave de endereço 7-bit 0x1A.

5.1 Condição START

O master inicia uma transação gerando START:

SDA cai enquanto SCL está alto

Isso "avisa" todos os slaves: "atenção, começou uma transação".

5.2 Endereço + bit R/W

Em seguida o master transmite 8 bits:

7 bits de endereço

1 bit R/W

0 = write

1 = read

Exemplo:

write para 0x1A → byte "address+W"

read de 0x1A → byte "address+R"

5.3 ACK/NACK do slave (bit 9)

Após cada byte enviado, existe um 9º "tempo de clock" para o receptor responder:

ACK (0): puxou SDA para 0 → "recebi"

NACK (1): deixou SDA em 1 → "não aceito / acabou"

Isso vale para:

endereço

dados

leitura (onde o ACK/NACK é dado pelo master ao final de cada byte recebido)

5.4 Transferência de dados

Em write: master envia bytes → slave ACKa cada byte.

Em read: slave envia bytes → master ACKa cada byte, e no último byte o master manda NACK para encerrar a leitura.

5.5 STOP

O master encerra com STOP:

SDA sobe enquanto SCL está alto

---

6. O caso mais comum do mundo real: "registradores" (Mem Read/Write)

A maioria dos sensores I²C tem registradores internos. O padrão típico:

6.1 Escrever registrador (Write Register)

START

Address + W

RegAddr

Data…

STOP

6.2 Ler registrador (Read Register) com Repeated START

START

Address + W

RegAddr

REPEATED START (sem STOP no meio)

Address + R

Data…

NACK no último byte

STOP

Repeated START é crucial: muitos devices esperam que você não "libere" o barramento entre apontar o registrador e ler.

---

7. Configuração no CubeMX (STM32G070 + HAL)

Pontos que o leitor precisa dominar:

Pinos SCL/SDA em Alternate Function Open-Drain

Pull-ups externos (recomendado)

Timing do I²C (o mais importante no STM32G0)

7.1 Timing do I²C (por que é crítico)

No STM32G0, o I²C usa um registrador de timing (derivado do clock do periférico). O CubeMX calcula isso para você.

Regra didática do livro:

confie no CubeMX para gerar o timing inicialmente

só mexa manualmente se você souber exatamente o clock fonte e os requisitos (rise/fall)

7.2 Filtros analógico/digital

O I²C do STM32 costuma ter:

Analog filter (remove glitches muito rápidos)

Digital filter (remove ruído com amostragem)

Em barramentos ruidosos, filtros ajudam bastante (com custo de atraso).

---

8. HAL: modelos de uso (polling → interrupção → DMA)

8.1 Endereço no HAL (pegadinha clássica)

No HAL, o endereço 7-bit costuma ser passado shiftado para esquerda (bit0 reservado ao R/W).

Ou seja, frequentemente você usa:

```
devAddr = (0x1A << 1)
```

Se você passar 0x1A direto, pode falhar dependendo da função.

8.2 Polling (bloqueante) — simples e ótimo para começar

8.2.1 Write “cru” (bytes)

```
uint8_t data[] = { 0x10, 0xAB, 0xCD }; // exemplo

HAL_I2C_Master_Transmit(&hi2c1, (0x1A << 1), data, sizeof(data), 100);
```

8.2.2 Read “cru”

```
uint8_t rx[4];
```

```
HAL_I2C_Master_Receive(&hi2c1, (0x1A << 1), rx, sizeof(rx), 100);
```

8.2.3 Mem Write (registrador + dados)

Essa é a função mais usada com sensores:

```
uint8_t value = 0x55;
HAL_I2C_Mem_Write(&hi2c1,
        (0x1A << 1),
        0x20,           // reg addr
        I2C_MEMADD_SIZE_8BIT,
        &value, 1,
        100);
```

8.2.4 Mem Read (registrador)

```
uint8_t value;
HAL_I2C_Mem_Read(&hi2c1,
        (0x1A << 1),
        0x20,
        I2C_MEMADD_SIZE_8BIT,
        &value, 1,
        100);
```

Por que essas funções são boas?

Elas implementam automaticamente o fluxo com repeated start quando necessário.

8.3 Interrupção (IT) — sem travar CPU

Quando usar: tráfego moderado e você quer continuar rodando outras coisas.

8.3.1 Mem Read IT

```
HAL_I2C_Mem_Read_IT(&hi2c1,
        (0x1A << 1),
        0x20, I2C_MEMADD_SIZE_8BIT,
```

```
                rx, 8);
```

Callback de conclusão:

```
void HAL_I2C_MemRxCpltCallback(I2C_HandleTypeDef *hi2c)
{
   if (hi2c->Instance == I2C1)
   {
      // rx pronto
   }
}
```

8.3.2 Mem Write IT

```
HAL_I2C_Mem_Write_IT(&hi2c1,
                (0x1A << 1),
                0x20, I2C_MEMADD_SIZE_8BIT,
                &value, 1);
```

Callback:

```
void HAL_I2C_MemTxCpltCallback(I2C_HandleTypeDef *hi2c)
{
   if (hi2c->Instance == I2C1)
   {
      // write terminou
   }
}
```

8.4 DMA — alto desempenho e baixa carga de CPU

Quando usar: blocos maiores (ex.: leitura de buffers de sensores, EEPROMs, displays I²C), ou quando você quer CPU livre.

8.4.1 Mem Read DMA

```
HAL_I2C_Mem_Read_DMA(&hi2c1,
```

```
                (0x1A << 1),
                0x20, I2C_MEMADD_SIZE_8BIT,
                rx, 64);
```

Callback:

```
void HAL_I2C_MemRxCpltCallback(I2C_HandleTypeDef *hi2c)
{
   if (hi2c->Instance == I2C1)
   {
      // rx pronto
   }
}
```

8.4.2 Mem Write DMA

```
HAL_I2C_Mem_Write_DMA(&hi2c1,
                (0x1A << 1),
                0x20, I2C_MEMADD_SIZE_8BIT,
                tx, 64);
```

---

9. Diagnóstico e erros (o que o "especialista" verifica)

9.1 Erros típicos

NACK no endereço: device não existe, endereço errado, device sem alimentação, pull-up ausente

NACK no dado: registrador inválido, sequência errada, timing/ruído

Bus busy travado: SDA preso baixo (slave travado no meio de byte)

Timeout: clock stretching longo, ruído, travamento físico

9.2 Callback de erro no HAL

```
void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c)
{
   if (hi2c->Instance == I2C1)
```

```
  {
    // registre erro, reinicie transação, recupere barramento, etc.
  }
}
```

O hi2c->ErrorCode indica categorias de erro (NACK, BUS, ARLO, OVR, TIMEOUT, etc.).

---

10. Recuperação de barramento travado (SDA preso baixo)

Isso acontece quando um slave resetou no meio de uma transferência e ficou "esperando clocks".

A técnica clássica de recuperação (conceito):

- reconfigurar SCL como GPIO open-drain
- gerar ~9 pulsos em SCL (o slave "termina" o byte pendente)
- gerar um STOP "manual" (SDA sobe com SCL alto)
- voltar os pinos para AF I²C e reinitializar o I²C

No livro, é suficiente apresentar esse procedimento como "estratégia padrão", porque resolve muitos casos reais.

---

11. Checklist final (o leitor sai especialista se dominar isto)

- Entender open-drain + pull-up e dimensionar o barramento
- Saber a sequência START → Address → ACK → Data → STOP
- Dominar repeated start para leitura de registradores
- Saber usar HAL:
- Master_Transmit/Receive
- Mem_Read/Write (8/16-bit address)
- versões _IT e _DMA
- Diagnosticar NACK, BUSY travado e timeouts
- Ter estratégia de recuperação do barramento

Capítulo 6.1 — Driver I²C de registradores: presença, retries, timeout e “scan” de endereços

1. Objetivo

A maioria dos dispositivos I²C é controlada por registradores. Um driver “de livro” precisa oferecer:

verificar presença do dispositivo

read/write de registradores (8 e 16 bits de endereço interno)

retries (tentativas) para lidar com ruído/NACK momentâneo

timeout para não travar a aplicação

scan do barramento (descobrir quem está conectado)

Tudo isso mantendo uma API simples para os próximos capítulos.

---

2. Convenções do HAL (endereço 7-bit)

No HAL, passe o endereço 7-bit shiftado:

```
#define I2C_ADDR_7B(x)   ((x) << 1)
```

Ex.: dispositivo 0x1A → I2C_ADDR_7B(0x1A).

---

3. Funções base: presença e scan

3.1 Verificar presença (ACK no endereço)

O HAL tem uma função direta para isso:

```
HAL_StatusTypeDef i2c_is_ready(I2C_HandleTypeDef *hi2c, uint8_t addr7, uint32_t timeout_ms)
{
   return HAL_I2C_IsDeviceReady(hi2c, I2C_ADDR_7B(addr7), 1, timeout_ms);
}
```

Uso típico:

no boot: checar se sensor está conectado

antes de uma leitura crítica

3.2 Scan do barramento (descobrir dispositivos)

O scan é simplesmente testar todos os endereços válidos 7-bit (0x08 a 0x77):

```
int i2c_scan(I2C_HandleTypeDef *hi2c, uint8_t *found, int max_found)
{
  int n = 0;
  for (uint8_t a = 0x08; a <= 0x77; a++)
  {
    if (HAL_I2C_IsDeviceReady(hi2c, I2C_ADDR_7B(a), 1, 5) == HAL_OK)
    {
      if (n < max_found) found[n] = a;
      n++;
    }
  }
  return n; // quantidade total encontrada
}
```

Scan é excelente para debug e validação elétrica (pull-ups, SCL/SDA, endereços).

---

4. Read/Write de registradores (polling) — API recomendada

4.1 Write de registrador 8-bit

```
HAL_StatusTypeDef i2c_write_reg8(I2C_HandleTypeDef *hi2c,
                                 uint8_t addr7,
                                 uint8_t reg,
                                 const uint8_t *data,
                                 uint16_t len,
                                 uint32_t timeout_ms)
{
  return HAL_I2C_Mem_Write(hi2c,
                           I2C_ADDR_7B(addr7),
```

```
                    reg,
                    I2C_MEMADD_SIZE_8BIT,
                    (uint8_t*)data,
                    len,
                    timeout_ms);
}
```

4.2 Read de registrador 8-bit

```
HAL_StatusTypeDef i2c_read_reg8(I2C_HandleTypeDef *hi2c,
                      uint8_t addr7,
                      uint8_t reg,
                      uint8_t *data,
                      uint16_t len,
                      uint32_t timeout_ms)
{
   return HAL_I2C_Mem_Read(hi2c,
                    I2C_ADDR_7B(addr7),
                    reg,
                    I2C_MEMADD_SIZE_8BIT,
                    data,
                    len,
                    timeout_ms);
}
```

4.3 Variante com registrador 16-bit (muito comum em EEPROM/sensores)

```
HAL_StatusTypeDef i2c_read_reg16(I2C_HandleTypeDef *hi2c,
                      uint8_t addr7,
                      uint16_t reg,
                      uint8_t *data,
                      uint16_t len,
```

```
                    uint32_t timeout_ms)
{
  return HAL_I2C_Mem_Read(hi2c,
                  I2C_ADDR_7B(addr7),
                  reg,
                  I2C_MEMADD_SIZE_16BIT,
                  data,
                  len,
                  timeout_ms);
}
```

---

5. Re-tentativas (retries) e tratamento de falhas

Em campo, é normal ter um NACK momentâneo por:

ruído,

clock stretching inesperado,

dispositivo ocupado (ex.: EEPROM durante write cycle),

barramento com capacitância no limite.

Uma política simples e eficaz é:

tentar até N vezes

se falhar, dar uma pequena espera

se persistir, reportar erro e (opcional) tentar recuperação de barramento

5.1 Wrapper com retries

```
HAL_StatusTypeDef i2c_read_reg8_retry(I2C_HandleTypeDef *hi2c,
                        uint8_t addr7,
                        uint8_t reg,
                        uint8_t *data,
                        uint16_t len,
                        uint32_t timeout_ms,
```

```
                                int retries,
                                uint32_t delay_ms)
{
    HAL_StatusTypeDef st;
    for (int i = 0; i <= retries; i++)
    {
        st = i2c_read_reg8(hi2c, addr7, reg, data, len, timeout_ms);
        if (st == HAL_OK)
            return HAL_OK;


        if (delay_ms) HAL_Delay(delay_ms);
    }
    return st;
}
```

Para EEPROMs, retries são praticamente obrigatórios após write, porque o dispositivo pode ficar “busy” internamente por alguns ms.

---

6. Timeouts: por que são obrigatórios

Toda função HAL de I²C (polling) tem timeout. Isso evita:

- travar a aplicação se SDA ficar preso baixo,
- travar em clock stretching infinito,
- travar em ruído severo.

Regra prática:

- leituras de 1–16 bytes: timeouts curtos (5–20 ms)
- blocos grandes: timeouts maiores, mas sempre finitos

---

7. Exemplo de uso (fluxo típico de sensor)

```
uint8_t whoami;
```

```
if (i2c_is_ready(&hi2c1, 0x1A, 10) != HAL_OK)
{
   // sensor não responde
}
else
{
   if (i2c_read_reg8_retry(&hi2c1, 0x1A, 0x0F, &whoami, 1, 20, 2, 1) == HAL_OK)
   {
      // whoami válido
   }
}
```

---

8. Encerramento

Com esse mini-driver de registradores, o leitor fica pronto para qualquer periférico I²C do livro, porque já tem:

- verificação de presença (IsDeviceReady)
- scan de endereços
- read/write 8 e 16-bit
- retries e timeouts

Capítulo 7 — Timers (TIM) no STM32G070RB: visão completa, modos e exemplos com HAL

1. O que é um Timer (TIM) e por que ele é central no STM32

Timers são periféricos de contagem baseados em clock que permitem ao microcontrolador medir tempo e gerar eventos temporizados com alta precisão, sem depender de “delays” por software. Com timers você implementa:

- timebase (tick periódico), timeouts, agendamentos
- interrupções periódicas (ex.: 1 kHz, 10 kHz)

PWM (controle de motor, LED, buzzer, fontes, etc.)

Input Capture (medir frequência, período, duty, tempo entre bordas)

Output Compare (gerar pulsos e eventos em instantes exatos)

One-Pulse (gerar um pulso único com largura precisa)

Encoder (quadratura) (leitura de encoder incremental)

Gatilhos internos para sincronizar ADC, DAC e outros blocos

Timers são “o osciloscópio interno” do firmware: se você domina timers, domina grande parte da parte de controle e aquisição de sinais.

---

2. Quais timers existem no STM32G070RB e como classificá-los

No STM32G070RB, o datasheet descreve estes grupos: TIM1 (advanced-control), TIM3/TIM14/TIM15/TIM16/TIM17 (general-purpose), e TIM6/TIM7 (basic), além de SysTick e watchdogs.

2.1 Advanced-control timer

TIM1 (16-bit, up/down/up-down, PWM avançado, saídas complementares e dead-time)

2.2 General-purpose timers

TIM3: “full-featured”, 4 canais, suporta PWM, Input Capture/Output Compare, One-Pulse e quadrature encoder

TIM14: 16-bit, 1 canal, bom para timebase simples, PWM simples, capture/compare simples

TIM15: 2 canais + 1 complementar (recursos úteis para PWM com complementar)

TIM16/TIM17: 1 canal + 1 complementar cada

2.3 Basic timers

TIM6 e TIM7: timers básicos 16-bit (sem canais de capture/compare), ideais para tick periódico e disparos internos (TRGO), e com suporte a DMA (conforme tabela do datasheet)

Observação: SysTick (do core) e watchdogs existem, mas não substituem TIMx quando você precisa PWM, captura, compare, encoder, etc.

---

3. Hardware do Timer: como ele funciona por dentro

Mesmo sem olhar registradores, você pode enxergar um timer como:

Clock do timer (vem do domínio APB e prescalers)

Prescaler (PSC): divide o clock do timer por um fator

Counter (CNT): contador (16-bit) que sobe/ desce

Auto-reload (ARR): valor máximo (top). Ao atingir ARR, gera evento de update e reinicia (ou muda direção, conforme modo)

Canais (CCR1..CCR4): comparadores que geram eventos/saídas quando CNT == CCRx

Interrupções e DMA requests: em eventos como update, compare match, capture, trigger

Fórmulas essenciais (o leitor precisa decorar)

Frequência do tick do contador

$$f_CNT=f_TIM/(PSC+1)$$

Frequência de update (overflow) em contagem "up"

$$f_UPD=f_CNT/(ARR+1)=f_TIM/((PSC+1)(ARR+1))$$

Período do update

$$T_UPD=((PSC+1)(ARR+1))/f_TIM$$

Essas três fórmulas explicam 99% de timebases e PWM.

No STM32G070, os timers podem operar até 64 MHz (ver tabela do datasheet).

---

4. Modos fundamentais (e quando usar cada um)

4.1 Base timer (timebase)

Gera interrupção periódica (Update Event). Ideal para:

"tick" de 1 ms/1 kHz

agendador simples sem RTOS

timeouts

4.2 Output Compare (OC)

Gera um evento quando CNT atinge CCRx. Ideal para:

gerar pulsos em instantes exatos

criar temporizações sem depender do overflow

“agenda” de eventos dentro do ciclo do timer

4.3 PWM

Usa CCRx como limiar para duty cycle:

PWM Mode 1/2 (mais comum Mode 1)

duty = CCRx / (ARR+1)

4.4 Input Capture (IC)

O timer “carimba” o valor do CNT quando chega uma borda no pino:

mede período/frequência

mede largura de pulso (duty)

mede tempo entre eventos externos

4.5 Encoder (quadratura)

Usa dois sinais A/B para contar posição e direção (TIM3 suporta).

4.6 One-Pulse

Gera um único pulso com duração definida (muito útil para disparos e protocolos simples).

---

5. Configuração no CubeMX (regras de ouro)

Escolha o timer certo:

TIM6/TIM7: tick periódico

TIM3/TIM1: PWM multi-canal, capture, encoder

TIM14/16/17: PWM/capture simples

Configure clock/prescaler/period (PSC/ARR)

Habilite NVIC se houver interrupção

Para PWM/IC/OC: configure pinos em Alternate Function e o canal correspondente

O HAL organiza as APIs por “grupos” (base, PWM, IC, OC, encoder).

---

6. Exemplos completos com HAL (independentes)

Abaixo estão exemplos “fechados” (com init + callback + uso). Ajuste apenas o timer (TIMx) e o clock do seu projeto.

6.1 Exemplo 1 — Tick de 1 kHz com TIM6 (interrupção periódica)

Objetivo: gerar uma interrupção a cada 1 ms, sem depender de SysTick.

Código

```
#include "stm32g0xx_hal.h"


TIM_HandleTypeDef htim6;
volatile uint32_t g_tick_1ms = 0;


static void MX_TIM6_Init_1kHz(void)
{
  __HAL_RCC_TIM6_CLK_ENABLE();


  // Supondo fTIM = 64 MHz (ajuste conforme seu clock real)
  // Queremos fUPD = 1 kHz.
  // Escolha fCNT = 1 MHz => PSC = 64-1 = 63
  // Então ARR = (1MHz/1kHz)-1 = 999
  htim6.Instance = TIM6;
  htim6.Init.Prescaler = 63;
  htim6.Init.Period = 999;
  htim6.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim6.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
```

```
    if (HAL_TIM_Base_Init(&htim6) != HAL_OK)
    {
        Error_Handler();
    }

    HAL_NVIC_SetPriority(TIM6_DAC_LPTIM1_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(TIM6_DAC_LPTIM1_IRQn);
}

void TIM6_DAC_LPTIM1_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&htim6);
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM6)
    {
        g_tick_1ms++;
    }
}

void app_start_tick_1ms(void)
{
    MX_TIM6_Init_1kHz();
    HAL_TIM_Base_Start_IT(&htim6);
}
```

Uso:

```
int main(void)
{
   HAL_Init();
   SystemClock_Config();

   app_start_tick_1ms();

   while (1)
   {
      // Ex.: timeout simples
      static uint32_t t0 = 0;
      if ((g_tick_1ms - t0) >= 500)
      {
         t0 = g_tick_1ms;
         // ação a cada 500 ms
      }
   }
}
```

6.2 Exemplo 2 — PWM em TIM3 (canal 1) com ajuste de duty em runtime

Objetivo: gerar PWM e mudar duty sem reconfigurar tudo.

Código

```
#include "stm32g0xx_hal.h"

TIM_HandleTypeDef htim3;

static void MX_TIM3_PWM_Init(uint32_t pwm_hz)
```

```
{
    __HAL_RCC_TIM3_CLK_ENABLE();

    // Supondo fTIM = 64 MHz
    // Escolha fCNT = 1 MHz => PSC = 63
    // ARR = (1MHz / pwm_hz) - 1
    uint32_t psc = 63;
    uint32_t arr = (1000000UL / pwm_hz) - 1UL;

    htim3.Instance = TIM3;
    htim3.Init.Prescaler = psc;
    htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim3.Init.Period = arr;
    htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

    if (HAL_TIM_PWM_Init(&htim3) != HAL_OK)
        Error_Handler();

    TIM_OC_InitTypeDef sConfigOC = {0};
    sConfigOC.OCMode = TIM_OCMODE_PWM1;
    sConfigOC.Pulse = 0; // duty inicial = 0
    sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
    sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;

    if (HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_1) !=
HAL_OK)
        Error_Handler();
```

```
    // Pino do TIM3_CH1 deve estar configurado em AF no CubeMX (GPIO).
}

static void pwm_set_duty_percent(uint8_t duty_percent)
{
    if (duty_percent > 100) duty_percent = 100;

    uint32_t arr = __HAL_TIM_GET_AUTORELOAD(&htim3);
    uint32_t ccr = (arr + 1UL) * duty_percent / 100UL;

    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, ccr);
}

void app_start_pwm(void)
{
    MX_TIM3_PWM_Init(20000); // 20 kHz, por exemplo
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);

    pwm_set_duty_percent(50);
}
```

6.3 Exemplo 3 — Input Capture: medir frequência (TIM3_CH1)

Objetivo: medir a frequência de um sinal externo (bordas de subida).

Ideia: capturar dois timestamps consecutivos e calcular período.

```
#include "stm32g0xx_hal.h"

TIM_HandleTypeDef htim3;
```

```
volatile uint32_t ic_last = 0;
volatile uint32_t ic_period_ticks = 0;
volatile uint8_t  ic_ready = 0;

static void MX_TIM3_IC_Init_1MHz(void)
{
  __HAL_RCC_TIM3_CLK_ENABLE();

  // fCNT = 1 MHz para facilitar cálculo (1 tick = 1 us)
  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 63;        // 64MHz/(63+1)=1MHz
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim3.Init.Period = 0xFFFF;
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

  if (HAL_TIM_IC_Init(&htim3) != HAL_OK)
    Error_Handler();

  TIM_IC_InitTypeDef sConfigIC = {0};
  sConfigIC.ICPolarity  = TIM_INPUTCHANNELPOLARITY_RISING;
  sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
  sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
  sConfigIC.ICFilter    = 0;

  if (HAL_TIM_IC_ConfigChannel(&htim3, &sConfigIC, TIM_CHANNEL_1) != HAL_OK)
    Error_Handler();
```

```
    HAL_NVIC_SetPriority(TIM3_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(TIM3_IRQn);
}

void TIM3_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&htim3);
}

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        uint32_t now = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
        ic_period_ticks = (now >= ic_last) ? (now - ic_last) : ((0x10000UL - ic_last) + now);
        ic_last = now;
        ic_ready = 1;
    }
}

void app_start_input_capture(void)
{
    MX_TIM3_IC_Init_1MHz();
    HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_1);
}
```

Uso (frequência em Hz):

```
if (ic_ready)
```

```
{
   ic_ready = 0;
   // f = 1e6 / period_ticks (porque fCNT = 1 MHz)
   float freq_hz = 1000000.0f / (float)ic_period_ticks;
}
```

6.4 Exemplo 4 — Encoder (quadratura) com TIM3

Objetivo: contar posição e direção com encoder incremental A/B (TIM3 suporta).

```
#include "stm32g0xx_hal.h"

TIM_HandleTypeDef htim3;

static void MX_TIM3_Encoder_Init(void)
{
   __HAL_RCC_TIM3_CLK_ENABLE();

   htim3.Instance = TIM3;
   htim3.Init.Prescaler = 0;
   htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
   htim3.Init.Period = 0xFFFF;
   htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

   TIM_Encoder_InitTypeDef sEnc = {0};
   sEnc.EncoderMode = TIM_ENCODERMODE_TI12;

   sEnc.IC1Polarity  = TIM_INPUTCHANNELPOLARITY_RISING;
   sEnc.IC1Selection = TIM_ICSELECTION_DIRECTTI;
   sEnc.IC1Prescaler = TIM_ICPSC_DIV1;
```

```
    sEnc.IC1Filter    = 2;

    sEnc.IC2Polarity  = TIM_INPUTCHANNELPOLARITY_RISING;
    sEnc.IC2Selection = TIM_ICSELECTION_DIRECTTI;
    sEnc.IC2Prescaler = TIM_ICPSC_DIV1;
    sEnc.IC2Filter    = 2;

    if (HAL_TIM_Encoder_Init(&htim3, &sEnc) != HAL_OK)
        Error_Handler();

    // Pinos CH1 e CH2 precisam estar em AF (CubeMX).
}

void app_start_encoder(void)
{
    MX_TIM3_Encoder_Init();
    HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
}

int16_t encoder_get_count(void)
{
    return (int16_t)__HAL_TIM_GET_COUNTER(&htim3);
}
```

6.5 Exemplo 5 — PWM com DMA (atualizando CCR automaticamente)

Objetivo: variar duty automaticamente sem CPU escrevendo CCR em loop (útil para:

- formas de onda “tabeladas”
- dimmer suave

geração de padrões)

Conceito: o DMA escreve uma sequência de valores em CCR1 a cada evento de update do timer.

TIM1 e TIM3 suportam geração de DMA request (ver tabela do datasheet).

Código (conceitualmente completo)

```
#include "stm32g0xx_hal.h"


TIM_HandleTypeDef htim3;

DMA_HandleTypeDef hdma_tim3_up;


#define LUT_LEN 64

uint16_t duty_lut[LUT_LEN];


static void make_lut(uint16_t arr)

{

   // Exemplo simples: rampa (0% -> 100%)

   for (int i = 0; i < LUT_LEN; i++)

      duty_lut[i] = (uint16_t)(((uint32_t)(arr + 1) * i) / (LUT_LEN - 1));

}


static void MX_TIM3_PWM_DMA_Init(void)

{

   __HAL_RCC_TIM3_CLK_ENABLE();

   __HAL_RCC_DMA1_CLK_ENABLE();


   // Timer em 20 kHz, fCNT=1MHz

   htim3.Instance = TIM3;

   htim3.Init.Prescaler = 63;
```

```
    htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim3.Init.Period = 49; // 1MHz/(49+1)=20kHz
    htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

    if (HAL_TIM_PWM_Init(&htim3) != HAL_OK)
        Error_Handler();

    TIM_OC_InitTypeDef sOC = {0};
    sOC.OCMode = TIM_OCMODE_PWM1;
    sOC.Pulse = 0;
    sOC.OCPolarity = TIM_OCPOLARITY_HIGH;

    if (HAL_TIM_PWM_ConfigChannel(&htim3, &sOC, TIM_CHANNEL_1) != HAL_OK)
        Error_Handler();

    // --- DMA: aqui o CubeMX normalmente gera a ligação correta (request/channel).
    // Neste capítulo (independente), o ponto é o padrão:
    // 1) hdma configurado para memory->peripheral (CCR1)
    // 2) link com __HAL_LINKDMA(&htim3, hdma[TIM_DMA_ID_UPDATE],
hdma_tim3_up);

    // Em projetos reais: configure no CubeMX:
    // TIM3 Update event -> DMA1 Channel X
    // e gere o código. Depois use HAL_TIM_DMABurst_WriteStart, etc.

    make_lut(htim3.Init.Period);
}
```

```
void app_start_pwm_dma(void)
{
   MX_TIM3_PWM_DMA_Init();
   HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);


   // Forma mais comum no HAL para atualizar CCR em burst é usar DMABurst:
   // (depende do setup gerado pelo CubeMX)
   HAL_TIM_DMABurst_WriteStart(&htim3,
                        TIM_DMABASE_CCR1,
                        TIM_DMA_UPDATE,
                        (uint32_t*)duty_lut,
                        TIM_DMABURSTLENGTH_1TRANSFER,
                        LUT_LEN);
}
```

Nota didática: para DMA em timers, o CubeMX é o caminho recomendado porque o mapeamento DMA request↔canal varia. O leitor precisa entender o conceito (DMA escrevendo CCR em eventos do timer) e usar o gerador para a ligação exata.

---

7. Recursos avançados que você também deve conhecer

7.1 TIM1: complementar + dead-time (motor control / half-bridge)

TIM1 tem recursos de "timer avançado": saídas complementares e inserção de dead-time, essenciais para meia-ponte e controle de potência.

7.2 Trigger interno (TRGO) e sincronismo

Timers podem gerar eventos internos para sincronizar outros blocos (ex.: disparar ADC em instantes fixos). O datasheet menciona explicitamente a conexão de eventos de TIMx como gatilho do ADC.

---

8. Diagnóstico e "pegadinhas" clássicas

PWM não sai no pino: pino não está em AF correto / canal errado / HAL_TIM_PWM_Start() não foi chamado.

Frequência errada: cálculo de PSC/ARR incorreto ou clock do timer não é o que você assumiu.

Interrupção não entra: NVIC não habilitado, handler errado, ou HAL_TIM_Base_Start_IT() não foi chamado.

Input capture “maluco”: falta filtro, sinal ruidoso, borda errada, ou overflow não tratado.

Encoder invertido: polaridade/ligação A/B trocadas (corrija invertendo polaridade ou fios).

---

9. Checklist final: o leitor sai especialista se dominar

calcular PSC/ARR para qualquer frequência

usar TIM6/TIM7 como timebase com interrupção

gerar PWM e ajustar duty em runtime

medir frequência/período via Input Capture

usar Encoder mode para quadratura

entender OC/One-pulse e quando aplicar

compreender o papel do DMA em timers (atualização de CCR e bursts)

sincronizar periféricos via TRGO quando necessário

Capítulo 7.1 — Output Compare e One-Pulse: eventos e pulsos com precisão (HAL)

1. Output Compare (OC): “evento quando CNT == CCR”

Output Compare usa um comparador: quando o contador CNT atinge um valor CCR, o timer gera um evento. Esse evento pode:

apenas gerar interrupção (sem sair em pino),

ou alterar um pino (toggle/set/reset) via canal OC.

Quando usar OC

agendar eventos dentro do período do timer (sem jitter de software),

gerar pulsos ou trens de pulsos com marcações precisas,

criar temporizações “em hardware”.

Exemplo OC com interrupção (evento em CCR1)

A ideia: timer roda em 1 MHz (1 tick = 1 µs). Você programa CCR1 para “daqui a X µs”.

```
#include "stm32g0xx_hal.h"

TIM_HandleTypeDef htim3;
volatile uint32_t oc_events = 0;

static void MX_TIM3_OC_1MHz_Init(void)
{
  __HAL_RCC_TIM3_CLK_ENABLE();

  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 63;            // 64MHz/(63+1)=1MHz
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim3.Init.Period = 0xFFFF;
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

  if (HAL_TIM_OC_Init(&htim3) != HAL_OK) Error_Handler();

  TIM_OC_InitTypeDef sOC = {0};
  sOC.OCMode = TIM_OCMODE_TIMING;        // OC “timing” (evento/IRQ)
  sOC.Pulse = 1000;                      // 1000 us
  sOC.OCPolarity = TIM_OCPOLARITY_HIGH;

  if (HAL_TIM_OC_ConfigChannel(&htim3, &sOC, TIM_CHANNEL_1) != HAL_OK)
Error_Handler();

  HAL_NVIC_SetPriority(TIM3_IRQn, 2, 0);
```

```
    HAL_NVIC_EnableIRQ(TIM3_IRQn);
}

void TIM3_IRQHandler(void) { HAL_TIM_IRQHandler(&htim3); }

void HAL_TIM_OC_DelayElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        oc_events++;

        // agenda próximo evento 1000 us à frente (periodicidade por OC)
        uint32_t now = __HAL_TIM_GET_COUNTER(htim);
        __HAL_TIM_SET_COMPARE(htim, TIM_CHANNEL_1, (now + 1000) & 0xFFFF);
    }
}

void app_start_oc(void)
{
    MX_TIM3_OC_1MHz_Init();
    HAL_TIM_OC_Start_IT(&htim3, TIM_CHANNEL_1);
}
```

---

2. One-Pulse: “um pulso único e termina”

One-Pulse Mode (OPM) faz o timer gerar apenas um pulso (ou uma sequência única) e parar automaticamente. É excelente para:

- gerar pulso de trigger,
- controle de “strobe”,

sinais de teste sem jitter.

Exemplo One-Pulse (um pulso de 100 µs após 1 ms)

Aqui vamos usar PWM/OC em um canal e habilitar OPM.

O pino do canal precisa estar configurado em AF no CubeMX.

```
#include "stm32g0xx_hal.h"

TIM_HandleTypeDef htim1;

static void MX_TIM1_OnePulse_Init(void)
{
  __HAL_RCC_TIM1_CLK_ENABLE();

  // Timer em 1 MHz (1 us por tick)
  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 63;
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 2000 - 1;          // janela total: 2000 us
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim1.Init.RepetitionCounter = 0;

  if (HAL_TIM_OnePulse_Init(&htim1, TIM_OPMODE_SINGLE) != HAL_OK)
Error_Handler();

  TIM_OnePulse_InitTypeDef sOP = {0};

  // Saída (CH1) como PWM1: pulso inicia em 1000us e termina em 1100us (100us
largura)
  sOP.OCMode = TIM_OCMODE_PWM1;
  sOP.Pulse = 1100;                    // CCR1 (fim do pulso em PWM1 quando CNT < CCR)
```

```
  sOP.OCPolarity = TIM_OCPOLARITY_HIGH;
  sOP.OCNPolarity = TIM_OCNPOLARITY_HIGH;
  sOP.OCFastMode = TIM_OCFAST_DISABLE;
  sOP.OCIdleState = TIM_OCIDLESTATE_RESET;
  sOP.OCNIdleState = TIM_OCNIDLESTATE_RESET;

  // Entrada de gatilho pode ser interna/externa; aqui usamos “software start”.
  // Para pulso após 1ms, usamos CCR1 e manipulamos o início via compare:
  // Solução simples: usar ARR=2000us e CCR=100us de duty e iniciar o timer no instante desejado (software).
  // (Para OPM realmente “atrasado em hardware”, combina-se OC + OPM ou slave mode trigger.)

  if (HAL_TIM_OnePulse_ConfigChannel(&htim1, &sOP, TIM_CHANNEL_1, TIM_CHANNEL_2) != HAL_OK)
      Error_Handler();
}

void app_fire_one_pulse(void)
{
  // Gera um pulso único e para
  HAL_TIM_OnePulse_Start(&htim1, TIM_CHANNEL_1);
}
```

Em aplicações reais, o “atraso” e a “largura” costumam ser feitos com OC (atraso) + PWM (largura) e OPM para parar automaticamente.

Exemplo completo: Timer disparando ADC + DMA circular (taxa ajustável tipo osciloscópio)

A seguir está um exemplo independente e pronto para “colar” (ajuste apenas: instâncias, canais ADC e clock real). A arquitetura é:

TIM6 gera TRGO em uma taxa configurável (sample_rate_hz)

ADC1 converte em cada TRGO (external trigger)

DMA circular armazena em adc_buf[]

callbacks Half/Full sinalizam blocos prontos

função scope_set_sample_rate() muda a taxa em runtime

1. Estrutura do exemplo

Buffer: 320 pontos

DMA circular

Taxa inicial: 1 kHz (mudável para 2 kHz etc.)

---

2. Código

```
#include "stm32g0xx_hal.h"


/* ===== Configurações do “osciloscópio” ===== */
#define ADC_BUF_LEN  320


ADC_HandleTypeDef hadc1;
DMA_HandleTypeDef hdma_adc1;
TIM_HandleTypeDef htim6;


uint16_t adc_buf[ADC_BUF_LEN];
volatile uint8_t adc_half_ready = 0;
volatile uint8_t adc_full_ready = 0;


/* Ajuste para o clock real do timer (fTIM).
   Se você usa SystemClock 64MHz e TIM6 em APB sem multiplicação, normalmente fTIM ~ 64MHz. */
static uint32_t g_tim_clk_hz = 64000000UL;
```

```
/* ===== Prototipos ===== */
static void MX_TIM6_TRGO_Init(uint32_t sample_rate_hz);
static void MX_ADC1_Triggered_DMA_Init(void);
static void MX_DMA_Init_For_ADC1(void);

static void Error_Handler(void)
{
    __disable_irq();
    while (1) { }
}

/* ===== Timer TIM6 como gerador de trigger (TRGO) =====
   A cada update event, TIM6 gera TRGO -> ADC converte.
*/
static void MX_TIM6_TRGO_Init(uint32_t sample_rate_hz)
{
    __HAL_RCC_TIM6_CLK_ENABLE();

    // Escolha um fCNT “confortável” (1 MHz) para facilitar ARR:
    // fCNT = fTIM / (PSC+1) -> PSC = (fTIM / 1MHz) - 1
    uint32_t fcnt = 1000000UL;
    uint32_t psc = (g_tim_clk_hz / fcnt) - 1UL;

    // ARR = (fCNT / sample_rate) - 1
    uint32_t arr = (fcnt / sample_rate_hz) - 1UL;

    htim6.Instance = TIM6;
```

```
    htim6.Init.Prescaler = psc;
    htim6.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim6.Init.Period = arr;
    htim6.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

    if (HAL_TIM_Base_Init(&htim6) != HAL_OK) Error_Handler();

    // TRGO no update event
    TIM_MasterConfigTypeDef sMaster = {0};
    sMaster.MasterOutputTrigger = TIM_TRGO_UPDATE;
    sMaster.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

    if (HAL_TIMEx_MasterConfigSynchronization(&htim6, &sMaster) != HAL_OK)
Error_Handler();
}

/* ===== DMA para ADC1 ===== */
static void MX_DMA_Init_For_ADC1(void)
{
    __HAL_RCC_DMA1_CLK_ENABLE();

    // IRQ do canal DMA: o CubeMX normalmente configura isso.
    // Aqui deixamos o NVIC habilitado de forma genérica (ajuste conforme o canal real
do ADC1 no seu .ioc).
    // Exemplo comum em G0: ADC1 em DMA1_Channel1.
    HAL_NVIC_SetPriority(DMA1_Channel1_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(DMA1_Channel1_IRQn);
}
```

```
/* Handler IRQ (ajuste se seu ADC estiver em outro canal DMA) */
void DMA1_Channel1_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_adc1);
}

/* ===== ADC1 acionado por trigger externo (TIM6 TRGO) + DMA circular ===== */
static void MX_ADC1_Triggered_DMA_Init(void)
{
  __HAL_RCC_ADC_CLK_ENABLE();

  hadc1.Instance = ADC1;
  hadc1.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV1;     // exemplo; pode variar
  hadc1.Init.Resolution = ADC_RESOLUTION_12B;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;

  // Importante: conversão NÃO contínua, porque quem manda é o trigger.
  hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc1.Init.ContinuousConvMode = DISABLE;
  hadc1.Init.DiscontinuousConvMode = DISABLE;

  // Trigger externo: TIM6 TRGO (Update)
  hadc1.Init.ExternalTrigConv = ADC_EXTERNALTRIG_T6_TRGO;
  hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_RISING;

  hadc1.Init.DMAContinuousRequests = ENABLE;  // para DMA circular fluido
  hadc1.Init.Overrun = ADC_OVR_DATA_OVERWRITTEN;
```

```
    if (HAL_ADC_Init(&hadc1) != HAL_OK) Error_Handler();

    // Configure 1 canal (ex.: ADC_CHANNEL_0). Ajuste conforme seu hardware.
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = ADC_CHANNEL_0;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLINGTIME_160CYCLES_5;   // escolha segura para
fontes altas
    if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK) Error_Handler();

    // Link DMA ao ADC (normalmente gerado pelo CubeMX)
    // Aqui montamos um exemplo direto:
    hdma_adc1.Instance = DMA1_Channel1;
    hdma_adc1.Init.Request = DMA_REQUEST_ADC1;
    hdma_adc1.Init.Direction = DMA_PERIPH_TO_MEMORY;
    hdma_adc1.Init.PeriphInc = DMA_PINC_DISABLE;
    hdma_adc1.Init.MemInc = DMA_MINC_ENABLE;
    hdma_adc1.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
    hdma_adc1.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
    hdma_adc1.Init.Mode = DMA_CIRCULAR;
    hdma_adc1.Init.Priority = DMA_PRIORITY_HIGH;

    if (HAL_DMA_Init(&hdma_adc1) != HAL_OK) Error_Handler();
    __HAL_LINKDMA(&hadc1, DMA_Handle, hdma_adc1);
}

/* Callbacks do ADC (meio e cheio) */
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
```

```
{
   if (hadc->Instance == ADC1) adc_half_ready = 1;
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
   if (hadc->Instance == ADC1) adc_full_ready = 1;
}

/* ===== Inicializa e inicia o “osciloscópio” ===== */
void scope_start(uint32_t sample_rate_hz)
{
   MX_DMA_Init_For_ADC1();
   MX_TIM6_TRGO_Init(sample_rate_hz);
   MX_ADC1_Triggered_DMA_Init();

   adc_half_ready = 0;
   adc_full_ready = 0;

   // Inicia ADC em DMA (circular). Ele só converte quando o timer gerar TRGO.
   if (HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buf, ADC_BUF_LEN) != HAL_OK)
Error_Handler();

   // Inicia o timer (gerador de trigger)
   if (HAL_TIM_Base_Start(&htim6) != HAL_OK) Error_Handler();
}

/* ===== Ajuste de taxa em runtime (1 kHz, 2 kHz, etc.) =====
```

```
  Estratégia robusta: parar timer -> atualizar ARR/PSC -> gerar update -> voltar.
  ADC+DMA pode continuar armado; sem triggers não há novas amostras.
*/
void scope_set_sample_rate(uint32_t sample_rate_hz)
{
   // Para de gerar triggers
   HAL_TIM_Base_Stop(&htim6);

   // Recalcula PSC/ARR mantendo fCNT=1MHz
   uint32_t fcnt = 1000000UL;
   uint32_t psc = (g_tim_clk_hz / fcnt) - 1UL;
   uint32_t arr = (fcnt / sample_rate_hz) - 1UL;

   __HAL_TIM_SET_PRESCALER(&htim6, psc);
   __HAL_TIM_SET_AUTORELOAD(&htim6, arr);
   __HAL_TIM_SET_COUNTER(&htim6, 0);

   // Força atualização dos registradores
   __HAL_TIM_GENERATE_EVENT(&htim6, TIM_EVENTSOURCE_UPDATE);

   // Volta a gerar triggers
   HAL_TIM_Base_Start(&htim6);
}

/* ===== Exemplo de uso no main ===== */
int main(void)
{
   HAL_Init();
```

```
  SystemClock_Config();

  // Inicia em 1 kHz
  scope_start(1000);

  while (1)
  {
    // Processa blocos prontos (metade e final)
    if (adc_half_ready)
    {
      adc_half_ready = 0;
      // Processar adc_buf[0 .. ADC_BUF_LEN/2 - 1]
    }

    if (adc_full_ready)
    {
      adc_full_ready = 0;
      // Processar adc_buf[ADC_BUF_LEN/2 .. ADC_BUF_LEN - 1]
    }

    // Exemplo: trocar taxa (como osciloscópio)
    // (Aqui é apenas ilustrativo: você faria isso por botão/menu)
    // scope_set_sample_rate(2000);  // 2 kHz
  }
}
```

3. O que este exemplo ensina (do jeito “osciloscópio”)

A taxa de aquisição não depende do “clock do ADC”; ela é controlada por um trigger preciso (TIM6 TRGO).

O ADC fica “armado” e converte exatamente 1 amostra por trigger.

O DMA circular garante que você sempre tenha um buffer contínuo e possa processar por meia-cheia / cheia.

Mudar a taxa é só recalcular PSC/ARR do timer.

4. Observações importantes de robustez

Para fontes de alta impedância, mantenha SamplingTime alto (como no capítulo do ADC).

Se você aumentar muito a taxa, confirme que:

o tempo de conversão do ADC + sampling time cabe entre triggers,

o DMA e seu processamento conseguem acompanhar (senão você “perde” no nível do aplicativo).

Se quiser uma troca de taxa “sem nenhum degrau”, você pode aplicar a mudança somente quando receber adc_full_ready (fim de bloco), para alinhar com a tela/escala do osciloscópio.

Capítulo 7.2 — Medição de frequência e duty-cycle com Timer (Input Capture) e sincronismo “estilo osciloscópio” (HAL)

1. Objetivo

Além de “capturar forma de onda” com ADC, um osciloscópio automotivo/industrial normalmente precisa medir:

frequência (Hz)

período (s)

duty-cycle (%)

largura de pulso (Ton/Toff)

A forma mais precisa de medir isso no STM32 é usando Timer + Input Capture, porque:

a medição acontece em hardware,

com resolução definida pelo clock do timer (ex.: 1 MHz → 1 µs),

com baixa carga de CPU.

---

2. Como o Input Capture funciona (passo a passo)

O timer está contando livremente (CNT).

Chega uma borda no pino do canal (ex.: TIM3_CH1).

O hardware “congela” o valor atual do contador em um registrador CCRx.

Opcionalmente gera interrupção para a CPU ler esse valor.

A diferença entre capturas consecutivas fornece tempo entre eventos.

Conceito-chave: o timer vira um “relógio” e cada borda vira um timestamp.

---

3. Medindo frequência/período (1 canal, bordas de subida)

3.1 Estratégia

Capturar duas bordas de subida consecutivas.

Período = diferença (com tratamento de overflow).

Frequência = f=f_CNT/Δticks

3.2 Exemplo completo (TIM3_CH1, 1 MHz)

Ajuste apenas os pinos AF no CubeMX (TIM3_CH1 em um pino válido).

```
#include "stm32g0xx_hal.h"

TIM_HandleTypeDef htim3;

volatile uint32_t g_last_rise = 0;
volatile uint32_t g_period_ticks = 0;
volatile uint8_t  g_period_ready = 0;

static void Error_Handler(void)
{
  __disable_irq();
```

```
    while (1) { }
}

static void MX_TIM3_IC_Freq_Init_1MHz(void)
{
    __HAL_RCC_TIM3_CLK_ENABLE();

    // fTIM assumido = 64 MHz -> PSC=63 => fCNT=1 MHz (1 us por tick)
    htim3.Instance = TIM3;
    htim3.Init.Prescaler = 63;
    htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim3.Init.Period = 0xFFFF;
    htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

    if (HAL_TIM_IC_Init(&htim3) != HAL_OK) Error_Handler();

    TIM_IC_InitTypeDef sIC = {0};
    sIC.ICPolarity  = TIM_INPUTCHANNELPOLARITY_RISING;
    sIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
    sIC.ICPrescaler = TIM_ICPSC_DIV1;
    sIC.ICFilter    = 2; // pequeno filtro contra ruído

    if (HAL_TIM_IC_ConfigChannel(&htim3, &sIC, TIM_CHANNEL_1) != HAL_OK)
        Error_Handler();

    HAL_NVIC_SetPriority(TIM3_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(TIM3_IRQn);
```

```
}

void TIM3_IRQHandler(void) { HAL_TIM_IRQHandler(&htim3); }

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
   if (htim->Instance == TIM3 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
   {
      uint32_t now = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);

      // delta com overflow em 16 bits
      uint32_t dt = (now >= g_last_rise) ? (now - g_last_rise) : ((0x10000UL - g_last_rise)
+ now);

      g_last_rise = now;
      g_period_ticks = dt;
      g_period_ready = 1;
   }
}

void freq_measure_start(void)
{
   MX_TIM3_IC_Freq_Init_1MHz();
   HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_1);
}
```

Uso (no loop principal):

```
if (g_period_ready)
{
```

```
    g_period_ready = 0;

    // fCNT=1MHz => período em us = g_period_ticks
    float period_s = (float)g_period_ticks / 1000000.0f;
    float freq_hz  = 1000000.0f / (float)g_period_ticks;
}
```

---

4. Medindo duty-cycle (2 capturas: subida e descida)

Há dois caminhos clássicos:

Caminho A (mais didático): alternar borda no mesmo canal

Captura subida → salva t_rise

Troca polaridade para descida

Captura descida → salva t_fall

Troca polaridade para subida

Ton = t_fall - t_rise

Período = t_rise(next) - t_rise

Funciona bem e é simples de explicar.

Exemplo completo: duty medindo Ton e período (TIM3_CH1)

```
#include "stm32g0xx_hal.h"

TIM_HandleTypeDef htim3;

typedef enum { WAIT_RISE = 0, WAIT_FALL } cap_state_t;

volatile cap_state_t g_state = WAIT_RISE;
volatile uint32_t g_t_rise = 0;
volatile uint32_t g_t_fall = 0;
volatile uint32_t g_t_rise2 = 0;
```

```
volatile uint32_t g_period_ticks = 0;
volatile uint32_t g_ton_ticks = 0;
volatile uint8_t  g_duty_ready = 0;

static void set_ic_polarity_rising(void)
{
   TIM_IC_InitTypeDef sIC = {0};
   sIC.ICPolarity  = TIM_INPUTCHANNELPOLARITY_RISING;
   sIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
   sIC.ICPrescaler = TIM_ICPSC_DIV1;
   sIC.ICFilter    = 2;
   HAL_TIM_IC_ConfigChannel(&htim3, &sIC, TIM_CHANNEL_1);
}

static void set_ic_polarity_falling(void)
{
   TIM_IC_InitTypeDef sIC = {0};
   sIC.ICPolarity  = TIM_INPUTCHANNELPOLARITY_FALLING;
   sIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
   sIC.ICPrescaler = TIM_ICPSC_DIV1;
   sIC.ICFilter    = 2;
   HAL_TIM_IC_ConfigChannel(&htim3, &sIC, TIM_CHANNEL_1);
}

static uint32_t diff16(uint32_t a, uint32_t b)
{
   return (a >= b) ? (a - b) : ((0x10000UL - b) + a);
```

```
}

static void MX_TIM3_IC_Duty_Init_1MHz(void)
{
  __HAL_RCC_TIM3_CLK_ENABLE();

  htim3.Instance = TIM3;
  htim3.Init.Prescaler = 63;            // 1MHz
  htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim3.Init.Period = 0xFFFF;
  htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

  if (HAL_TIM_IC_Init(&htim3) != HAL_OK) Error_Handler();

  set_ic_polarity_rising();

  HAL_NVIC_SetPriority(TIM3_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(TIM3_IRQn);
}

void TIM3_IRQHandler(void) { HAL_TIM_IRQHandler(&htim3); }

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM3 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
  {
    uint32_t now = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
```

```
if (g_state == WAIT_RISE)
{
    // primeira subida
    g_t_rise = now;
    g_state = WAIT_FALL;
    set_ic_polarity_falling();
}
else
{
    // descida
    g_t_fall = now;

    // agora esperamos a próxima subida para fechar período
    g_state = WAIT_RISE;
    set_ic_polarity_rising();

    // Ton já pode ser calculado
    g_ton_ticks = diff16(g_t_fall, g_t_rise);

    // Para período, precisamos do próximo rise: guardamos e calculamos na
próxima subida.
    // Truque simples: reutilizar g_t_rise como “rise anterior” e, no próximo rise,
calcular.
    // Aqui, marcamos que Ton está pronto, e período será atualizado quando vier o
próximo rise.
}

// Quando a próxima subida ocorrer, calculamos período usando o rise anterior.
```

```
        // Implementação: no próximo WAIT_RISE (subida), se já temos g_t_fall válido, calculamos período.
        if (g_state == WAIT_FALL)
        {
            // nada
        }
        else
        {
            // acabou de capturar FALL e voltamos a WAIT_RISE
            // período será calculado na próxima subida
        }
    }
}

// Complemento: calcular período na borda de subida seguinte
// Forma simples: usar uma flag "have_ton" e calcular período no próximo rise.
volatile uint8_t g_have_ton = 0;

void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim); // já declarado acima
```

Para deixar o exemplo completo e limpo, use esta versão do callback (substitui a anterior):

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        uint32_t now = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);

        if (g_state == WAIT_RISE)
        {
```

```
        if (g_have_ton)
        {
            // agora temos rise anterior (g_t_rise) e rise atual (now) => período
            g_t_rise2 = now;
            g_period_ticks = diff16(g_t_rise2, g_t_rise);
            g_duty_ready = 1;
            g_have_ton = 0; // consumimos a medição completa
        }

        g_t_rise = now;          // atualiza rise base
        g_state = WAIT_FALL;
        set_ic_polarity_falling();
    }
    else
    {
        // FALL
        g_t_fall = now;
        g_ton_ticks = diff16(g_t_fall, g_t_rise);
        g_have_ton = 1;

        g_state = WAIT_RISE;
        set_ic_polarity_rising();
    }
  }
}
```

Inicialização e start:

```
void duty_measure_start(void)
{
```

```
    g_state = WAIT_RISE;
    g_have_ton = 0;
    g_duty_ready = 0;

    MX_TIM3_IC_Duty_Init_1MHz();
    HAL_TIM_IC_Start_IT(&htim3, TIM_CHANNEL_1);
}
```

Uso:

```
if (g_duty_ready)
{
    g_duty_ready = 0;

    float period_s = (float)g_period_ticks / 1000000.0f;
    float ton_s    = (float)g_ton_ticks    / 1000000.0f;

    float freq_hz  = 1.0f / period_s;
    float duty_pct = (ton_s / period_s) * 100.0f;
}
```

---

5. “Sincronismo estilo osciloscópio”: alinhando aquisição ADC com evento digital

Em instrumentos, é comum sincronizar a aquisição (ADC) com um evento (borda). Existem duas abordagens:

Abordagem 1 (simples, funciona muito bem)

- Timer de trigger do ADC roda em frequência fixa (1 kHz, 2 kHz…)
- Input Capture detecta a borda e você “marca” onde isso ocorreu no tempo
- Você usa a marca para desenhar trigger na tela e alinhar janela

Essa abordagem é a mais fácil e não depende de cascatas internas.

Abordagem 2 (hardware trigger real)

Um evento do timer (ou uma borda externa) inicia/arma o timer de trigger do ADC

ADC só passa a amostrar após o evento (mais complexo)

No STM32G0 dá para fazer triggers internos, mas para manter o capítulo independente e estável com HAL, a abordagem 1 é a recomendada.

---

6. Integração prática com o exemplo anterior (ADC+DMA circular + timer TRGO)

Você já tem:

TIM6 gerando TRGO para ADC (taxa ajustável)

DMA circular preenchendo adc_buf[]

Agora você adiciona:

TIM3 Input Capture medindo frequência/duty do sinal digital (ex.: ignição, PWM de atuador)

Usa essas métricas para:

mostrar frequência/duty no display

ajustar automaticamente a taxa do ADC (ex.: se sinal é rápido, aumentar sample rate)

Exemplo de "auto-range" didático:

```
void scope_autoset_rate_from_freq(float freq_hz)
{
   if (freq_hz < 50.0f)       scope_set_sample_rate(1000);   // 1 kHz
   else if (freq_hz < 200.0f) scope_set_sample_rate(2000);   // 2 kHz
   else if (freq_hz < 1000.0f)scope_set_sample_rate(5000);   // 5 kHz
   else                       scope_set_sample_rate(10000);  // 10 kHz
}
```

---

7. Checklist de especialista (Input Capture)

escolher um clock do timer conveniente (1 MHz facilita)

tratar overflow do contador (diff16)

usar filtro (ICFilter) quando sinal é ruidoso

medir período com bordas iguais (rise-rise)

medir duty alternando polaridade (rise/fall) e fechando período no próximo rise

manter callback curto (apenas cálculo e flags)

Capítulo 7.3 — Pré-trigger e Pós-trigger com ADC + DMA circular (osciloscópio "de verdade"), do zero (HAL)

A ideia de um osciloscópio é simples: você quer mostrar amostras antes do evento de disparo (pré-trigger) e amostras depois (pós-trigger), sem parar o ADC. Para isso, usamos:

ADC em DMA circular (buffer sempre sendo preenchido)

um trigger digital (borda) que marca "onde" no buffer o evento aconteceu

uma rotina que congela uma janela (cópia) do buffer circular para um buffer "de exibição" já alinhado (pré/pós)

Este capítulo é independente (código completo e autocontido).

---

1) Conceito: buffer circular e "ponteiro de escrita"

Com DMA circular, o buffer adc_ring[] é escrito continuamente e "dá a volta". O ponto crucial é saber qual índice está sendo escrito agora.

No STM32, o DMA tem o registrador NDTR (número de transferências restantes). Com isso:

write_index = (BUF_LEN - NDTR) % BUF_LEN

Esse write_index aponta o "próximo" local onde o DMA vai escrever (ou o progresso atual). A partir dele, dá para determinar onde está o "passado" e o "futuro" dentro do ring.

---

2) Estratégia de trigger

Para manter o capítulo universal e independente, usaremos GPIO EXTI como trigger (borda de subida/descida), porque serve para qualquer sinal digital. Ao ocorrer o trigger:

capturamos trigger_index (posição aproximada no ring)

entramos no estado “armed → triggered”

esperamos acumular POST_SAMPLES novas amostras

copiamos a janela completa (PRE + POST) para adc_view[]

---

3) Exemplo completo: ADC + DMA circular + Timer trigger + EXTI trigger + janela pré/pós

3.1 Configurações do “osciloscópio”

Buffer circular: 320 amostras (exemplo)

Janela de exibição: 240 amostras (exemplo)

Pré-trigger: 80 amostras

Pós-trigger: 160 amostras

(80 + 160 = 240)

Você pode ajustar esses números conforme a UI do seu projeto.

```
#include "stm32g0xx_hal.h"

/* ===================== Parâmetros ===================== */
#define ADC_RING_LEN     320
#define VIEW_LEN         240
#define PRE_SAMPLES      80
#define POST_SAMPLES     (VIEW_LEN - PRE_SAMPLES)

/* Segurança */
#if (PRE_SAMPLES + POST_SAMPLES) != VIEW_LEN
#error "PRE_SAMPLES + POST_SAMPLES must equal VIEW_LEN"
#endif
```

```
/* ===================== Handles ===================== */
ADC_HandleTypeDef hadc1;
DMA_HandleTypeDef hdma_adc1;
TIM_HandleTypeDef htim6;

/* ===================== Buffers ===================== */
static uint16_t adc_ring[ADC_RING_LEN];
static uint16_t adc_view[VIEW_LEN];

/* ===================== Estado do trigger ===================== */
typedef enum {
   SCOPE_ARMED = 0,
   SCOPE_TRIGGERED_WAIT_POST,
   SCOPE_FRAME_READY
} scope_state_t;

static volatile scope_state_t g_state = SCOPE_ARMED;
static volatile uint16_t g_trig_index = 0;      // índice no ring no instante do trigger
static volatile uint16_t g_post_count = 0;      // quantas amostras pós-trigger já
passaram

/* Clock do timer (ajuste conforme seu clock real) */
static const uint32_t TIM_CLK_HZ = 64000000UL;

/* ===================== Prototipos ===================== */
static void SystemClock_Config(void);
static void Error_Handler(void);
```

```
static void MX_DMA_Init_ADC1(void);
static void MX_TIM6_TRGO_Init(uint32_t sample_rate_hz);
static void MX_ADC1_DMA_Triggered_Init(void);

static inline uint16_t dma_write_index(void);
static void scope_extract_frame(uint16_t trig_index);

static void scope_start(uint32_t sample_rate_hz);
static void scope_set_sample_rate(uint32_t sample_rate_hz);

/* ===================================================== */

static void Error_Handler(void)
{
  __disable_irq();
  while (1) { }
}
```

3.2 Timer TIM6 como “sample clock” do ADC (TRGO ajustável)

O timer define a taxa de amostragem (1 kHz, 2 kHz, etc.) com precisão.

```
static void MX_TIM6_TRGO_Init(uint32_t sample_rate_hz)
{
  __HAL_RCC_TIM6_CLK_ENABLE();

  /* Escolha fCNT=1 MHz para facilitar: 1 tick = 1 us */
  uint32_t fcnt = 1000000UL;
  uint32_t psc  = (TIM_CLK_HZ / fcnt) - 1UL;
```

```
    uint32_t arr  = (fcnt / sample_rate_hz) - 1UL;

    htim6.Instance = TIM6;
    htim6.Init.Prescaler = psc;
    htim6.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim6.Init.Period = arr;
    htim6.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

    if (HAL_TIM_Base_Init(&htim6) != HAL_OK) Error_Handler();

    TIM_MasterConfigTypeDef master = {0};
    master.MasterOutputTrigger = TIM_TRGO_UPDATE;   // TRGO em update
    master.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

    if (HAL_TIMEx_MasterConfigSynchronization(&htim6, &master) != HAL_OK)
Error_Handler();
}
```

3.3 ADC1 disparado pelo TIM6 TRGO + DMA circular

Aqui o ADC converte uma amostra por trigger do timer e o DMA guarda no ring.

```
static void MX_DMA_Init_ADC1(void)
{
    __HAL_RCC_DMA1_CLK_ENABLE();

    /* Ajuste IRQ conforme o canal real do ADC1 no seu CubeMX.
       Exemplo comum: DMA1_Channel1 para ADC1. */
    HAL_NVIC_SetPriority(DMA1_Channel1_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(DMA1_Channel1_IRQn);
```

```
}

void DMA1_Channel1_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_adc1);
}

static void MX_ADC1_DMA_Triggered_Init(void)
{
  __HAL_RCC_ADC_CLK_ENABLE();

  hadc1.Instance = ADC1;
  hadc1.Init.ClockPrescaler       = ADC_CLOCK_ASYNC_DIV1;
  hadc1.Init.Resolution           = ADC_RESOLUTION_12B;
  hadc1.Init.DataAlign            = ADC_DATAALIGN_RIGHT;
  hadc1.Init.ScanConvMode         = ADC_SCAN_DISABLE;
  hadc1.Init.ContinuousConvMode   = DISABLE;  // quem manda é o trigger
  hadc1.Init.DiscontinuousConvMode = DISABLE;

  /* Trigger externo vindo do TIM6 TRGO */
  hadc1.Init.ExternalTrigConv     = ADC_EXTERNALTRIG_T6_TRGO;
  hadc1.Init.ExternalTrigConvEdge  = ADC_EXTERNALTRIGCONVEDGE_RISING;

  hadc1.Init.DMAContinuousRequests = ENABLE;
  hadc1.Init.Overrun              = ADC_OVR_DATA_OVERWRITTEN;

  if (HAL_ADC_Init(&hadc1) != HAL_OK) Error_Handler();
```

```
  ADC_ChannelConfTypeDef ch = {0};
  ch.Channel      = ADC_CHANNEL_0;              // ajuste ao seu pino/canal
  ch.Rank         = ADC_REGULAR_RANK_1;
  ch.SamplingTime = ADC_SAMPLINGTIME_160CYCLES_5;  // seguro para fontes mais
"altas"
  if (HAL_ADC_ConfigChannel(&hadc1, &ch) != HAL_OK) Error_Handler();

  /* DMA config (exemplo direto; CubeMX normalmente gera isso) */
  hdma_adc1.Instance              = DMA1_Channel1;
  hdma_adc1.Init.Request          = DMA_REQUEST_ADC1;
  hdma_adc1.Init.Direction        = DMA_PERIPH_TO_MEMORY;
  hdma_adc1.Init.PeriphInc        = DMA_PINC_DISABLE;
  hdma_adc1.Init.MemInc           = DMA_MINC_ENABLE;
  hdma_adc1.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
  hdma_adc1.Init.MemDataAlignment    = DMA_MDATAALIGN_HALFWORD;
  hdma_adc1.Init.Mode             = DMA_CIRCULAR;
  hdma_adc1.Init.Priority         = DMA_PRIORITY_HIGH;

  if (HAL_DMA_Init(&hdma_adc1) != HAL_OK) Error_Handler();
  __HAL_LINKDMA(&hadc1, DMA_Handle, hdma_adc1);
}
```

3.4 "Ponteiro de escrita" do DMA (NDTR → índice)

```
static inline uint16_t dma_write_index(void)
{
  /* NDTR = quantas transferências faltam para completar o ciclo.
     write = quantas já foram escritas no ciclo atual. */
  uint32_t ndtr = __HAL_DMA_GET_COUNTER(&hdma_adc1);   // restante
```

```
    uint32_t w    = (ADC_RING_LEN - ndtr) % ADC_RING_LEN;
    return (uint16_t)w;
}
```

3.5 Trigger por EXTI: capturar índice e iniciar pós-trigger

Configure o pino do trigger como GPIO_MODE_IT_RISING ou GPIO_MODE_IT_FALLING no CubeMX.

Callback:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    /* Ajuste para o pino real de trigger */
    if (GPIO_Pin == GPIO_PIN_13)
    {
        if (g_state == SCOPE_ARMED)
        {
            /* Marca o índice no instante do trigger */
            g_trig_index = dma_write_index();
            g_post_count = 0;
            g_state = SCOPE_TRIGGERED_WAIT_POST;
        }
    }
}
```

3.6 Contar pós-trigger: usar callback de “meio/cheio” do ADC

Para saber quando já passaram POST_SAMPLES depois do trigger, precisamos contar amostras. Em DMA circular, os callbacks de ADC acontecem em meia-transferência e transferência completa. Cada um indica que uma quantidade conhecida de amostras avançou.

Half complete → avançou ADC_RING_LEN/2

Complete → avançou ADC_RING_LEN/2 (a outra metade)

```
static void scope_on_samples_advanced(uint16_t advanced)
{
    if (g_state == SCOPE_TRIGGERED_WAIT_POST)
    {
        uint32_t new_count = (uint32_t)g_post_count + advanced;
        if (new_count >= POST_SAMPLES)
        {
            /* Já temos pós-trigger suficiente: extrai frame */
            scope_extract_frame(g_trig_index);
            g_state = SCOPE_FRAME_READY;
        }
        else
        {
            g_post_count = (uint16_t)new_count;
        }
    }
}

void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance == ADC1)
        scope_on_samples_advanced(ADC_RING_LEN / 2);
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance == ADC1)
```

```
    scope_on_samples_advanced(ADC_RING_LEN / 2);
}
```

Esse método não conta “amostra por amostra”, mas funciona muito bem para osciloscópio: o “ponto de trigger” fica alinhado com erro máximo de até meia-metade do ring (dependendo do tamanho). Para precisão máxima, você pode usar um ring maior ou contar por leitura de NDTR em loop de baixa prioridade.

3.7 Extrair a janela pré/pós do ring para um buffer linear (adc_view)

Aqui está o “coração” do pré-trigger.

Definição:

trig_index é o índice aproximado do evento no ring.

Queremos que adc_view[PRE_SAMPLES] corresponda ao ponto do trigger (ou bem próximo).

Então o início da janela no ring é:

start = trig_index - PRE_SAMPLES (com wrap)

Cópia com wrap:

```
static void scope_extract_frame(uint16_t trig_index)
{
  /* Ponto onde começa a janela no ring */
  int32_t start = (int32_t)trig_index - (int32_t)PRE_SAMPLES;
  while (start < 0) start += ADC_RING_LEN;
  uint16_t s = (uint16_t)start;

  /* Copia VIEW_LEN amostras do ring para o view, com wrap */
  for (uint16_t i = 0; i < VIEW_LEN; i++)
  {
    adc_view[i] = adc_ring[s];
    s++;
    if (s >= ADC_RING_LEN) s = 0;
```

```
    }
}
```

Agora você tem um frame pronto:

adc_view[0 .. PRE_SAMPLES-1] = pré-trigger

adc_view[PRE_SAMPLES .. VIEW_LEN-1] = pós-trigger

---

4) Controle de aquisição: iniciar, rearmar e mudar taxa (como osciloscópio)

4.1 Start

```
static void scope_start(uint32_t sample_rate_hz)
{
    MX_DMA_Init_ADC1();
    MX_TIM6_TRGO_Init(sample_rate_hz);
    MX_ADC1_DMA_Triggered_Init();

    g_state = SCOPE_ARMED;

    if (HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_ring, ADC_RING_LEN) != HAL_OK)
        Error_Handler();

    if (HAL_TIM_Base_Start(&htim6) != HAL_OK)
        Error_Handler();
}
```

4.2 Rearmar para capturar o próximo evento

Depois que SCOPE_FRAME_READY ocorrer e você desenhar/usar adc_view, rearme:

```
static void scope_arm(void)
{
    g_state = SCOPE_ARMED;
}
```

4.3 Mudar taxa de aquisição (1 kHz, 2 kHz, etc.)

Só alteramos o timer — o ADC continua armado. Isso é exatamente o comportamento de um osciloscópio.

```
static void scope_set_sample_rate(uint32_t sample_rate_hz)
{
  HAL_TIM_Base_Stop(&htim6);

  uint32_t fcnt = 1000000UL;
  uint32_t psc  = (TIM_CLK_HZ / fcnt) - 1UL;
  uint32_t arr  = (fcnt / sample_rate_hz) - 1UL;

  __HAL_TIM_SET_PRESCALER(&htim6, psc);
  __HAL_TIM_SET_AUTORELOAD(&htim6, arr);
  __HAL_TIM_SET_COUNTER(&htim6, 0);
  __HAL_TIM_GENERATE_EVENT(&htim6, TIM_EVENTSOURCE_UPDATE);

  HAL_TIM_Base_Start(&htim6);
}
```

---

5) main() de demonstração (independente)

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  /* Inicia em 1 kHz */
  scope_start(1000);
```

```
  while (1)
  {
    if (g_state == SCOPE_FRAME_READY)
    {
      /* adc_view agora contém PRE+POST alinhado */
      /* Aqui você desenha na tela / calcula RMS / etc. */

      scope_arm();  // pronto para próximo trigger
    }

    /* Exemplo de mudança de taxa "tipo osciloscópio":
      - em um produto real, isso viria do menu/encoder/botão */
    // scope_set_sample_rate(2000);
  }
}
```

---

6) O que não pode faltar (dicas de especialista)

Trigger sem parar DMA: você nunca para o ADC; só "recorta" uma janela.

Pré-trigger depende do ring: se PRE_SAMPLES for grande, aumente ADC_RING_LEN.

Evite processamento pesado em callbacks: o callback só marca estado; a UI processa fora.

Confiabilidade do índice: NDTR dá o progresso do DMA; em aplicações críticas você pode "amostrar NDTR duas vezes" e usar a mais estável, ou reduzir o tamanho de meia-metade para diminuir erro de alinhamento.

Proteja a leitura do ring durante cópia: a cópia acontece enquanto DMA escreve. Como é um ring circular, o segredo é recortar uma janela que já "passou" (pré-trigger) e esperar pós-trigger suficiente antes de copiar. Por isso a extração é feita após POST_SAMPLES acumularem.

A seguir vai um capítulo independente, do zero, implementando trigger analógico por nível com histerese usando ADC Analog Watchdog (AWD) + ADC por Timer TRGO + DMA circular, e extraindo um frame pré/pós-trigger (estilo osciloscópio), tudo com HAL.

Capítulo 7.4 — Trigger analógico por nível com histerese (ADC Analog Watchdog) + ADC DMA circular + Timer (HAL)

1) Ideia (o "trigger analógico" de verdade)

Em vez de usar um pino digital (EXTI), o próprio ADC detecta quando o sinal cruza um nível. O recurso chama Analog Watchdog (AWD):

você define uma janela [LOW, HIGH]

o ADC gera um evento/interrupt quando a conversão fica fora dessa janela

com uma lógica simples, você cria histerese:

"dispara" quando sobe acima de HIGH

"rearma" quando desce abaixo de LOW

Isso imita o comportamento de trigger estável do osciloscópio (evita disparos múltiplos por ruído perto do limiar).

---

2) Código completo (cole e ajuste apenas canal/pino ADC e clocks se necessário)

Suposições típicas: TIM_CLK_HZ = 64 MHz, ADC1, DMA1_Channel1 para ADC1, TIM6 TRGO como trigger do ADC.

Ajuste ADC_CHANNEL_0 para o canal real do seu pino.

```
#include "stm32g0xx_hal.h"


/* ===================== Parâmetros do Osciloscópio ===================== */

#define ADC_RING_LEN     320

#define VIEW_LEN         240

#define PRE_SAMPLES      80
```

```
#define POST_SAMPLES     (VIEW_LEN - PRE_SAMPLES)

#if (PRE_SAMPLES + POST_SAMPLES) != VIEW_LEN
#error "PRE_SAMPLES + POST_SAMPLES must equal VIEW_LEN"
#endif

/* ===================== Taxa e clocks ===================== */
static const uint32_t TIM_CLK_HZ = 64000000UL;   // ajuste para o clock real do TIM6

/* ===================== Trigger analógico (histerese) ===================== */
/* ADC 12-bit: 0..4095. Ajuste níveis conforme seu sinal. */
#define TRIG_LEVEL_HIGH   2600   // dispara quando subir acima disso
#define TRIG_LEVEL_LOW    2400   // rearma quando cair abaixo disso (histerese)

typedef enum {
   TRIG_ARMED_RISING = 0,   // esperando cruzar HIGH (subindo)
   TRIG_LOCKED       = 1    // já disparou; espera cair abaixo de LOW para rearmar
} trig_state_t;

/* ===================== Handles ===================== */
ADC_HandleTypeDef hadc1;
DMA_HandleTypeDef hdma_adc1;
TIM_HandleTypeDef htim6;

/* ===================== Buffers ===================== */
static uint16_t adc_ring[ADC_RING_LEN];
static uint16_t adc_view[VIEW_LEN];
```

```
/* ===================== Estados ===================== */
typedef enum {
   SCOPE_ARMED = 0,
   SCOPE_TRIGGERED_WAIT_POST,
   SCOPE_FRAME_READY
} scope_state_t;

static volatile scope_state_t g_scope_state = SCOPE_ARMED;
static volatile trig_state_t  g_trig_state  = TRIG_ARMED_RISING;

static volatile uint16_t g_trig_index = 0;
static volatile uint16_t g_post_count = 0;

/* ===================== Prototipos ===================== */
static void SystemClock_Config(void);
static void Error_Handler(void);

static void MX_DMA_Init_ADC1(void);
static void MX_TIM6_TRGO_Init(uint32_t sample_rate_hz);
static void MX_ADC1_DMA_Triggered_Init(void);
static void MX_ADC1_AnalogWatchdog_Init(void);

static inline uint16_t dma_write_index(void);
static void scope_extract_frame(uint16_t trig_index);

static void scope_start(uint32_t sample_rate_hz);
static void scope_arm(void);
static void scope_set_sample_rate(uint32_t sample_rate_hz);
```

```
static void scope_on_samples_advanced(uint16_t advanced);

/* ===================== Erro ===================== */
static void Error_Handler(void)
{
  __disable_irq();
  while (1) { }
}

/* ===================== DMA: init + IRQ ===================== */
static void MX_DMA_Init_ADC1(void)
{
  __HAL_RCC_DMA1_CLK_ENABLE();

  /* ADC1 costuma usar DMA1_Channel1 em muitos projetos G0 (ajuste se
necessário). */
  HAL_NVIC_SetPriority(DMA1_Channel1_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(DMA1_Channel1_IRQn);
}

void DMA1_Channel1_IRQHandler(void)
{
  HAL_DMA_IRQHandler(&hdma_adc1);
}

/* ===================== Timer TIM6: TRGO em update (taxa ajustável)
===================== */
static void MX_TIM6_TRGO_Init(uint32_t sample_rate_hz)
```

```
{
    __HAL_RCC_TIM6_CLK_ENABLE();

    /* fCNT = 1 MHz para facilitar: 1 tick = 1 us */
    uint32_t fcnt = 1000000UL;
    uint32_t psc  = (TIM_CLK_HZ / fcnt) - 1UL;
    uint32_t arr  = (fcnt / sample_rate_hz) - 1UL;

    htim6.Instance = TIM6;
    htim6.Init.Prescaler = psc;
    htim6.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim6.Init.Period = arr;
    htim6.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;

    if (HAL_TIM_Base_Init(&htim6) != HAL_OK) Error_Handler();

    TIM_MasterConfigTypeDef master = {0};
    master.MasterOutputTrigger = TIM_TRGO_UPDATE;
    master.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;

    if (HAL_TIMEx_MasterConfigSynchronization(&htim6, &master) != HAL_OK)
Error_Handler();
}

/* ===================== ADC1: triggerado por TIM6 TRGO + DMA circular
===================== */
static void MX_ADC1_DMA_Triggered_Init(void)
{
    __HAL_RCC_ADC_CLK_ENABLE();
```

```
hadc1.Instance = ADC1;
hadc1.Init.ClockPrescaler       = ADC_CLOCK_ASYNC_DIV1;
hadc1.Init.Resolution           = ADC_RESOLUTION_12B;
hadc1.Init.DataAlign            = ADC_DATAALIGN_RIGHT;
hadc1.Init.ScanConvMode         = ADC_SCAN_DISABLE;
hadc1.Init.ContinuousConvMode    = DISABLE;    // trigger externo define a taxa
hadc1.Init.DiscontinuousConvMode = DISABLE;

/* Trigger externo do ADC = TIM6 TRGO */
hadc1.Init.ExternalTrigConv      = ADC_EXTERNALTRIG_T6_TRGO;
hadc1.Init.ExternalTrigConvEdge  = ADC_EXTERNALTRIGCONVEDGE_RISING;

hadc1.Init.DMAContinuousRequests = ENABLE;
hadc1.Init.Overrun               = ADC_OVR_DATA_OVERWRITTEN;

if (HAL_ADC_Init(&hadc1) != HAL_OK) Error_Handler();

ADC_ChannelConfTypeDef ch = {0};
ch.Channel      = ADC_CHANNEL_0;                // AJUSTE para seu canal
ch.Rank         = ADC_REGULAR_RANK_1;
ch.SamplingTime = ADC_SAMPLINGTIME_160CYCLES_5;  // seguro para fontes altas
if (HAL_ADC_ConfigChannel(&hadc1, &ch) != HAL_OK) Error_Handler();

/* DMA config (exemplo direto) */
hdma_adc1.Instance              = DMA1_Channel1;
hdma_adc1.Init.Request          = DMA_REQUEST_ADC1;
hdma_adc1.Init.Direction        = DMA_PERIPH_TO_MEMORY;
```

```
    hdma_adc1.Init.PeriphInc          = DMA_PINC_DISABLE;
    hdma_adc1.Init.MemInc             = DMA_MINC_ENABLE;
    hdma_adc1.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
    hdma_adc1.Init.MemDataAlignment    = DMA_MDATAALIGN_HALFWORD;
    hdma_adc1.Init.Mode               = DMA_CIRCULAR;
    hdma_adc1.Init.Priority           = DMA_PRIORITY_HIGH;

    if (HAL_DMA_Init(&hdma_adc1) != HAL_OK) Error_Handler();
    __HAL_LINKDMA(&hadc1, DMA_Handle, hdma_adc1);
}

/* ===================== ADC Analog Watchdog (AWD) + IRQ
===================== */
static void MX_ADC1_AnalogWatchdog_Init(void)
{
    /* Habilita IRQ do ADC (para receber evento do watchdog) */
    HAL_NVIC_SetPriority(ADC1_IRQn, 2, 0);
    HAL_NVIC_EnableIRQ(ADC1_IRQn);

    ADC_AnalogWDGConfTypeDef awd = {0};
    awd.WatchdogMode  = ADC_ANALOGWATCHDOG_SINGLE_REG;  // vigia o canal
regular
    awd.Channel       = ADC_CHANNEL_0;                // mesmo canal do ADC
    awd.ITMode        = ENABLE;
    awd.HighThreshold = TRIG_LEVEL_HIGH;
    awd.LowThreshold  = TRIG_LEVEL_LOW;

    if (HAL_ADC_AnalogWDGConfig(&hadc1, &awd) != HAL_OK) Error_Handler();
}
```

```
void ADC1_IRQHandler(void)
{
   HAL_ADC_IRQHandler(&hadc1);
}

/* ===================== Ponteiro de escrita do DMA via NDTR
===================== */
static inline uint16_t dma_write_index(void)
{
   uint32_t ndtr = __HAL_DMA_GET_COUNTER(&hdma_adc1);
   uint32_t w    = (ADC_RING_LEN - ndtr) % ADC_RING_LEN;
   return (uint16_t)w;
}

/* ===================== Extrair frame pré/pós do ring ===================== */
static void scope_extract_frame(uint16_t trig_index)
{
   int32_t start = (int32_t)trig_index - (int32_t)PRE_SAMPLES;
   while (start < 0) start += ADC_RING_LEN;
   uint16_t s = (uint16_t)start;

   for (uint16_t i = 0; i < VIEW_LEN; i++)
   {
      adc_view[i] = adc_ring[s];
      s++;
      if (s >= ADC_RING_LEN) s = 0;
   }
```

```
}

/* ===================== Contar pós-trigger por avanço de amostras (HT/TC)
===================== */
static void scope_on_samples_advanced(uint16_t advanced)
{
    if (g_scope_state == SCOPE_TRIGGERED_WAIT_POST)
    {
        uint32_t new_count = (uint32_t)g_post_count + advanced;
        if (new_count >= POST_SAMPLES)
        {
            scope_extract_frame(g_trig_index);
            g_scope_state = SCOPE_FRAME_READY;
        }
        else
        {
            g_post_count = (uint16_t)new_count;
        }
    }
}

void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance == ADC1)
        scope_on_samples_advanced(ADC_RING_LEN / 2);
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
```

```
{
   if (hadc->Instance == ADC1)
      scope_on_samples_advanced(ADC_RING_LEN / 2);
}

/* ===================== Trigger analógico: callback do Watchdog
===================== */
/* HAL chama quando o valor sai da janela [LOW..HIGH]. */
void HAL_ADC_LevelOutOfWindowCallback(ADC_HandleTypeDef *hadc)
{
   if (hadc->Instance != ADC1)
      return;

   /* Valor atual (última conversão disponível) */
   uint16_t v = (uint16_t)HAL_ADC_GetValue(hadc);

   /* Histerese:
     - ARMADO: dispara quando v > HIGH
     - LOCKED: só rearma quando v < LOW
   */
   if (g_trig_state == TRIG_ARMED_RISING)
   {
      if (v > TRIG_LEVEL_HIGH)
      {
         if (g_scope_state == SCOPE_ARMED)
         {
            g_trig_index = dma_write_index();
            g_post_count = 0;
```

```
            g_scope_state = SCOPE_TRIGGERED_WAIT_POST;
        }
        g_trig_state = TRIG_LOCKED;
    }
  }
  else /* TRIG_LOCKED */
  {
    if (v < TRIG_LEVEL_LOW)
    {
        g_trig_state = TRIG_ARMED_RISING;
    }
  }
}

/* ===================== Controle do “osciloscópio” ===================== */
static void scope_arm(void)
{
  g_scope_state = SCOPE_ARMED;
  g_trig_state  = TRIG_ARMED_RISING;
}

static void scope_start(uint32_t sample_rate_hz)
{
  MX_DMA_Init_ADC1();
  MX_TIM6_TRGO_Init(sample_rate_hz);
  MX_ADC1_DMA_Triggered_Init();
  MX_ADC1_AnalogWatchdog_Init();
```

```
    scope_arm();

    if (HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_ring, ADC_RING_LEN) != HAL_OK)
        Error_Handler();

    if (HAL_TIM_Base_Start(&htim6) != HAL_OK)
        Error_Handler();
}

static void scope_set_sample_rate(uint32_t sample_rate_hz)
{
    HAL_TIM_Base_Stop(&htim6);

    uint32_t fcnt = 1000000UL;
    uint32_t psc  = (TIM_CLK_HZ / fcnt) - 1UL;
    uint32_t arr  = (fcnt / sample_rate_hz) - 1UL;

    __HAL_TIM_SET_PRESCALER(&htim6, psc);
    __HAL_TIM_SET_AUTORELOAD(&htim6, arr);
    __HAL_TIM_SET_COUNTER(&htim6, 0);
    __HAL_TIM_GENERATE_EVENT(&htim6, TIM_EVENTSOURCE_UPDATE);

    HAL_TIM_Base_Start(&htim6);
}

/* ===================== main() demonstrativo ===================== */
int main(void)
{
```

```
  HAL_Init();
  SystemClock_Config();

  /* Inicia como osciloscópio em 1 kHz */
  scope_start(1000);

  while (1)
  {
    if (g_scope_state == SCOPE_FRAME_READY)
    {
      /* adc_view contém PRE + POST alinhado:
        adc_view[PRE_SAMPLES] ~ ponto de trigger (aprox.) */

      /* Aqui você desenha na tela / calcula métricas / etc. */

      /* Rearma para próxima captura */
      scope_arm();
    }

    /* Exemplo (manual) de troca de taxa:
      Em produto real, isso vem do menu. */
    // scope_set_sample_rate(2000);
  }
}

/* ===================== Stub do clock (substitua pelo seu)
===================== */
static void SystemClock_Config(void)
```

```
{
  /* Use o SystemClock_Config gerado pelo CubeMX no seu projeto. */
}
```

---

3) Como ajustar para “trigger de descida” ou outro modo

O exemplo acima é trigger de subida com histerese:

dispara quando v > HIGH

rearma quando v < LOW

Para trigger de descida, basta inverter a lógica:

dispara quando v < LOW

rearma quando v > HIGH

---

4) Observações de especialista (curtas e importantes)

O AWD dispara quando sai da janela, então ele é perfeito para histerese (janela LOW/HIGH).

O “índice de trigger” é aproximado (é o índice do DMA no instante do evento). Para UI/diagnóstico isso é ótimo.

Você pode combinar isso com “nível de trigger” configurável (menu do osciloscópio) mudando TRIG_LEVEL_HIGH/LOW e reconfigurando o watchdog.

Aqui vai a função completa independente para ajustar nível de trigger + histerese em runtime, reconfigurando o Analog Watchdog do ADC (HAL), incluindo os cuidados para não gerar triggers espúrios durante a troca.

Você pode colar direto no mesmo arquivo do exemplo anterior.

```
/* Ajusta trigger analógico em runtime:
  - level_counts: nível central do trigger (0..4095 para 12-bit)
  - hyst_counts:  histerese em contagens (ex.: 50, 100, 200...)
  - mode_rising:  1 = trigger na subida (dispara acima do HIGH, rearma abaixo do LOW)
                  0 = trigger na descida (dispara abaixo do LOW, rearma acima do HIGH)
```

```
  Requer:
  - hadc1 já inicializado
  - ADC IRQ habilitada (ADC1_IRQn) e AWD configurável via
HAL_ADC_AnalogWDGConfig
*/
typedef enum {
   TRIG_MODE_RISING = 0,
   TRIG_MODE_FALLING
} trig_mode_t;

static volatile trig_mode_t g_trig_mode = TRIG_MODE_RISING;

/* Atualiza variáveis globais usadas no callback */
static volatile uint16_t g_trig_high = TRIG_LEVEL_HIGH;
static volatile uint16_t g_trig_low  = TRIG_LEVEL_LOW;

static uint16_t clamp_u16(int32_t x, uint16_t lo, uint16_t hi)
{
   if (x < (int32_t)lo) return lo;
   if (x > (int32_t)hi) return hi;
   return (uint16_t)x;
}

HAL_StatusTypeDef scope_set_trigger_level(uint16_t level_counts,
                                          uint16_t hyst_counts,
                                          trig_mode_t mode)
{
   /* 1) Calcula janela LOW/HIGH com clamp */
```

```
int32_t low  = (int32_t)level_counts - (int32_t)hyst_counts;
int32_t high = (int32_t)level_counts + (int32_t)hyst_counts;

uint16_t low_u  = clamp_u16(low,  0, 4095);
uint16_t high_u = clamp_u16(high, 0, 4095);

if (low_u >= high_u)
    return HAL_ERROR;

/* 2) Atualiza modo e janela global (usado pelo callback) */
g_trig_mode = mode;
g_trig_low  = low_u;
g_trig_high = high_u;

/* 3) Rearma estados do trigger para evitar disparos “fantasmas” */
g_scope_state = SCOPE_ARMED;
if (mode == TRIG_MODE_RISING)
    g_trig_state = TRIG_ARMED_RISING;
else
    g_trig_state = TRIG_ARMED_RISING; // vamos reutilizar a mesma enum (lógica
muda no callback)

/* 4) Reconfigura o Analog Watchdog no ADC.
      Para segurança, limpamos flags antes.
      (Em algumas famílias, o AWD já está ativo; reconfigurar é suficiente.) */
__HAL_ADC_CLEAR_FLAG(&hadc1, ADC_FLAG_AWD);

ADC_AnalogWDGConfTypeDef awd = {0};
```

```
    awd.WatchdogMode  = ADC_ANALOGWATCHDOG_SINGLE_REG;
    awd.Channel       = ADC_CHANNEL_0;    // ajuste se seu canal for outro
    awd.ITMode        = ENABLE;
    awd.HighThreshold = high_u;
    awd.LowThreshold  = low_u;

    return HAL_ADC_AnalogWDGConfig(&hadc1, &awd);
}
```

Agora ajuste o callback do watchdog para suportar os dois modos (subida/descida) usando g_trig_mode, g_trig_low e g_trig_high:

```
void HAL_ADC_LevelOutOfWindowCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance != ADC1)
        return;

    uint16_t v = (uint16_t)HAL_ADC_GetValue(hadc);

    if (g_trig_mode == TRIG_MODE_RISING)
    {
        /* Dispara quando subir acima do HIGH; rearma quando cair abaixo do LOW */
        if (g_trig_state == TRIG_ARMED_RISING)
        {
            if (v > g_trig_high)
            {
                if (g_scope_state == SCOPE_ARMED)
                {
                    g_trig_index = dma_write_index();
                    g_post_count = 0;
```

```
                g_scope_state = SCOPE_TRIGGERED_WAIT_POST;
            }
            g_trig_state = TRIG_LOCKED;
        }
    }
    else /* LOCKED */
    {
        if (v < g_trig_low)
        {
            g_trig_state = TRIG_ARMED_RISING;
        }
    }
}
else /* TRIG_MODE_FALLING */
{
    /* Dispara quando descer abaixo do LOW; rearma quando subir acima do HIGH */
    if (g_trig_state == TRIG_ARMED_RISING) // reaproveitamos o mesmo estado
“armed”
    {
        if (v < g_trig_low)
        {
            if (g_scope_state == SCOPE_ARMED)
            {
                g_trig_index = dma_write_index();
                g_post_count = 0;
                g_scope_state = SCOPE_TRIGGERED_WAIT_POST;
            }
            g_trig_state = TRIG_LOCKED;
```

```
        }
      }
      else /* LOCKED */
      {
        if (v > g_trig_high)
        {
          g_trig_state = TRIG_ARMED_RISING;
        }
      }
    }
}
```

Exemplo de uso (menu do osciloscópio)

Trigger central em 2500 contagens, histerese 100, modo subida:

```
scope_set_trigger_level(2500, 100, TRIG_MODE_RISING);
```

Trigger central em 1800 contagens, histerese 80, modo descida:

```
scope_set_trigger_level(1800, 80, TRIG_MODE_FALLING);
```

Uma função utilitária de conversão entre volts e contagens ADC ajuda a integrar a aquisição com menus, telas e ajustes de trigger em unidades de engenharia.

```
/* Conversão Volts <-> ADC counts (independente)
   - Para ADC 12-bit: counts 0..4095
   - vref: tensão de referência usada pelo ADC (ex.: 3.3f)
   Observação: Vin deve estar dentro de [0..vref] (ou do range permitido no pino). */


static float clamp_f(float x, float lo, float hi)
{
    if (x < lo) return lo;
    if (x > hi) return hi;
    return x;
```

```
}

static uint16_t adc_volts_to_counts(float vin, float vref, uint8_t bits)
{
    if (bits == 0 || bits > 16) return 0;

    vin  = clamp_f(vin, 0.0f, vref);
    float max_counts = (float)((1u << bits) - 1u);

    float c = (vin * max_counts) / vref;

    /* arredondamento */
    if (c < 0.0f) c = 0.0f;
    if (c > max_counts) c = max_counts;

    return (uint16_t)(c + 0.5f);
}

static float adc_counts_to_volts(uint16_t counts, float vref, uint8_t bits)
{
    if (bits == 0 || bits > 16) return 0.0f;

    uint32_t max_counts = (1u << bits) - 1u;
    if (counts > max_counts) counts = (uint16_t)max_counts;

    return ((float)counts * vref) / (float)max_counts;
}
```

```
/* Exemplo: configurar trigger por VOLTS
   - level_v: nível central em volts
   - hyst_v:  histerese em volts (± hyst_v)
   - bits: resolução do ADC (ex.: 12)
   - vref: referência (ex.: 3.3f) */
HAL_StatusTypeDef scope_set_trigger_level_volts(float level_v,
                                                float hyst_v,
                                                float vref,
                                                uint8_t bits,
                                                trig_mode_t mode)
{
    uint16_t level_c = adc_volts_to_counts(level_v, vref, bits);
    uint16_t hyst_c  = adc_volts_to_counts(hyst_v,  vref, bits);

    return scope_set_trigger_level(level_c, hyst_c, mode);
}

/* Exemplos de uso:
   // Trigger subida em 1.65V, histerese 0.05V, vref=3.3, 12-bit
   scope_set_trigger_level_volts(1.65f, 0.05f, 3.3f, 12, TRIG_MODE_RISING);

   // Converter um valor lido para volts
   float vin = adc_counts_to_volts(adc_ring[0], 3.3f, 12);
*/
```

Capítulo 8 — RTC, TAMP e Backup Registers no STM32G070RB (calendário, alarme, wakeup, tamper, referência e calibração) — HAL

Escopo deste capítulo: deixar você capaz de configurar e usar RTC (data/hora), alarmes, wakeup timer, timestamp, tamper e backup registers, entender clock de referência (LSE/LSI) e aplicar calibração digital.

---

1) O que é o RTC e por que ele existe

O RTC (Real-Time Clock) é um periférico dedicado a manter data e hora (calendário) com consumo muito baixo, inclusive quando o MCU está em modos de baixo consumo e/ou alimentado por VBAT (domínio de backup), dependendo do hardware e do modo.

O conjunto "RTC + TAMP + Backup Registers" permite:

manter calendário (hora, minutos, segundos; dia, mês, ano; dia da semana)

gerar Alarm A/B, Wakeup Timer e Timestamp

guardar dados pequenos em backup registers que sobrevivem a reset e, quando VBAT presente, sobrevivem a queda da alimentação principal

detectar violação física/lógica via Tamper e opcionalmente apagar o domínio de backup (proteção)

No STM32G070, o datasheet cita RTC + TAMP + 5 registradores de backup de 32 bits no domínio RTC.

---

2) Domínio de backup e clocks do RTC (LSE vs LSI)

2.1 O "domínio RTC"

RTC vive em um domínio separado do resto do chip: backup domain.

Ele não é resetado como o restante do MCU em várias condições; por isso, o software precisa saber quando:

inicializar de verdade (primeira vez)

apenas reaproveitar a configuração anterior (já rodando em VBAT/backup)

O padrão de projeto é:

escrever um “magic value” em um backup register

no boot, se o magic existe, não reconfigurar hora/data novamente (evita “zerar o relógio”).

2.2 Fonte de clock: LSE (32.768 kHz) e LSI

LSE (cristal 32.768 kHz) é a referência clássica para RTC com boa precisão.

LSI (oscilador interno) funciona sem cristal, mas tende a ser menos preciso e mais dependente de temperatura/variação.

O CubeMX normalmente gera o setup do RCC_BDCR e prescalers do RTC; neste capítulo nós mostramos também o caminho “manual” via HAL.

---

3) Recursos do RTC que você precisa dominar

Calendário (BCD interno): set/get de hora/data

Alarm A/B: evento em data/hora programada (pode gerar IRQ e acordar)

Wakeup timer: “tick” periódico independente do calendário (bom para acordar a cada 1 s, 10 s, 1 min…)

Timestamp: “carimbo” de data/hora quando ocorre um evento (pino timestamp, troca para VBAT e/ou tamper, dependendo da config)

TAMP (Tamper): detecção de evento (pino/condição) com filtros e possibilidade de apagar backup regs

Calibração digital (smooth calibration): ajuste fino do RTC (ppm)

Clock de referência / saída RTC: pode expor sinal (ex.: 1 Hz/512 Hz) em um pino RTC_OUT para calibração/diagnóstico (depende de roteamento do chip/pacote; CubeMX ajuda).

---

4) Código completo e independente (HAL)

A seguir um exemplo “fechado” em C com:

init do domínio backup + clock RTC (LSE com fallback)

init do RTC + “first boot” por backup register

set/get calendário

Alarm A

Wakeup timer periódico

Tamper 1 + timestamp

calibração digital (API)

leitura/escrita de backup registers

Observação: alguns símbolos (pinos RTC_OUT/TAMP1) dependem do seu package e do CubeMX. O código abaixo foca no periférico e na API; a seleção de pinos AF é feita no CubeMX.

```
#include "stm32g0xx_hal.h"

/* ===================== Config ===================== */
RTC_HandleTypeDef hrtc;

#define RTC_BKP_MAGIC_REG   RTC_BKP_DR0
#define RTC_BKP_MAGIC_VALUE 0x32F2A1B5u

/* Exemplo: use DR1 para armazenar uma flag do app */
#define RTC_BKP_APPFLAG_REG RTC_BKP_DR1

/* ===================== Prototipos ===================== */
static void SystemClock_Config(void);
static void Error_Handler(void);

static void RTC_BackupDomain_EnableAccess(void);
static void RTC_Clock_Select_LSE_WithFallback(void);

static void MX_RTC_Init_Common(void);
static void RTC_Set_DefaultDateTime(void);
static void RTC_Init_IfNeeded(void);

static void RTC_PrintLikeExample(void);
```

```
static void RTC_AlarmA_SetInSeconds(uint32_t seconds_from_now);
static void RTC_Wakeup_SetPeriodSeconds(uint16_t period_s);

static void RTC_Tamper1_EnableWithTimestamp(void);

static void RTC_Calibration_Apply(int32_t ppm);   // exemplo didático
static uint32_t RTC_BKP_Read(uint32_t reg);
static void     RTC_BKP_Write(uint32_t reg, uint32_t value);

/* ===================== Error ===================== */
static void Error_Handler(void)
{
    __disable_irq();
    while (1) { }
}

/* ===================== Backup domain access ===================== */
static void RTC_BackupDomain_EnableAccess(void)
{
    __HAL_RCC_PWR_CLK_ENABLE();
    HAL_PWR_EnableBkUpAccess(); // libera escrita em BDCR/RTC regs
}

/* ===================== Seleção de clock do RTC =====================
   Preferir LSE; se falhar, cair para LSI (opcional).
   Em muitos projetos, isso é gerado pelo CubeMX. */
static void RTC_Clock_Select_LSE_WithFallback(void)
```

```
{
  RTC_BackupDomain_EnableAccess();

  /* Habilita LSE */
  __HAL_RCC_LSE_CONFIG(RCC_LSE_ON);

  /* Espera estabilizar (timeout simples) */
  uint32_t t0 = HAL_GetTick();
  while (__HAL_RCC_GET_FLAG(RCC_FLAG_LSERDY) == RESET)
  {
    if ((HAL_GetTick() - t0) > 2000)
    {
      /* Fallback para LSI */
      __HAL_RCC_LSE_CONFIG(RCC_LSE_OFF);
      __HAL_RCC_LSI_ENABLE();
      while (__HAL_RCC_GET_FLAG(RCC_FLAG_LSIRDY) == RESET) { }

      __HAL_RCC_RTC_CONFIG(RCC_RTCCLKSOURCE_LSI);
      __HAL_RCC_RTC_ENABLE();
      return;
    }
  }

  /* LSE OK */
  __HAL_RCC_RTC_CONFIG(RCC_RTCCLKSOURCE_LSE);
  __HAL_RCC_RTC_ENABLE();
}
```

```
/* ===================== RTC init (prescalers, format) ===================== */
static void MX_RTC_Init_Common(void)
{
   hrtc.Instance = RTC;

   /* 24h + BCD interno (HAL trabalha com bin/BCD conforme parâmetro) */
   hrtc.Init.HourFormat = RTC_HOURFORMAT_24;

   /* Prescalers típicos para LSE 32.768k:
      Asynch=127, Synch=255 -> 1 Hz
      (CubeMX normalmente calcula isso.) */
   hrtc.Init.AsynchPrediv = 127;
   hrtc.Init.SynchPrediv  = 255;

   hrtc.Init.OutPut = RTC_OUTPUT_DISABLE;
   hrtc.Init.OutPutPolarity = RTC_OUTPUT_POLARITY_HIGH;
   hrtc.Init.OutPutType = RTC_OUTPUT_TYPE_OPENDRAIN;

   if (HAL_RTC_Init(&hrtc) != HAL_OK)
      Error_Handler();
}

/* ===================== Default date/time (primeiro boot)
===================== */
static void RTC_Set_DefaultDateTime(void)
{
   RTC_TimeTypeDef t = {0};
   RTC_DateTypeDef d = {0};
```

```
    /* 12:00:00 */
    t.Hours = 12;
    t.Minutes = 0;
    t.Seconds = 0;
    t.DayLightSaving = RTC_DAYLIGHTSAVING_NONE;
    t.StoreOperation = RTC_STOREOPERATION_RESET;

    /* 2026-01-01 (Quinta, por exemplo) */
    d.Year = 26;        // HAL usa 0..99
    d.Month = RTC_MONTH_JANUARY;
    d.Date = 1;
    d.WeekDay = RTC_WEEKDAY_THURSDAY;

    if (HAL_RTC_SetTime(&hrtc, &t, RTC_FORMAT_BIN) != HAL_OK) Error_Handler();
    if (HAL_RTC_SetDate(&hrtc, &d, RTC_FORMAT_BIN) != HAL_OK) Error_Handler();
}

/* ===================== Init condicional por backup reg =====================
*/
static void RTC_Init_IfNeeded(void)
{
    RTC_Clock_Select_LSE_WithFallback();
    MX_RTC_Init_Common();

    uint32_t magic = RTC_BKP_Read(RTC_BKP_MAGIC_REG);

    if (magic != RTC_BKP_MAGIC_VALUE)
```

```
  {
    /* Primeira inicialização */
    RTC_Set_DefaultDateTime();

    /* Marca como inicializado */
    RTC_BKP_Write(RTC_BKP_MAGIC_REG, RTC_BKP_MAGIC_VALUE);

    /* Exemplo: app flag */
    RTC_BKP_Write(RTC_BKP_APPFLAG_REG, 0x00000001u);
  }
}

/* ===================== BKP read/write ===================== */
static uint32_t RTC_BKP_Read(uint32_t reg)
{
  return HAL_RTCEx_BKUPRead(&hrtc, reg);
}

static void RTC_BKP_Write(uint32_t reg, uint32_t value)
{
  HAL_RTCEx_BKUPWrite(&hrtc, reg, value);
}

/* ===================== Get date/time (exemplo) ===================== */
static void RTC_PrintLikeExample(void)
{
  RTC_TimeTypeDef t;
  RTC_DateTypeDef d;
```

```
    HAL_RTC_GetTime(&hrtc, &t, RTC_FORMAT_BIN);
    HAL_RTC_GetDate(&hrtc, &d, RTC_FORMAT_BIN); // IMPORTANTE: ler Date após Time

    /* Aqui você faria printf/log. Mantemos independente. */
    (void)t;
    (void)d;
}

/* ===================== Alarm A (disparar em X segundos) =====================
   Exemplo didático: programa um alarme relativo.
   OBS: RTC alarme trabalha com hora/min/seg; aqui fazemos soma simples (sem calendário completo). */
static void RTC_AlarmA_SetInSeconds(uint32_t seconds_from_now)
{
    RTC_TimeTypeDef now;
    RTC_DateTypeDef date;

    HAL_RTC_GetTime(&hrtc, &now, RTC_FORMAT_BIN);
    HAL_RTC_GetDate(&hrtc, &date, RTC_FORMAT_BIN);

    uint32_t total = now.Hours * 3600u + now.Minutes * 60u + now.Seconds;
    total = (total + seconds_from_now) % (24u * 3600u);

    RTC_AlarmTypeDef alarm = {0};
    alarm.Alarm = RTC_ALARM_A;
```

```
    alarm.AlarmTime.Hours   = total / 3600u;
    alarm.AlarmTime.Minutes = (total % 3600u) / 60u;
    alarm.AlarmTime.Seconds = total % 60u;

    alarm.AlarmTime.SubSeconds = 0;
    alarm.AlarmTime.DayLightSaving = RTC_DAYLIGHTSAVING_NONE;
    alarm.AlarmTime.StoreOperation = RTC_STOREOPERATION_RESET;

    /* Ignora data/dia da semana (alarme diário por hora/min/seg) */
    alarm.AlarmMask = RTC_ALARMMASK_DATEWEEKDAY;
    alarm.AlarmSubSecondMask = RTC_ALARMSUBSECONDMASK_ALL;
    alarm.AlarmDateWeekDaySel = RTC_ALARMDATEWEEKDAYSEL_DATE;
    alarm.AlarmDateWeekDay = 1;

    if (HAL_RTC_SetAlarm_IT(&hrtc, &alarm, RTC_FORMAT_BIN) != HAL_OK)
        Error_Handler();
}

/* Callback do Alarm A */
void HAL_RTC_AlarmAEventCallback(RTC_HandleTypeDef *hrtc_cb)
{
    if (hrtc_cb->Instance == RTC)
    {
        /* Evento de alarme ocorreu:
           - acordar sistema
           - setar flag
           - etc. */
    }
```

```
}

/* IRQ Handler RTC (alarme/timestamp/wakeup/tamper compartilham dependendo da
família/startup) */
void RTC_TAMP_IRQHandler(void)
{
   HAL_RTCEx_TamperIRQHandler(&hrtc);
}

void RTC_IRQHandler(void)
{
   HAL_RTC_IRQHandler(&hrtc);
}

/* ===================== Wakeup Timer (periódico) =====================
  Bom para "tick" de baixa potência e acordar periodicamente. */
static void RTC_Wakeup_SetPeriodSeconds(uint16_t period_s)
{
   /* Desativa antes de reconfigurar */
   if (HAL_RTCEx_DeactivateWakeUpTimer(&hrtc) != HAL_OK)
      Error_Handler();

   /* Seleciona clock do wakeup:
     RTC_WAKEUPCLOCK_CK_SPRE_16BITS: usa 1 Hz (ck_spre) e contador 16-bit.
     Então: period_s = contador (1..65535) */
   if (HAL_RTCEx_SetWakeUpTimer_IT(&hrtc,
                                   period_s,
                                   RTC_WAKEUPCLOCK_CK_SPRE_16BITS) != HAL_OK)
```

```
        Error_Handler();
}

/* Callback do Wakeup */
void HAL_RTCEx_WakeUpTimerEventCallback(RTC_HandleTypeDef *hrtc_cb)
{
    if (hrtc_cb->Instance == RTC)
    {
        /* Executa ação periódica (ex.: log, medição, acordar UI) */
    }
}

/* ===================== Tamper + Timestamp ===================== */
static void RTC_Tamper1_EnableWithTimestamp(void)
{
    /* Habilita timestamp em evento de tamper (conceito do
RTC/TAMP) :contentReference[oaicite:12]{index=12} */

    RTC_TamperTypeDef tamp = {0};
    tamp.Tamper = RTC_TAMPER_1;

    /* Trigger: escolha conforme seu hardware (rising/falling). */
    tamp.Trigger = RTC_TAMPERTRIGGER_FALLINGEDGE;

    /* Filtro: reduz falsos triggers por ruído */
    tamp.Filter = RTC_TAMPERFILTER_4SAMPLE;

    /* Sampling freq para o filtro */
```

```
    tamp.SamplingFrequency = RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV32768;

    /* Precharge/discharge e pull-up dependem de como sua placa liga o pino */
    tamp.PrechargeDuration = RTC_TAMPERPRECHARGEDURATION_1RTCCLK;
    tamp.TamperPullUp = RTC_TAMPER_PULLUP_ENABLE;

    /* Timestamp on tamper */
    tamp.TimeStampOnTamperDetection =
RTC_TIMESTAMPONTAMPERDETECTION_ENABLE;

    /* Opcional: apagar backup regs ao detectar tamper */
    tamp.NoErase = RTC_TAMPER_ERASE_BACKUP_ENABLE; // proteção: apaga domínio
    // se quiser NÃO apagar: RTC_TAMPER_ERASE_BACKUP_DISABLE

    if (HAL_RTCEx_SetTamper_IT(&hrtc, &tamp) != HAL_OK)
        Error_Handler();
}

/* Callback do Tamper */
void HAL_RTCEx_Tamper1EventCallback(RTC_HandleTypeDef *hrtc_cb)
{
    if (hrtc_cb->Instance == RTC)
    {
        /* Tamper detectado.
           Se NoErase habilitado para erase, seus BKP regs podem ter sido apagados.
           Você pode:
           - registrar evento
           - disparar alarme
```

```
        - bloquear funcionalidades, etc.
      */

      /* Se timestamp-on-tamper estiver habilitado, você pode ler TS registers via HAL.
*/
      RTC_TimeTypeDef ts_t;
      RTC_DateTypeDef ts_d;
      HAL_RTCEx_GetTimeStamp(hrtc_cb, &ts_t, &ts_d, RTC_FORMAT_BIN);
      (void)ts_t; (void)ts_d;
   }
}

/* ===================== Calibração digital (Smooth Calibration)
=====================
  Ajuste fino do RTC para compensar erro do cristal (ppm).
  Guia de procedimento/ideia geral: medir erro e aplicar
CALR :contentReference[oaicite:13]{index=13} */
static void RTC_Calibration_Apply(int32_t ppm)
{
   /* ppm > 0: RTC está rápido -> precisamos “retardar”
      ppm < 0: RTC está lento -> precisamos “adiantar”

      O HAL expõe API de smooth calibration (veja
UM2319) :contentReference[oaicite:14]{index=14}

      A calibração do STM32 usa:
      - CALP: adicionar pulsos (para acelerar)
      - CALM: subtrair pulsos (para desacelerar)
      - janela de 32s/16s/8s (dependendo do modelo/config)
```

O cálculo exato depende da janela e do RM, então aqui mostramos um método prático:

você mede e ajusta iterativamente.

```
*/


/* Exemplo didático:
   - assumindo janela 32s padrão (muito comum)
   - ajuste “por passos” baseado em ppm medido
   Você pode mapear ppm->pulsos com mais precisão usando RM e o artigo
ST. :contentReference[oaicite:15]{index=15} */


uint32_t calp = RTC_SMOOTHCALIB_PLUSPULSES_RESET;
uint32_t calm = 0;


if (ppm > 0)
{
   /* RTC rápido: desacelerar -> usar CALM (remover pulsos) */
   /* Valor ilustrativo: clamp em 0..511 */
   int32_t n = ppm; // em um projeto real, converta ppm para pulsos
   if (n < 0) n = 0;
   if (n > 511) n = 511;
   calm = (uint32_t)n;
   calp = RTC_SMOOTHCALIB_PLUSPULSES_RESET;
}
else if (ppm < 0)
{
   /* RTC lento: acelerar -> CALP = add pulses, e CALM pequeno */
   calp = RTC_SMOOTHCALIB_PLUSPULSES_SET;
   int32_t n = -ppm;
```

```
        if (n < 0) n = 0;
        if (n > 511) n = 511;
        calm = (uint32_t)n;
    }

    if (HAL_RTCEx_SetSmoothCalib(&hrtc,
                                 RTC_SMOOTHCALIB_PERIOD_32SEC,
                                 calp,
                                 (uint32_t)calm) != HAL_OK)
    {
        Error_Handler();
    }
}

/* ===================== main ===================== */
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    RTC_Init_IfNeeded();

    /* Exemplos de uso */
    RTC_PrintLikeExample();

    /* Alarme em 10s */
    RTC_AlarmA_SetInSeconds(10);
```

```
    /* Wakeup a cada 1s */
    RTC_Wakeup_SetPeriodSeconds(1);

    /* Tamper + timestamp */
    RTC_Tamper1_EnableWithTimestamp();

    /* Exemplo: calibração (ppm medido externamente) */
    // RTC_Calibration_Apply(+12);  // RTC 12 ppm rápido
    // RTC_Calibration_Apply(-8);   // RTC 8 ppm lento

    while (1)
    {
        /* Aplicação */
    }
}

/* Stub: use o gerado pelo CubeMX */
static void SystemClock_Config(void)
{
}
```

---

5) Passo a passo do I/O de calendário (boas práticas)

Sempre leia GetTime() e em seguida GetDate() (o HAL e o hardware exigem essa ordem para “travar” coerência de leitura).

Use RTC_FORMAT_BIN no HAL para facilitar lógica; o RTC internamente é BCD, mas o HAL converte para você.

---

6) Alarmes A/B e Wakeup: quando usar cada um

Alarm A/B: eventos “no calendário” (ex.: 07:00 todo dia; ou daqui 10s como exemplo).

Wakeup Timer: eventos “periódicos” independentes do calendário (ex.: acordar a cada 1s, 30s, 5 min).

Ambos podem acordar o MCU dependendo do modo e da configuração do sistema/power.

---

7) Tamper e Backup Registers: comportamento e segurança

Backup regs são úteis para:

“magic” de inicialização do RTC

contadores persistentes

flags de reset reason / estado da aplicação

Tamper pode:

gerar evento e timestamp

opcionalmente apagar o domínio de backup (proteção contra extração/violação)

---

8) Clock de referência e calibração (como se faz “de verdade”)

O método clássico recomendado é:

habilitar uma saída de referência (ex.: 1 Hz ou 512 Hz) em RTC_OUT

medir com instrumento de referência por janela longa (reduz erro)

aplicar smooth calibration

repetir até ficar dentro do erro desejado

O cálculo de ppm→CALR depende da janela e do modelo (RM). No livro, a abordagem mais robusta é ensinar procedimento + API, e fornecer uma tabela/rotina auxiliar depois (opcional).

Checklist final (o leitor sai especialista se dominar)

diferenças LSE vs LSI e implicações de precisão

backup domain e “first init” via backup reg

set/get calendário com HAL e ordem correta

Alarm A/B e Wakeup timer com callbacks

tamper com filtro, timestamp e política de erase

calibração digital via HAL_RTCEx_SetSmoothCalib

uso de RTC_OUT como referência para medir e calibrar

Capítulo 9 — Independent Watchdog (IWDG) no STM32G070RB: teoria, configuração e exemplos completos (HAL)

1) O que é o IWDG e para que ele serve

O IWDG (Independent Watchdog) é um periférico de segurança que reseta o microcontrolador caso o firmware pare de "alimentá-lo" (refresh/kick) dentro de um tempo configurado.

Características essenciais (família STM32G0):

clock independente do sistema: usa o LSI ~32 kHz interno

continua funcionando mesmo em Stop e Standby

possui prescaler de 8 bits e downcounter de 12 bits

pode trabalhar em modo watchdog (reset por travamento) ou como timer de timeout

comportamento configurável por Option Bytes (HW/SW)

pode ser congelado em debug (dependendo de configuração)

Esses pontos constam no datasheet e no RM da família G0.

---

2) Por que "independente" é importante

Um watchdog baseado no clock principal pode falhar justamente quando o clock falha ou quando o firmware entra em um estado em que o clock é alterado inesperadamente. O IWDG:

não depende do PLL/HCLK

não depende de SysTick

não depende de timers comuns

Isso é exatamente o que você quer em mecanismos de segurança: algo que continue rodando “por fora” do resto do sistema.

---

3) Como o hardware funciona (modelo mental)

Pense no IWDG como:

um oscilador LSI (~32 kHz)

um divisor (prescaler) que reduz a taxa

um contador regressivo (12-bit, 0..4095)

quando chega em 0 → reset do MCU

quando você escreve a chave de refresh → contador recarrega e continua

Além disso, muitos STM32 oferecem “window mode”: existe uma janela (limite) onde o refresh é permitido. Isso evita refresh “cedo demais” (caso de bug onde o código fica preso em um loop rápido e continua chutando o watchdog, mascarando o problema).
A família G0 suporta watchdog window conforme o RM.

---

4) Como calcular o timeout do IWDG

O tempo de estouro (aproximado) depende de:

frequência do LSI (que tem tolerância)

prescaler

valor de reload (12-bit)

A relação típica é:

$T \approx ((Reload+1) \cdot Prescaler)/f_LSI$

Onde:

Reload vai de 0 a 4095

Prescaler é um divisor (ex.: 4, 8, 16, 32, 64, 128, 256)

f_LSI ~ 32 kHz (pode variar bastante)

Implicação prática: o timeout nunca é “milimetricamente exato”. Para segurança, dimensione com margem.

---

5) Quando usar IWDG (e quando não usar)

Use IWDG quando:

- o sistema precisa se recuperar sozinho de travamentos
- existe risco de deadlock, corrupção, loop infinito, travamento de task
- você tem requisitos de robustez (produto em campo)

Evite usar IWDG como “delay/timer normal” para precisão:

- o LSI varia com temperatura e lote
- o IWDG é ótimo para timeout, não para temporização precisa

---

6) Configuração via CubeMX / Option Bytes (entendendo sem depender deles)

Existem duas formas típicas de habilitar IWDG:

- Software: firmware inicia e configura o IWDG via registradores/HAL (mais comum)
- Hardware (Option Bytes): IWDG já inicia automaticamente no boot (mais “failsafe”)

No modo “hardware”, o firmware não pode “desabilitar” o IWDG e precisa viver com ele desde o reset.

---

7) Exemplos completos e independentes com HAL

Exemplo A — IWDG simples (reset se travar)

Objetivo

- configurar um watchdog com timeout ~2 s
- no loop principal, alimentar normalmente
- simular travamento parando de alimentar → MCU reseta

Código completo

```
#include "stm32g0xx_hal.h"


IWDG_HandleTypeDef hiwdg;
```

```
static void SystemClock_Config(void);
static void Error_Handler(void);

static void MX_IWDG_Init_Approx2s(void)
{
   // Timeout aproximado:
   // T ≈ (Reload+1)*Prescaler / fLSI
   // Supondo fLSI ~ 32000 Hz:
   // Escolha Prescaler=64, Reload=999:
   // T ≈ (1000*64)/32000 = 2.0 s

   hiwdg.Instance = IWDG;
   hiwdg.Init.Prescaler = IWDG_PRESCALER_64;
   hiwdg.Init.Reload = 999;
   hiwdg.Init.Window = IWDG_WINDOW_DISABLE; // sem janela (refresh sempre
permitido)

   if (HAL_IWDG_Init(&hiwdg) != HAL_OK)
      Error_Handler();
}

int main(void)
{
   HAL_Init();
   SystemClock_Config();

   MX_IWDG_Init_Approx2s();
```

```
  while (1)
  {
    // Alimenta o watchdog
    HAL_IWDG_Refresh(&hiwdg);

    // Simule trabalho normal
    HAL_Delay(100);

    // Para testar reset: comente o refresh ou trave aqui:
    // while (1) { } // travamento intencional: watchdog vai resetar
  }
}

static void Error_Handler(void)
{
  __disable_irq();
  while (1) { }
}

static void SystemClock_Config(void)
{
  // Use a configuração gerada pelo CubeMX no seu projeto real.
}
```

Como testar:

rode normal → não reseta

comente HAL_IWDG_Refresh() → em ~2 s o MCU reseta

Exemplo B — IWDG com “window mode” (detecta refresh cedo demais)

Por que isso existe

Um bug comum: o sistema trava em um loop rápido e continua “chutando” o watchdog, então ele nunca reseta. O window mode impede refresh cedo demais.

Objetivo

timeout total ~2 s

refresh permitido somente após ~1 s (janela)

se der refresh cedo → reset

Obs.: a semântica do “window” é definida no RM (janela de refresh). Este exemplo mostra o uso via HAL. (st.com)

Código completo

```
#include "stm32g0xx_hal.h"


IWDG_HandleTypeDef hiwdg;


static void SystemClock_Config(void);
static void Error_Handler(void);


static void MX_IWDG_Window_Init(void)
{
  // Exemplo conceitual:
  // Prescaler 64, Reload 999 (~2s total)
  // Window 500: refresh só é permitido quando o contador estiver abaixo de 500
  // (ou seja, após “passar um tempo” desde o último reload)


  hiwdg.Instance = IWDG;
  hiwdg.Init.Prescaler = IWDG_PRESCALER_64;
  hiwdg.Init.Reload = 999;
  hiwdg.Init.Window = 500;
```

```
    if (HAL_IWDG_Init(&hiwdg) != HAL_OK)
        Error_Handler();
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_IWDG_Window_Init();

    while (1)
    {
        // Se você chamar refresh cedo demais, ele pode resetar.
        // Aqui atrasamos ~1200ms para cair dentro da janela.
        HAL_Delay(1200);

        HAL_IWDG_Refresh(&hiwdg);
    }
}

static void Error_Handler(void)
{
    __disable_irq();
    while (1) { }
}
```

```
static void SystemClock_Config(void)
{
}
```

Teste prático:

troque HAL_Delay(1200) por HAL_Delay(100) → tende a resetar por refresh fora da janela.

Exemplo C — IWDG como "timeout manager" (sem RTOS)

Objetivo

Demonstrar o uso como "último recurso": você chama Refresh() somente quando o sistema está saudável.

Estratégia:

o loop principal só alimenta o IWDG se um conjunto de "health flags" estiver OK

se alguma parte travar, o watchdog não é alimentado e ocorre reset

Código completo

```
#include "stm32g0xx_hal.h"

IWDG_HandleTypeDef hiwdg;

static volatile uint8_t health_comm_ok = 0;
static volatile uint8_t health_ui_ok   = 0;
static volatile uint8_t health_adc_ok  = 0;

static void SystemClock_Config(void);
static void Error_Handler(void);

static void MX_IWDG_Init_Approx3s(void)
{
  // ~3s: Prescaler=128, Reload=749 -> (750*128)/32000 = 3.0s
```

```
    hiwdg.Instance = IWDG;
    hiwdg.Init.Prescaler = IWDG_PRESCALER_128;
    hiwdg.Init.Reload = 749;
    hiwdg.Init.Window = IWDG_WINDOW_DISABLE;

    if (HAL_IWDG_Init(&hiwdg) != HAL_OK)
        Error_Handler();
}

static void app_update_health_flags(void)
{
    // Em projeto real, estas flags seriam atualizadas por tarefas/ISRs:
    // Aqui, simulamos que tudo está OK:
    health_comm_ok = 1;
    health_ui_ok   = 1;
    health_adc_ok  = 1;
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_IWDG_Init_Approx3s();

    while (1)
    {
        app_update_health_flags();
```

```
        // Só alimenta se tudo estiver OK
        if (health_comm_ok && health_ui_ok && health_adc_ok)
        {
            HAL_IWDG_Refresh(&hiwdg);
        }

        HAL_Delay(100);
    }
}

static void Error_Handler(void)
{
    __disable_irq();
    while (1) { }
}

static void SystemClock_Config(void)
{
}
```

---

8) Debug: “congelar no debug mode”

Em muitos STM32, existe a possibilidade de parar o IWDG quando o core está em debug (breakpoint). Isso evita reset enquanto você está parado no debugger. Essa opção é controlada por bits do DBGMCU (depende da família) e/ou Option Bytes, e deve ser usada com cuidado: em produção, geralmente você não quer watchdog congelado. Consulte o RM e o datasheet da família para o comportamento exato no G0.

---

9) Pegadinhas e boas práticas

Timeout baseado em LSI varia: sempre use margem.

Se você habilitar IWDG por Option Bytes, ele pode iniciar muito cedo (antes do main). Seu firmware deve estar pronto para alimentá-lo desde o início.

Use window mode em produtos onde um travamento “rápido” poderia mascarar falhas.

Não faça refresh em qualquer lugar: centralize o refresh numa função de “supervisão” (health manager).

Se o sistema tem RTOS, a política típica é:

cada task “reporta alive”,

um supervisor alimenta o watchdog se todas reportarem dentro do período.

Checklist final

entender por que o IWDG é independente (LSI) e roda em Stop/Standby

calcular timeout com prescaler/reload e tolerância do LSI

usar IWDG simples e window mode

projetar uma política de refresh baseada em “saúde do sistema”

entender impacto de debug freeze e option bytes

Exemplo D — Detectar reset por IWDG e registrar a causa (backup register), independente (HAL)

A ideia é: no boot, você verifica as flags de reset do RCC para saber se o reset foi causado pelo IWDG. Se foi, você incrementa um contador em backup register (ou guarda uma “assinatura”) para diagnóstico.

No STM32G0, as flags de reset (incluindo IWDGRSTF/WWDGRSTF) ficam em registradores do RCC e são acessíveis por macros do HAL. (st.com)

Código completo

```
#include "stm32g0xx_hal.h"


/* ===================== Handles ===================== */

IWDG_HandleTypeDef hiwdg;

RTC_HandleTypeDef  hrtc;
```

```
/* ===================== Backup registers (no STM32G070 há poucos DRs) */
#define BKP_MAGIC_REG   RTC_BKP_DR0
#define BKP_MAGIC_VAL   0xA55A1234u
#define BKP_IWDG_CNT    RTC_BKP_DR1
#define BKP_LAST_CAUSE  RTC_BKP_DR2

/* Códigos de causa (exemplo) */
#define CAUSE_POWERON   1
#define CAUSE_PINRST    2
#define CAUSE_IWDG      3
#define CAUSE_WWDG      4
#define CAUSE_SFTRST    5
#define CAUSE_OTHER     0xFF

/* ===================== Prototipos ===================== */
static void SystemClock_Config(void);
static void Error_Handler(void);

static void BKP_Init_Minimal(void);
static void BKP_Write(uint32_t reg, uint32_t v);
static uint32_t BKP_Read(uint32_t reg);

static uint8_t detect_reset_cause_and_log(void);
static void MX_IWDG_Init_Approx2s(void);

/* ===================== Error ===================== */
static void Error_Handler(void)
```

```
{
    __disable_irq();
    while (1) { }
}

/* ===================== Backup Domain minimal (RTC clock só para BKP regs)
===================== */
static void BKP_Init_Minimal(void)
{
    /* Para acessar backup regs, precisamos:
       - habilitar PWR clock
       - liberar escrita no backup domain
       - habilitar RTC clock (mesmo que não use calendário)
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    HAL_PWR_EnableBkUpAccess();

    /* Seleciona clock do RTC de forma simples: LSI (não requer cristal) */
    __HAL_RCC_LSI_ENABLE();
    while (__HAL_RCC_GET_FLAG(RCC_FLAG_LSIRDY) == RESET) { }

    __HAL_RCC_RTC_CONFIG(RCC_RTCCLKSOURCE_LSI);
    __HAL_RCC_RTC_ENABLE();

    /* Init mínimo do RTC handle para BKUP read/write */
    hrtc.Instance = RTC;
    hrtc.Init.HourFormat = RTC_HOURFORMAT_24;
    hrtc.Init.AsynchPrediv = 127;
```

```
    hrtc.Init.SynchPrediv  = 255;
    hrtc.Init.OutPut = RTC_OUTPUT_DISABLE;

    if (HAL_RTC_Init(&hrtc) != HAL_OK)
        Error_Handler();

    /* Se primeira vez, grava magic e zera contadores */
    if (BKP_Read(BKP_MAGIC_REG) != BKP_MAGIC_VAL)
    {
        BKP_Write(BKP_MAGIC_REG, BKP_MAGIC_VAL);
        BKP_Write(BKP_IWDG_CNT, 0);
        BKP_Write(BKP_LAST_CAUSE, 0);
    }
}

static void BKP_Write(uint32_t reg, uint32_t v)
{
    HAL_RTCEx_BKUPWrite(&hrtc, reg, v);
}

static uint32_t BKP_Read(uint32_t reg)
{
    return HAL_RTCEx_BKUPRead(&hrtc, reg);
}

/* ===================== Detectar causa do reset ===================== */
static uint8_t detect_reset_cause_and_log(void)
{
```

```
uint8_t cause = CAUSE_OTHER;

/* A ordem é importante: várias flags podem estar setadas.
   Aqui priorizamos watchdogs. */
if (__HAL_RCC_GET_FLAG(RCC_FLAG_IWDGRST) != RESET)
    cause = CAUSE_IWDG;
else if (__HAL_RCC_GET_FLAG(RCC_FLAG_WWDGRST) != RESET)
    cause = CAUSE_WWDG;
else if (__HAL_RCC_GET_FLAG(RCC_FLAG_SFTRST) != RESET)
    cause = CAUSE_SFTRST;
else if (__HAL_RCC_GET_FLAG(RCC_FLAG_PINRST) != RESET)
    cause = CAUSE_PINRST;
else if (__HAL_RCC_GET_FLAG(RCC_FLAG_PORRST) != RESET)
    cause = CAUSE_POWERON;

/* Log em BKP */
BKP_Write(BKP_LAST_CAUSE, cause);

if (cause == CAUSE_IWDG)
{
    uint32_t cnt = BKP_Read(BKP_IWDG_CNT);
    BKP_Write(BKP_IWDG_CNT, cnt + 1);
}

/* Limpa flags para o próximo boot */
__HAL_RCC_CLEAR_RESET_FLAGS();

return cause;
```

```
}

/* ===================== IWDG simples (~2s) ===================== */
static void MX_IWDG_Init_Approx2s(void)
{
    hiwdg.Instance = IWDG;
    hiwdg.Init.Prescaler = IWDG_PRESCALER_64;
    hiwdg.Init.Reload = 999;              // ~2s com LSI ~32k
    hiwdg.Init.Window = IWDG_WINDOW_DISABLE;

    if (HAL_IWDG_Init(&hiwdg) != HAL_OK)
        Error_Handler();
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    /* Prepara BKP regs e registra causa do reset */
    BKP_Init_Minimal();
    uint8_t cause = detect_reset_cause_and_log();
    (void)cause;

    /* Liga watchdog (para testar: travar e ver contador subir) */
    MX_IWDG_Init_Approx2s();

    while (1)
```

```
  {
    /* Normalmente você faria refresh com política de saúde */
    HAL_IWDG_Refresh(&hiwdg);

    HAL_Delay(100);

    /* Para testar reset por IWDG:
      comente o Refresh acima ou trave aqui */
    // while(1) { }
  }
}

/* Stub: substitua pelo gerado no seu projeto real */
static void SystemClock_Config(void)
{
}
```

Como usar na prática

Leia BKP_LAST_CAUSE para saber o motivo do último reset.

Leia BKP_IWDG_CNT para saber quantas vezes o sistema resetou por IWDG ao longo da vida (enquanto o backup domain estiver preservado).

Capítulo 10 — Memória FLASH interna no STM32G070RB: como funciona, organização, proteção e gravação/leitura (HAL)

1) O que é memória Flash e por que ela existe

Flash é uma memória não volátil (não perde dados sem energia) baseada em transistores de porta flutuante. Em resumo:

Leitura: muito rápida (acesso tipo “memória”).

Gravação: mais lenta e feita por “programação” de células.

Apagamento (erase): acontece em blocos (no STM32G0, páginas) e zera um bloco inteiro de uma vez.

Limitação física: cada página suporta um número finito de ciclos de erase/program.

No STM32G070RB, a Flash embutida é usada para armazenar código e dados (constantes, tabelas, parâmetros). O dispositivo tem 128 Kbytes de Flash.

---

2) Flash interna no STM32G070RB: tamanho, páginas, “banco” e granularidade

2.1 Tamanho total

Flash total: 128 Kbytes (no STM32G070RB).

2.2 Páginas (erase unit)

Na família STM32G0, o apagamento é por página de 2 Kbytes.

Logo, no G070RB:

128 KB / 2 KB = 64 páginas (páginas 0 a 63).

2.3 Granularidade de programação

Programação “normal” é feita em double word de 8 bytes (64 bits).

Ou seja: você não programa “1 byte” direto — você programa em blocos mínimos (8 bytes) com alinhamento.

2.4 “Bank” no G070RB

O datasheet trata a Flash como “até um banco de 128 KB”; no G070RB (128 KB) você trabalha, na prática, com um banco para a área principal.

(Em outros STM32G0 com Flash maior, existe opção de dual-bank via option bytes, mas no G070RB isso não muda seu dia a dia.)

---

3) Endurance e retenção (o limite físico)

No STM32G070, a Flash tem (ordem de grandeza):

Endurance: 1 kcycle (mínimo) no intervalo de temperatura especificado

Retenção: 15 anos (após 1 kcycle, a 85 °C)

Implicação prática: não grave parâmetros “a cada loop”. Use técnicas de:

gravação por evento,

bufferização,

wear leveling (alternar páginas),

e journaling (registro com sequência + CRC).

---

4) Modelo mental: leitura é memória; gravação exige erase + program

4.1 Leitura

A Flash aparece no mapa de memória e pode ser lida como um ponteiro:

Base típica do código: 0x08000000 (padrão STM32).

Você faz leitura com const uint8_t* / memcpy — sem HAL.

4.2 Programação e apagamento

Apagar (erase) sempre apaga uma página inteira (2 KB).

Programar escreve 8 bytes por operação (64-bit).

Você só consegue programar bits de 1 → 0; para voltar de 0 → 1, precisa apagar a página.

---

5) Cuidados críticos (quem grava Flash em produto precisa saber)

Nunca grave a Flash sem apagar a página quando estiver reescrevendo dados.

Não grave com alimentação instável (risco de corrupção). Em produto, valide nível de bateria/VDDA.

Durante erase/program, a CPU pode sofrer stalls se estiver executando da Flash (depende da arquitetura), então:

mantenha rotinas curtas,

evite interrupções longas,

ou mova rotinas críticas para RAM (quando necessário).

Sempre use verificação (CRC/assinatura) para garantir integridade.

---

6) API HAL para Flash: unlock → erase → program → lock

A HAL usa:

HAL_FLASH_Unlock() / HAL_FLASH_Lock()

HAL_FLASHEx_Erase(...)

HAL_FLASH_Program(FLASH_TYPEPROGRAM_DOUBLEWORD, addr, data64)

A seguir, exemplos independentes e completos.

6.1 Exemplo 1 — Apagar 1 página e programar 1 doubleword (64-bit)

```
#include "stm32g0xx_hal.h"
#include <string.h>
#include <stdint.h>

static void Error_Handler(void)
{
   __disable_irq();
   while (1) { }
}

#define FLASH_BASE_ADDR      0x08000000UL
#define FLASH_TOTAL_BYTES    (128UL * 1024UL)
#define FLASH_PAGE_BYTES     2048UL

/* Última página para “dados do usuário” (exemplo) */
#define USER_PAGE_INDEX      ((FLASH_TOTAL_BYTES / FLASH_PAGE_BYTES) - 1UL)  //
63
#define USER_PAGE_ADDR       (FLASH_BASE_ADDR + USER_PAGE_INDEX *
FLASH_PAGE_BYTES)

static void flash_erase_page(uint32_t page_index)
{
   FLASH_EraseInitTypeDef erase = {0};
```

```
    uint32_t page_error = 0;

    erase.TypeErase = FLASH_TYPEERASE_PAGES;
    erase.Banks     = FLASH_BANK_1;
    erase.Page      = page_index;
    erase.NbPages   = 1;

    HAL_FLASH_Unlock();

    if (HAL_FLASHEx_Erase(&erase, &page_error) != HAL_OK)
    {
        HAL_FLASH_Lock();
        Error_Handler();
    }

    HAL_FLASH_Lock();
}

static void flash_program_doubleword(uint32_t addr, uint64_t data64)
{
    /* Endereço deve ser alinhado em 8 bytes */
    if ((addr & 0x7) != 0) Error_Handler();

    HAL_FLASH_Unlock();

    if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_DOUBLEWORD, addr, data64) !=
HAL_OK)
    {
```

```
        HAL_FLASH_Lock();
        Error_Handler();
    }

    HAL_FLASH_Lock();
}

int main(void)
{
    HAL_Init();
    // SystemClock_Config();  // use a do seu projeto

    /* 1) Apaga a última página */
    flash_erase_page(USER_PAGE_INDEX);

    /* 2) Programa um 64-bit no início da página */
    uint64_t value = 0x1122334455667788ULL;
    flash_program_doubleword(USER_PAGE_ADDR, value);

    /* 3) Lê de volta (leitura direta) */
    uint64_t readback = *(const uint64_t*)USER_PAGE_ADDR;
    (void)readback;

    while (1) { }
}
```

6.2 Exemplo 2 — Escrever um buffer arbitrário (bytes) na Flash com alinhamento 8 bytes

A Flash programa em 64-bit. Então, para gravar um buffer qualquer, você:

agrupa em blocos de 8 bytes,

completa com 0xFF (valor “apagado”) no final,

programa doubleword por doubleword.

```
#include "stm32g0xx_hal.h"
#include <stdint.h>
#include <string.h>

static void Error_Handler(void)
{
   __disable_irq();
   while (1) { }
}

static void flash_write_bytes_64bit(uint32_t addr, const uint8_t *data, uint32_t len)
{
   if ((addr & 0x7) != 0) Error_Handler();

   HAL_FLASH_Unlock();

   uint32_t off = 0;
   while (off < len)
   {
      uint8_t block[8];
      uint32_t chunk = (len - off >= 8) ? 8 : (len - off);

      /* completa com 0xFF */
      memset(block, 0xFF, sizeof(block));
```

```
    memcpy(block, &data[off], chunk);

    uint64_t dw;
    memcpy(&dw, block, 8);

    if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_DOUBLEWORD, addr + off, dw) !=
HAL_OK)
    {
      HAL_FLASH_Lock();
      Error_Handler();
    }

    off += 8;
  }

  HAL_FLASH_Lock();
}
```

Regra de ouro: se você precisa “alterar” um dado numa página, você geralmente precisa apagar e reescrever a página (ou usar journaling/wear leveling). Não dá para sobrescrever livremente.

---

7) Projeto de armazenamento: parâmetros com integridade e wear leveling (2 páginas)

A técnica mais simples que dá resultado real:

Reserve 2 páginas no fim da Flash (página A e página B).

Estrutura gravada sempre com:

assinatura (magic)

versão

sequência (seq) crescente

payload

CRC32 do payload/estrutura

Sempre grave na página “alternada” (ping-pong).

Assim você distribui desgaste e sempre tem “última configuração válida”.

7.1 Estrutura de registro

```
#include <stdint.h>

#define CFG_MAGIC  0x43464731u /* "CFG1" */
#define CFG_VER    1u

typedef struct __attribute__((packed))
{
   uint32_t magic;
   uint16_t version;
   uint16_t reserved;
   uint32_t seq;
   uint32_t payload_len;

   /* Exemplo de payload fixo (pode ser struct real) */
   uint32_t sample_rate_hz;
   uint16_t trig_level_counts;
   uint16_t trig_hyst_counts;

   uint32_t crc32;
} cfg_record_t;
```

7.2 CRC32 (implementação simples, independente)

```
static uint32_t crc32_sw(const void *data, uint32_t len)
```

```
{
    const uint8_t *p = (const uint8_t*)data;
    uint32_t crc = 0xFFFFFFFFu;

    for (uint32_t i = 0; i < len; i++)
    {
        crc ^= p[i];
        for (int b = 0; b < 8; b++)
        {
            uint32_t mask = -(crc & 1u);
            crc = (crc >> 1) ^ (0xEDB88320u & mask);
        }
    }
    return ~crc;
}
```

7.3 Endereços das páginas A/B (exemplo: duas últimas páginas)

```
#define FLASH_BASE_ADDR      0x08000000UL
#define FLASH_TOTAL_BYTES    (128UL * 1024UL)
#define FLASH_PAGE_BYTES     2048UL
#define FLASH_PAGES          (FLASH_TOTAL_BYTES / FLASH_PAGE_BYTES)

#define PAGE_A_INDEX         (FLASH_PAGES - 2u)    // 62
#define PAGE_B_INDEX         (FLASH_PAGES - 1u)    // 63
#define PAGE_A_ADDR          (FLASH_BASE_ADDR + PAGE_A_INDEX *
FLASH_PAGE_BYTES)
#define PAGE_B_ADDR          (FLASH_BASE_ADDR + PAGE_B_INDEX *
FLASH_PAGE_BYTES)
```

7.4 Funções: validar, ler “mais novo” e salvar (ping-pong)

```
#include "stm32g0xx_hal.h"
#include <string.h>
#include <stdint.h>

static void Error_Handler(void)
{
    __disable_irq();
    while (1) { }
}

static int cfg_is_valid(const cfg_record_t *r)
{
    if (r->magic != CFG_MAGIC) return 0;
    if (r->version != CFG_VER) return 0;

    /* CRC cobre tudo menos o campo crc32 */
    uint32_t calc = crc32_sw(r, (uint32_t)offsetof(cfg_record_t, crc32));
    return (calc == r->crc32);
}

static const cfg_record_t* cfg_pick_newest(void)
{
    const cfg_record_t *a = (const cfg_record_t*)PAGE_A_ADDR;
    const cfg_record_t *b = (const cfg_record_t*)PAGE_B_ADDR;

    int va = cfg_is_valid(a);
    int vb = cfg_is_valid(b);
```

```
    if (va && vb) return (a->seq >= b->seq) ? a : b;
    if (va) return a;
    if (vb) return b;
    return NULL;
}

static void flash_erase_page(uint32_t page_index)
{
    FLASH_EraseInitTypeDef erase = {0};
    uint32_t page_error = 0;

    erase.TypeErase = FLASH_TYPEERASE_PAGES;
    erase.Banks     = FLASH_BANK_1;
    erase.Page      = page_index;
    erase.NbPages   = 1;

    if (HAL_FLASHEx_Erase(&erase, &page_error) != HAL_OK)
        Error_Handler();
}

static void flash_program_record(uint32_t base_addr, const void *rec, uint32_t bytes)
{
    if ((base_addr & 0x7) != 0) Error_Handler();
    if ((bytes & 0x7) != 0) Error_Handler(); /* grava em múltiplos de 8 */

    const uint8_t *p = (const uint8_t*)rec;

    for (uint32_t off = 0; off < bytes; off += 8)
```

```
    {
        uint64_t dw;
        memcpy(&dw, &p[off], 8);

        if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_DOUBLEWORD, base_addr + off,
dw) != HAL_OK)
            Error_Handler();
    }
}

static void cfg_save(const cfg_record_t *new_cfg)
{
    cfg_record_t tmp = *new_cfg;
    tmp.magic = CFG_MAGIC;
    tmp.version = CFG_VER;
    tmp.crc32 = crc32_sw(&tmp, (uint32_t)offsetof(cfg_record_t, crc32));

    const cfg_record_t *cur = cfg_pick_newest();

    /* Próxima sequência */
    uint32_t next_seq = (cur ? (cur->seq + 1u) : 1u);
    tmp.seq = next_seq;

    /* Decide destino: alterna A/B com base em quem é o “atual” */
    uint32_t dst_page = PAGE_A_INDEX;
    uint32_t dst_addr = PAGE_A_ADDR;

    if (cur == (const cfg_record_t*)PAGE_A_ADDR)
```

```
    {
        dst_page = PAGE_B_INDEX;
        dst_addr = PAGE_B_ADDR;
    }
    else if (cur == (const cfg_record_t*)PAGE_B_ADDR)
    {
        dst_page = PAGE_A_INDEX;
        dst_addr = PAGE_A_ADDR;
    }

    HAL_FLASH_Unlock();

    flash_erase_page(dst_page);

    /* grava record (tamanho múltiplo de 8) */
    uint32_t sz = sizeof(cfg_record_t);
    uint32_t pad = (8 - (sz % 8)) % 8;
    uint8_t buf[sizeof(cfg_record_t) + 8] = {0};

    memcpy(buf, &tmp, sizeof(cfg_record_t));
    memset(buf + sizeof(cfg_record_t), 0xFF, pad);

    flash_program_record(dst_addr, buf, sizeof(cfg_record_t) + pad);

    HAL_FLASH_Lock();
}

static int cfg_load(cfg_record_t *out)
```

```
{
    const cfg_record_t *r = cfg_pick_newest();
    if (!r) return 0;
    *out = *r;
    return 1;
}
```

7.5 Exemplo de uso

```
int main(void)
{
    HAL_Init();
    // SystemClock_Config();

    cfg_record_t cfg;

    if (!cfg_load(&cfg))
    {
        /* Sem config válida: cria padrão */
        cfg.sample_rate_hz = 1000;
        cfg.trig_level_counts = 2500;
        cfg.trig_hyst_counts  = 100;
        cfg.payload_len = 0; // aqui é fixo, mas você pode usar se tiver payload variável

        cfg_save(&cfg);
    }
    else
    {
        /* Ajusta um parâmetro e salva */
```

```
    cfg.sample_rate_hz = 2000;
    cfg_save(&cfg);
  }

  while (1) { }
}
```

Esse modelo deixa o leitor preparado para:

parametrização persistente,

integridade com CRC,

resiliência a reset durante gravação (sempre existe uma página anterior válida),

redução de desgaste (alternância entre páginas).

---

8) Proteções e option bytes: o que você precisa conhecer

A Flash no STM32G0 suporta proteções configuráveis via option bytes, como:

Write protection por página

Readout protection global

PCROP (execute-only por áreas alinhadas) e securable memory, alinhados por páginas de 2 KB

Isso é essencial em produto final (proteção de IP e travar regiões contra escrita acidental).

---

9) Checklist final

Flash é apagada por página e programada em doubleword (8 bytes)

128 KB no G070RB, páginas de 2 KB → 64 páginas

nunca “editar” sem apagar página

usar CRC/assinatura/seq para integridade

aplicar wear leveling simples (2 páginas ping-pong)

conhecer proteções (WRP/RDP/PCROP/securable)

respeitar endurance (ciclos) e retenção

Capítulo 10.1 — Reservando páginas da FLASH no linker script para “dados do usuário” (sem o firmware sobrescrever)

A ideia é simples: você escolhe uma região da Flash (normalmente as últimas páginas) para guardar configurações/logs, e faz o linker tratar essa região como “não disponível para código/constantes”. Assim, o compilador nunca vai colocar .text/.rodata lá.

No STM32G070RB:

Flash total: 128 KB

página: 2 KB

últimas 2 páginas = 4 KB (bom para config + wear leveling)

---

1) Definindo a região reservada (endereços)

Base Flash típica: 0x08000000.

Flash total = 128 * 1024 = 0x20000

Final da Flash = 0x08000000 + 0x20000 = 0x08020000

Últimas 2 páginas (4 KB):

RESERVED_START = 0x0801F000

RESERVED_END = 0x08020000

Porque 0x20000 - 0x1000 = 0x1F000.

---

2) Método recomendado: “cortar” a FLASH do linker e criar uma região extra

Você vai editar o arquivo .ld (linker script) do projeto.

2.1 No MEMORY, ajuste FLASH para não incluir a área reservada

Procure algo assim:

```
MEMORY
{
```

```
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 128K
  RAM   (xrw) : ORIGIN = 0x20000000, LENGTH = 36K
}
```

E altere para:

```
MEMORY
{
  /* 128K total - 4K reservados = 124K */
  FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 124K
  FLASH_USER (rx) : ORIGIN = 0x0801F000, LENGTH = 4K
  RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 36K
}
```

✅ **Resultado: o linker nunca colocará código/dados "normais" dentro de FLASH_USER.**

---

3) Criando uma seção dedicada para dados persistentes

Ainda no .ld, dentro de SECTIONS, crie uma seção para sua área. Por exemplo:

```
/* ========================= */
/* User Flash Reserved Area  */
/* ========================= */
.user_flash (NOLOAD) :
{
  . = ALIGN(8);
  __user_flash_start__ = .;
  KEEP(*(.user_flash))
  KEEP(*(.user_flash*))
  . = ALIGN(8);
  __user_flash_end__ = .;
```

} > FLASH_USER

Por que KEEP?

Para garantir que o linker não elimine objetos não referenciados (otimizações).

Por que ALIGN(8)?

Porque no G0 a programação é em doubleword (8 bytes).

Por que NOLOAD?

Porque você não quer que o startup copie nada para lá nem inicialize com zeros. Isso é "memória persistente", não variável normal.

---

4) Criando símbolos para usar no C (endereço e tamanho)

No C, você pode declarar os símbolos do linker:

```
#include <stdint.h>


extern const uint8_t __user_flash_start__;

extern const uint8_t __user_flash_end__;


static inline uint32_t user_flash_base(void)

{

   return (uint32_t)&__user_flash_start__;

}


static inline uint32_t user_flash_size(void)

{

   return (uint32_t)(&__user_flash_end__ - &__user_flash_start__);

}
```

Observação: esses símbolos existem "no endereço" e não ocupam RAM.

---

5) Método prático (e seguro): definir endereços por macro e não depender do section

Muita gente prefere definir diretamente:

```
#define FLASH_BASE_ADDR      0x08000000UL

#define FLASH_TOTAL_BYTES    (128UL * 1024UL)

#define FLASH_PAGE_BYTES     2048UL


#define USER_PAGES           2UL

#define USER_AREA_BYTES      (USER_PAGES * FLASH_PAGE_BYTES)


#define USER_AREA_START      (FLASH_BASE_ADDR + FLASH_TOTAL_BYTES -
USER_AREA_BYTES) // 0x0801F000

#define USER_AREA_END        (FLASH_BASE_ADDR + FLASH_TOTAL_BYTES)          //
0x08020000
```

E mesmo assim cortar a Flash no linker (seção 2).

Esse é o melhor dos dois mundos: o linker protege e você tem endereços claros.

---

6) Garantindo que o seu firmware realmente não usa a área reservada

Depois de compilar, verifique:

O arquivo .map (gerado pelo linker): procure .text, .rodata e confirme que não há endereços em 0x0801F000..0x0801FFFF.

Se você usa CubeIDE: abra o “Memory View” ou “Map file viewer”.

Se você usa arm-none-eabi-objdump -h firmware.elf, confirme que suas seções não invadem a região.

---

7) Exemplo completo: guardar uma struct “fixa” no início da área reservada

Você pode definir um endereço fixo dentro da área reservada:

```
#define CFG_ADDR   (USER_AREA_START)  // início da região reservada


typedef struct __attribute__((packed))

{
```

```
    uint32_t magic;
    uint32_t seq;
    uint32_t sample_rate;
    uint32_t crc;
} cfg_t;

static inline const cfg_t* cfg_flash_ptr(void)
{
    return (const cfg_t*)CFG_ADDR;
}
```

A leitura é direta:

```
cfg_t cfg = *cfg_flash_ptr();
```

A gravação segue o fluxo:

apagar página (ou páginas)

programar em 8 bytes (doubleword)

validar com CRC

---

8) Pegadinhas de especialista (as que dão dor de cabeça)

Não reserve páginas que o bootloader usa (se você tiver bootloader).

Cuidado com atualização de firmware: se o novo firmware crescer, ele pode "bater" na área reservada — o linker vai acusar erro de overflow (isso é bom).

Integre com o versionamento: se mudar a struct, use version/magic e migração.

Sempre alinhe em 8 bytes para programar no STM32G0.

Checklist final

cortar FLASH no MEMORY para remover a área reservada

criar FLASH_USER e uma seção .user_flash (opcional, mas organizado)

expor símbolos __user_flash_start__/end ou usar macros fixas

validar no .map que o código não invade a área reservada

Subcapítulo F.2 — Particionamento “produto final”: Bootloader + Aplicação + User Data (FLASH) no STM32G070RB

Este subcapítulo entrega um modelo completo de partição para 128 KB de Flash (STM32G070RB), com:

Bootloader no início da Flash

Aplicação logo após

User Data (config/log) no final, protegido pelo linker

Todos os blocos abaixo podem ser adaptados diretamente ao particionamento do projeto.

---

1) Mapa de memória proposto

Flash total: 128 KB → 0x0800_0000 até 0x0802_0000

Página: 2 KB → ótimo reservar em múltiplos de 2 KB.

Proposta (simples e robusta)

Bootloader: 16 KB (8 páginas)

Aplicação: 108 KB (54 páginas)

User Data: 4 KB (2 páginas)

Região	Tamanho	Endereço início	Endereço fim
Bootloader	16 KB	0x08000000	0x08004000
Aplicação	108 KB	0x08004000	0x0801F000
User Data	4 KB	0x0801F000	0x08020000

O User Data fica nas 2 últimas páginas (4 KB) e nunca será ocupado por código.

---

2) Arquivo comum de partição (para Bootloader e App)

Crie flash_layout.h:

```
#pragma once

#include <stdint.h>
```

```
#define FLASH_BASE_ADDR         0x08000000UL
#define FLASH_TOTAL_BYTES       (128UL * 1024UL)
#define FLASH_END_ADDR          (FLASH_BASE_ADDR + FLASH_TOTAL_BYTES)

#define FLASH_PAGE_BYTES        2048UL

/* --- Bootloader --- */
#define BL_SIZE_BYTES           (16UL * 1024UL)
#define BL_START_ADDR           (FLASH_BASE_ADDR)
#define BL_END_ADDR             (BL_START_ADDR + BL_SIZE_BYTES)

/* --- Application --- */
#define APP_START_ADDR          (BL_END_ADDR)           /* 0x08004000 */
#define USER_DATA_SIZE_BYTES    (4UL * 1024UL)
#define USER_DATA_START_ADDR    (FLASH_END_ADDR - USER_DATA_SIZE_BYTES) /* 0x0801F000 */
#define USER_DATA_END_ADDR      (FLASH_END_ADDR)

#define APP_END_ADDR            (USER_DATA_START_ADDR)
#define APP_MAX_SIZE_BYTES      (APP_END_ADDR - APP_START_ADDR)

/* Helpers */
static inline uint32_t is_in_app(uint32_t addr)
{
    return (addr >= APP_START_ADDR) && (addr < APP_END_ADDR);
}
```

---

3) Linker script do Bootloader (bootloader.ld)

A ideia: Bootloader usa somente os 16 KB iniciais de Flash.

Edite o .ld do bootloader e use algo assim:

```
/* bootloader.ld (STM32G070RB) */

ENTRY(Reset_Handler)

/* Ajuste para o seu RAM real */
MEMORY
{
  FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 16K
  RAM   (xrw): ORIGIN = 0x20000000, LENGTH = 36K
}

/* Símbolos úteis */
__flash_start__ = ORIGIN(FLASH);
__flash_end__   = ORIGIN(FLASH) + LENGTH(FLASH);

SECTIONS
{
  /* Vetor + código */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector))
    . = ALIGN(4);
  } > FLASH

  .text :
```

```
{
  . = ALIGN(4);
  *(.text*)
  *(.rodata*)
  *(.glue_7) *(.glue_7t)
  *(.eh_frame)
  . = ALIGN(4);
} > FLASH

/* Dados inicializados em RAM, carregados da FLASH */
.data : AT (ADDR(.text) + SIZEOF(.text))
{
  . = ALIGN(4);
  __data_start__ = .;
  *(.data*)
  . = ALIGN(4);
  __data_end__ = .;
} > RAM

/* BSS */
.bss :
{
  . = ALIGN(4);
  __bss_start__ = .;
  *(.bss*)
  *(COMMON)
  . = ALIGN(4);
  __bss_end__ = .;
```

```
  } > RAM

  /* Heap/Stack (opcional: seu startup pode usar símbolos diferentes) */
  ._user_heap_stack :
  {
    . = ALIGN(8);
    PROVIDE(end = .);
    . = . + 0x800;   /* heap exemplo 2 KB */
    . = . + 0x800;   /* stack exemplo 2 KB */
    . = ALIGN(8);
  } > RAM
}
```

Resultado: se o bootloader crescer além de 16 KB, o linker dá erro — isso é bom.

---

4) Linker script da Aplicação (app.ld)

Aqui a aplicação começa em 0x08004000 e a Flash disponível termina em 0x0801F000 (antes do User Data).

```
/* app.ld (STM32G070RB) */

ENTRY(Reset_Handler)

/* Ajuste para o seu RAM real */
MEMORY
{
  /* App ocupa 0x08004000 .. 0x0801F000 (108 KB) */
  FLASH (rx)      : ORIGIN = 0x08004000, LENGTH = 108K
  FLASH_USER (rx) : ORIGIN = 0x0801F000, LENGTH = 4K
  RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 36K
```

```
}

/* Símbolos para app e user data */
__app_flash_start__  = ORIGIN(FLASH);
__app_flash_end__    = ORIGIN(FLASH) + LENGTH(FLASH);
__user_flash_start__ = ORIGIN(FLASH_USER);
__user_flash_end__   = ORIGIN(FLASH_USER) + LENGTH(FLASH_USER);

SECTIONS
{
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector))
    . = ALIGN(4);
  } > FLASH

  .text :
  {
    . = ALIGN(4);
    *(.text*)
    *(.rodata*)
    *(.glue_7) *(.glue_7t)
    *(.eh_frame)
    . = ALIGN(4);
  } > FLASH

  .data : AT (ADDR(.text) + SIZEOF(.text))
```

```
{
  . = ALIGN(4);
  __data_start__ = .;
  *(.data*)
  . = ALIGN(4);
  __data_end__ = .;
} > RAM

.bss :
{
  . = ALIGN(4);
  __bss_start__ = .;
  *(.bss*)
  *(COMMON)
  . = ALIGN(4);
  __bss_end__ = .;
} > RAM

/* Área reservada na flash (opcional, mas útil) */
.user_flash (NOLOAD) :
{
  . = ALIGN(8);
  KEEP(*(.user_flash))
  KEEP(*(.user_flash*))
  . = ALIGN(8);
} > FLASH_USER

._user_heap_stack :
```

```
  {
    . = ALIGN(8);
    PROVIDE(end = .);
    . = . + 0x1000;  /* heap exemplo 4 KB */
    . = . + 0x1000;  /* stack exemplo 4 KB */
    . = ALIGN(8);
  } > RAM
}
```

✅ **Resultado:**

A aplicação nunca invade 0x0801F000..0x0801FFFF

Você pode usar __user_flash_start__ no C como base da área de config/log

---

5) Bootloader: como "pular" para a aplicação (código independente)

Para executar a aplicação corretamente, o bootloader precisa:

validar que existe uma aplicação válida

setar MSP (Main Stack Pointer) com o primeiro word do vetor da aplicação

setar VTOR para o vetor da aplicação (se o Cortex-M0+ suportar VTOR — no M0+ geralmente suporta)

pular para o Reset_Handler da aplicação

Crie boot_jump.c:

```
#include <stdint.h>
#include "stm32g0xx.h"
#include "flash_layout.h"

typedef void (*pfunc_t)(void);

static int app_vector_seems_valid(uint32_t app_base)
{
```

```
    uint32_t msp = *(volatile uint32_t*)(app_base + 0x00);
    uint32_t rst = *(volatile uint32_t*)(app_base + 0x04);

    /* MSP deve apontar para RAM (faixa típica 0x20000000...) */
    if ((msp & 0x2FFE0000UL) != 0x20000000UL) return 0;

    /* Reset handler deve estar dentro da região da aplicação */
    if (!is_in_app(rst)) return 0;

    return 1;
}

void boot_jump_to_app(void)
{
    uint32_t app_base = APP_START_ADDR;

    if (!app_vector_seems_valid(app_base))
        return; // fica no bootloader

    /* Desabilita IRQs globalmente */
    __disable_irq();

    /* Desliga SysTick se estiver ativo (defensivo) */
    SysTick->CTRL = 0;
    SysTick->LOAD = 0;
    SysTick->VAL  = 0;

    /* Opcional: desabilitar interrupções de periféricos usados pelo boot */
```

```
    /* Carrega stack pointer da aplicação */
    uint32_t app_msp = *(volatile uint32_t*)(app_base + 0x00);
    __set_MSP(app_msp);

    /* Ajusta vetor de interrupções para a aplicação (VTOR) */
    SCB->VTOR = app_base;

    /* Pula para Reset_Handler da aplicação */
    uint32_t app_reset = *(volatile uint32_t*)(app_base + 0x04);
    ((pfunc_t)app_reset)();
}
```

Se seu projeto usa HAL no bootloader, antes de pular pode ser interessante “deinit” clocks/periféricos, mas isso é opcional e depende do seu boot.

---

6) Aplicação: garantir VTOR correto (caso você não confie no boot)

Muitos projetos fazem o SCB->VTOR = APP_START_ADDR já no início da aplicação também (defensivo). Você pode colocar no começo do main():

```
#include "stm32g0xx.h"
#include "flash_layout.h"

int main(void)
{
    SCB->VTOR = APP_START_ADDR;
    // HAL_Init(); SystemClock_Config(); ...
    while (1) {}
}
```

---

7) Área de User Data: endereços e uso

Você pode usar direto os símbolos do linker (no app.ld) ou macros do flash_layout.h.

Exemplo de base:

```
#include "flash_layout.h"

#define CFG_BASE_ADDR USER_DATA_START_ADDR
```

E então usar as rotinas de Flash do capítulo anterior:

apagar página(s)

programar doubleword (8 bytes)

validar por CRC/seq

---

8) Atualização de firmware: como o bootloader programa a aplicação

Modelo clássico:

bootloader recebe firmware (UART/I²C/SPI/CAN)

apaga páginas da região da aplicação (APP_START_ADDR..APP_END_ADDR)

grava doublewords (8 bytes)

valida CRC/assinatura

troca “slot válido” e pula para app

O ponto importante do particionamento é: boot nunca apaga nem pisa o próprio boot, porque ele só opera dentro do range de app.

---

9) Checklist de validação

Compile o bootloader e confira no .map que ele está em 0x08000000..0x08003FFF.

Compile a app e confira que .isr_vector começa em 0x08004000.

Confira que nada ocupa 0x0801F000..0x0801FFFF.

Confirme que SCB->VTOR está sendo ajustado (no boot e/ou app).

Confirme que o primeiro word do vetor da app aponta para RAM e o segundo aponta para Flash dentro da app.

Capítulo 11 — Sistema de Clocks do STM32G070RB: fontes, PLL, prescalers, clocks de periféricos e como configurar (HAL)

1. Por que o sistema de clocks é “o coração” do STM32

Todo periférico depende de um clock para funcionar. No STM32G070RB, o bloco RCC (Reset and Clock Control) decide:

qual é o SYSCLK (clock raiz do sistema),

como ele é dividido em HCLK (AHB) e PCLK (APB),

quais periféricos recebem clocks independentes do SYSCLK (USART/I²C/ADC etc.),

como o sistema reage a falha de clock externo (CSS),

e quais clocks podem ser exportados para pinos (MCO/LSCO).

O datasheet destaca o limite de 64 MHz para SYSCLK e o uso de prescalers para AHB/APB.

---

2. Fontes de clock disponíveis (internas e externas)

2.1 Osciladores internos

HSI16: RC interno de 16 MHz, com boa precisão para uso geral e wakeup rápido.

LSI: RC interno ~32 kHz (±5% típico), usado para IWDG e pode alimentar RTC/low-power.

Observação prática: LSI não é para “tempo preciso”, mas é ótimo para watchdog e timeouts.

2.2 Osciladores externos

HSE: oscilador externo (cristal/resonador ou clock externo) 4 a 48 MHz, com Clock Security System (CSS).

LSE: cristal 32.768 kHz para RTC (alta estabilidade), com CSS e modos de drive.

2.3 Clocks que o RCC pode exportar

MCO: pode colocar em um pino clocks como HSI16/HSE/LSI/LSE/SYSCLK/PLLCLK.

LSCO: pode colocar LSI ou LSE em um pino, inclusive em vários modos low-power (exceto VBAT).

---

3. PLL no STM32G0: 3 saídas independentes (R, P e Q)

O STM32G0 tem uma PLL com três saídas úteis, e isso muda totalmente a forma de pensar clocks:

PLLRCLK: pode virar o SYSCLK (clock do sistema).

PLLPCLK: pode clockar periféricos como ADC (e I2S1 na família).

PLLQCLK: pode clockar periféricos como TIM1/TIM15 (e RNG em alguns G0).

A PLL pode ser alimentada por HSI16 ou HSE.

Consequência prática: você pode ter SYSCLK em 64 MHz e ainda alimentar ADC por PLLP com uma frequência "mais adequada", sem amarrar tudo no mesmo clock.

---

4. Árvore principal: SYSCLK → HCLK (AHB) → PCLK (APB)

A seleção típica é:

SYSCLK: vem de HSI16, HSE, LSI, LSE ou PLLR.

HCLK (AHB): SYSCLK dividido por prescaler AHB.

PCLK (APB): HCLK dividido por prescaler APB.

O datasheet reforça que você pode configurar essas divisões e que o máximo do domínio chega a 64 MHz.

---

5. "Periféricos com clock próprio": onde mora a maioria das dúvidas

No STM32G070RB, vários periféricos podem ter clock independente do SYSCLK, como USARTs, I^2Cs e ADC.

Isso significa:

você pode rodar o núcleo em 64 MHz

e ainda assim escolher, por exemplo, USART com clock HSI16 para operar em Stop (quando permitido)

Na prática, isso é configurado via RCC (ex.: registradores de seleção de clock de periféricos; no HAL, via HAL_RCCEx_PeriphCLKConfig).

---

6. Como saber “qual clock alimenta qual periférico”

Você usa dois níveis de checagem:

Clock gating (habilitar o clock do periférico no barramento)

Ex.: __HAL_RCC_USART2_CLK_ENABLE(), __HAL_RCC_SPI1_CLK_ENABLE(), etc.

Se não habilitar, o periférico “não existe” para o software (registradores não funcionam).

Clock source do periférico (quando existe MUX)

Alguns periféricos têm seleção de fonte (HSI16, PCLK, PLLP, LSE, etc.). O datasheet destaca essa possibilidade.

Regra prática para o livro:

Sempre habilite o clock do periférico (gating)

Só mexa na seleção de fonte quando:

você quer operar em low-power específico, ou

precisa de frequência/precisão específica, ou

o periférico “não bate” com o clock do barramento

---

7. HAL: as 2 funções que você tem que dominar

A ST descreve o fluxo de configuração de clock em dois passos principais:

HAL_RCC_OscConfig()

Liga/desliga osciladores e configura PLL (fonte, multiplicadores/divisores).

HAL_RCC_ClockConfig()

Escolhe SYSCLK e configura prescalers AHB/APB e latência de Flash.

Além disso, para clocks de periféricos:

HAL_RCCEx_PeriphCLKConfig() (seleção do clock de USART/I²C/ADC/RTC, etc.)

---

8. Exemplos completos (independentes)

Exemplo 1 — Clock simples: HSI16 como SYSCLK (16 MHz)

Bom para começar, baixo risco.

```
#include "stm32g0xx_hal.h"

static void Error_Handler(void)
{
    __disable_irq();
    while (1) {}
}

void SystemClock_Config_HSI16_16MHz(void)
{
    RCC_OscInitTypeDef osc = {0};
    RCC_ClkInitTypeDef clk = {0};

    /* 1) Oscilador: HSI16 ligado */
    osc.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    osc.HSIState = RCC_HSI_ON;
    osc.HSIDiv = RCC_HSI_DIV1;         // HSI16 / 1
    osc.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    osc.PLL.PLLState = RCC_PLL_OFF;

    if (HAL_RCC_OscConfig(&osc) != HAL_OK)
        Error_Handler();

    /* 2) SYSCLK = HSI16; AHB=1; APB=1 */
    clk.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_PCLK1;
    clk.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
    clk.AHBCLKDivider = RCC_SYSCLK_DIV1;
```

```
    clk.APB1CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_0) != HAL_OK)
        Error_Handler();
}
```

Exemplo 2 — SYSCLK a 64 MHz com PLL usando HSI16

Configuração típica “sem cristal”, ainda assim rápida.

A PLL do G0 permite SYSCLK via PLLR até 64 MHz.

```
#include "stm32g0xx_hal.h"

static void Error_Handler(void)
{
    __disable_irq();
    while (1) {}
}

void SystemClock_Config_PLL_64MHz_From_HSI16(void)
{
    RCC_OscInitTypeDef osc = {0};
    RCC_ClkInitTypeDef clk = {0};

    /* 1) HSI16 ligado e PLL ON */
    osc.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    osc.HSIState = RCC_HSI_ON;
    osc.HSIDiv = RCC_HSI_DIV1;
    osc.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
```

```
  osc.PLL.PLLState = RCC_PLL_ON;
  osc.PLL.PLLSource = RCC_PLLSOURCE_HSI;     // PLL alimentada por
HSI16 :contentReference[oaicite:21]{index=21}

  /* Ajustes típicos (exemplo):
    - objetivo: PLLR = 64 MHz
    Os fatores exatos dependem da família; em projeto real, deixe o CubeMX calcular.
    A ideia aqui é o padrão de uso com HAL (OscConfig + ClockConfig).
  */
  osc.PLL.PLLM = RCC_PLLM_DIV1;
  osc.PLL.PLLN = 8;                    // 16MHz * 8 = 128MHz (VCO)
  osc.PLL.PLLR = RCC_PLLR_DIV2;            // 128/2 = 64MHz (SYSCLK)
  osc.PLL.PLLP = RCC_PLLP_DIV2;            // saída P disponível
  osc.PLL.PLLQ = RCC_PLLQ_DIV2;            // saída Q disponível

  if (HAL_RCC_OscConfig(&osc) != HAL_OK)
     Error_Handler();

  /* 2) SYSCLK = PLLR; HCLK=64; PCLK=64 */
  clk.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_PCLK1;
  clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  clk.AHBCLKDivider = RCC_SYSCLK_DIV1;
  clk.APB1CLKDivider = RCC_HCLK_DIV1;

  /* Latência de Flash: depende da frequência e range. Use o que CubeMX indicar. */
  if (HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_2) != HAL_OK)
     Error_Handler();
}
```

Por que mencionar o CubeMX? Porque o G0 tem detalhes de “power range” e limites por modo; o treinamento do RCC mostra que a frequência máxima depende do range e modo (Range 1, Range 2, low-power).

Exemplo 3 — Como “ler” clocks em runtime (para debug e validação)

Essas funções são essenciais para o leitor virar especialista:

```
#include "stm32g0xx_hal.h"


typedef struct
{
   uint32_t sysclk;
   uint32_t hclk;
   uint32_t pclk1;
} clock_info_t;


clock_info_t clocks_read(void)
{
   clock_info_t c;
   c.sysclk = HAL_RCC_GetSysClockFreq();
   c.hclk   = HAL_RCC_GetHCLKFreq();
   c.pclk1  = HAL_RCC_GetPCLK1Freq();
   return c;
}
```

Uso: você pode armazenar o resultado em variáveis e inspecionar no debugger.

Exemplo 4 — Seleção de clock de periférico (USART) independente do SYSCLK

Alguns periféricos podem escolher fonte própria (HSI16/LSE/PCLK etc.).

Exemplo: configurar USART2 para usar PCLK1 (ou HSI16, se desejado) via HAL:

```
#include "stm32g0xx_hal.h"
```

```
static void Error_Handler(void)
{
   __disable_irq();
   while (1) {}
}

void PeriphClock_USART2_PCLK1(void)
{
   RCC_PeriphCLKInitTypeDef p = {0};

   p.PeriphClockSelection = RCC_PERIPHCLK_USART2;
   p.Usart2ClockSelection = RCC_USART2CLKSOURCE_PCLK1;

   if (HAL_RCCEx_PeriphCLKConfig(&p) != HAL_OK)
      Error_Handler();
}
```

Em aplicações low-power, escolher HSI16 como clock de UART pode permitir que a UART “acorde” o HSI16 em Stop dependendo do periférico/configuração.

---

9. Clock Security System (CSS): tolerância a falha do cristal

O datasheet do G070 descreve:

detecção de falha do HSE, com troca automática para HSI16 e possibilidade de interrupção

detecção de falha do LSE também pode gerar interrupção

Para produtos robustos (automotivo/industrial), CSS é parte do projeto de confiabilidade.

---

10. Checklist de especialista

Fontes: HSI16, HSE, LSI, LSE e quando escolher cada uma

PLL com saídas R (SYSCLK), P (ADC/I2S), Q (TIM1/TIM15/RNG)

Prescalers: SYSCLK → HCLK → PCLK

Periféricos com clock independente (USART/I²C/ADC, etc.)

Fluxo HAL correto: HAL_RCC_OscConfig() + HAL_RCC_ClockConfig() + HAL_RCCEx_PeriphCLKConfig()

Como validar em runtime: HAL_RCC_GetSysClockFreq(), HAL_RCC_GetHCLKFreq(), HAL_RCC_GetPCLK1Freq()

CSS e saídas MCO/LSCO para diagnóstico

Roteiro prático de configuração de clocks (STM32G070RB) — 4 cenários "de engenharia" (HAL)

A ideia aqui é você ter um checklist repetível (o que ligar / o que selecionar / como validar) e um SystemClock_Config completo para cada cenário, mais a seleção de clock de periféricos quando necessário.

---

0) Checklist universal (serve para qualquer cenário)

Decida a fonte do SYSCLK

HSI16 (simples) ou HSE (mais preciso) ou PLL (para 64 MHz).

Configure osciladores + PLL com HAL_RCC_OscConfig()

Escolha SYSCLK e prescalers AHB/APB com HAL_RCC_ClockConfig()

Ajuste Flash latency coerente com a frequência (CubeMX é referência prática)

(Opcional) Configure clocks de periféricos com HAL_RCCEx_PeriphCLKConfig()

Valide em runtime:

HAL_RCC_GetSysClockFreq(), HAL_RCC_GetHCLKFreq(), HAL_RCC_GetPCLK1Freq()

Dica de especialista: sempre valide também que HSE/LSE/PLL ficaram "READY" quando aplicável (flags do RCC), principalmente em produto.

---

1) Cenário A — "Sem cristal": HSI16 + PLL = 64 MHz (padrão rápido e simples)

Quando usar

Produto sem cristal externo, custo/complexidade menor.

Você quer CPU a 64 MHz.

Passos

Liga HSI16

PLL source = HSI16

PLLR = SYSCLK

AHB/APB = DIV1

Código (SystemClock_Config)

```
#include "stm32g0xx_hal.h"

static void Error_Handler(void)
{
  __disable_irq();
  while (1) {}
}

void SystemClock_Config_HSI16_PLL_64MHz(void)
{
  RCC_OscInitTypeDef osc = {0};
  RCC_ClkInitTypeDef clk = {0};

  osc.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  osc.HSIState = RCC_HSI_ON;
  osc.HSIDiv = RCC_HSI_DIV1;
  osc.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;

  osc.PLL.PLLState = RCC_PLL_ON;
```

```
  osc.PLL.PLLSource = RCC_PLLSOURCE_HSI;

  /* Exemplo típico para 64 MHz:
    HSI16 = 16 MHz
    VCO = 16 * N / M
    PLLR = VCO / R

    Aqui: M=1, N=8 => VCO=128 MHz
          R=2 => PLLR=64 MHz
  */
  osc.PLL.PLLM = RCC_PLLM_DIV1;
  osc.PLL.PLLN = 8;
  osc.PLL.PLLR = RCC_PLLR_DIV2;

  /* Ative P/Q também se for usar periféricos que dependem disso */
  osc.PLL.PLLP = RCC_PLLP_DIV2;
  osc.PLL.PLLQ = RCC_PLLQ_DIV2;

  if (HAL_RCC_OscConfig(&osc) != HAL_OK) Error_Handler();

  clk.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_PCLK1;
  clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  clk.AHBCLKDivider = RCC_SYSCLK_DIV1;
  clk.APB1CLKDivider = RCC_HCLK_DIV1;

  /* Flash latency: use o valor recomendado pelo CubeMX no seu projeto */
  if (HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_2) != HAL_OK) Error_Handler();
```

}

Validação rápida

uint32_t sys = HAL_RCC_GetSysClockFreq();

uint32_t hclk = HAL_RCC_GetHCLKFreq();

uint32_t pclk = HAL_RCC_GetPCLK1Freq();

---

2) Cenário B — “Com cristal”: HSE + PLL = 64 MHz (precisão melhor)

Quando usar

Você precisa de melhor precisão (UART baud exato, tempo mais estável).

Placa tem cristal/clock externo no HSE.

Passos

Liga HSE

PLL source = HSE

Ajusta PLL para obter 64 MHz

AHB/APB = DIV1

Código (SystemClock_Config)

Aqui deixo HSE em 8 MHz como exemplo (muito comum). Se seu cristal for outro, ajuste PLLN/PLLR.

```
#include "stm32g0xx_hal.h"


static void Error_Handler(void)
{
  __disable_irq();
  while (1) {}
}


void SystemClock_Config_HSE_PLL_64MHz(void)
{
```

```
  RCC_OscInitTypeDef osc = {0};
  RCC_ClkInitTypeDef clk = {0};

  osc.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  osc.HSEState = RCC_HSE_ON;
  osc.PLL.PLLState = RCC_PLL_ON;
  osc.PLL.PLLSource = RCC_PLLSOURCE_HSE;

  /* Exemplo para HSE=8 MHz:
    M=1, N=16 => VCO=128 MHz
    R=2 => PLLR=64 MHz
  */
  osc.PLL.PLLM = RCC_PLLM_DIV1;
  osc.PLL.PLLN = 16;
  osc.PLL.PLLR = RCC_PLLR_DIV2;
  osc.PLL.PLLP = RCC_PLLP_DIV2;
  osc.PLL.PLLQ = RCC_PLLQ_DIV2;

  if (HAL_RCC_OscConfig(&osc) != HAL_OK) Error_Handler();

  clk.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_PCLK1;
  clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  clk.AHBCLKDivider = RCC_SYSCLK_DIV1;
  clk.APB1CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_2) != HAL_OK) Error_Handler();
}
```

Dica de robustez

Se HSE falhar, habilite CSS (via CubeMX normalmente). Em produto, trate a interrupção e degrade para HSI.

---

3) Cenário C — RTC em LSE (32.768 kHz) + SYSCLK em PLL (64 MHz)

Quando usar

Você quer RTC preciso (calendário real, alarme correto, timestamp).

Ainda quer CPU rápida em 64 MHz (PLL).

Passos

Habilite acesso ao backup domain (HAL_PWR_EnableBkUpAccess)

Ligue LSE e selecione RTCCLKSOURCE_LSE

Ligue SYSCLK (HSI/HSE + PLL) normalmente

Inicialize RTC com prescalers típicos de 1 Hz

Código (RTC clock + SystemClock)

```
#include "stm32g0xx_hal.h"

static void Error_Handler(void)
{
  __disable_irq();
  while (1) {}
}

static void RTC_Clock_LSE_Enable(void)
{
  __HAL_RCC_PWR_CLK_ENABLE();
  HAL_PWR_EnableBkUpAccess();

  __HAL_RCC_LSE_CONFIG(RCC_LSE_ON);
```

```
    uint32_t t0 = HAL_GetTick();
    while (__HAL_RCC_GET_FLAG(RCC_FLAG_LSERDY) == RESET)
    {
        if ((HAL_GetTick() - t0) > 2000) Error_Handler();
    }

    __HAL_RCC_RTC_CONFIG(RCC_RTCCLKSOURCE_LSE);
    __HAL_RCC_RTC_ENABLE();
}

void SystemClock_Config_PLL_64MHz_and_RTC_LSE(void)
{
    /* 1) RTC em LSE (backup domain) */
    RTC_Clock_LSE_Enable();

    /* 2) SYSCLK em 64 MHz (ex.: HSI16->PLL) */
    RCC_OscInitTypeDef osc = {0};
    RCC_ClkInitTypeDef clk = {0};

    osc.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    osc.HSIState = RCC_HSI_ON;
    osc.HSIDiv = RCC_HSI_DIV1;
    osc.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;

    osc.PLL.PLLState = RCC_PLL_ON;
    osc.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    osc.PLL.PLLM = RCC_PLLM_DIV1;
    osc.PLL.PLLN = 8;
```

```
  osc.PLL.PLLR = RCC_PLLR_DIV2;
  osc.PLL.PLLP = RCC_PLLP_DIV2;
  osc.PLL.PLLQ = RCC_PLLQ_DIV2;

  if (HAL_RCC_OscConfig(&osc) != HAL_OK) Error_Handler();

  clk.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_PCLK1;
  clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  clk.AHBCLKDivider = RCC_SYSCLK_DIV1;
  clk.APB1CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_2) != HAL_OK) Error_Handler();
}
```

Depois disso, seu RTC init (prescalers 127/255 para LSE) pode ser feito normalmente no HAL.

---

4) Cenário D — ADC com clock via PLLP enquanto SYSCLK usa PLLR (clocks independentes de verdade)

Quando usar

- Você quer CPU em 64 MHz (PLLR)
- Quer alimentar ADC com um clock específico (PLLP), sem “amarrar” no SYSCLK
- Quer estabilidade / desempenho do ADC

Passos

- Configure PLL com R e P habilitados
- SYSCLK = PLLR
- Configure PeriphCLK do ADC para usar PLLP (HAL_RCCEx_PeriphCLKConfig)
- Inicialize ADC normalmente

Código (SystemClock + PeriphClock do ADC)

```
#include "stm32g0xx_hal.h"

static void Error_Handler(void)
{
    __disable_irq();
    while (1) {}
}

void SystemClock_Config_PLLR_64MHz_ADC_PLLP(void)
{
    RCC_OscInitTypeDef osc = {0};
    RCC_ClkInitTypeDef clk = {0};

    /* 1) HSI16 + PLL ON */
    osc.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    osc.HSIState = RCC_HSI_ON;
    osc.HSIDiv = RCC_HSI_DIV1;
    osc.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;

    osc.PLL.PLLState = RCC_PLL_ON;
    osc.PLL.PLLSource = RCC_PLLSOURCE_HSI;

    /* Exemplo: VCO 128 MHz, PLLR=64 MHz */
    osc.PLL.PLLM = RCC_PLLM_DIV1;
    osc.PLL.PLLN = 8;
    osc.PLL.PLLR = RCC_PLLR_DIV2;

    /* Habilite PLLP para ADC (frequência dependerá do divisor P) */
```

```
  osc.PLL.PLLP = RCC_PLLP_DIV2;  // 128/2 = 64 MHz em PLLP (exemplo)
  osc.PLL.PLLQ = RCC_PLLQ_DIV2;

  if (HAL_RCC_OscConfig(&osc) != HAL_OK) Error_Handler();

  /* 2) SYSCLK = PLLR */
  clk.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK |
RCC_CLOCKTYPE_PCLK1;
  clk.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  clk.AHBCLKDivider = RCC_SYSCLK_DIV1;
  clk.APB1CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_2) != HAL_OK) Error_Handler();

  /* 3) Clock do ADC vindo do PLLP */
  RCC_PeriphCLKInitTypeDef p = {0};
  p.PeriphClockSelection = RCC_PERIPHCLK_ADC;
  p.AdcClockSelection = RCC_ADCCLKSOURCE_PLLP;

  if (HAL_RCCEx_PeriphCLKConfig(&p) != HAL_OK) Error_Handler();
}
```

Como saber “qual clock meu ADC está usando?”

Em HAL, você garante pela PeriphCLKConfig.

Para depurar, valide o SYSCLK/HCLK/PCLK e confirme no .ioc/CubeMX a seleção do ADC clock.

---

5) “Como eu sei quais clocks preciso ligar para cada periférico?”

Regra de ouro (prática de projeto):

5.1 Primeiro nível: habilitar clock do periférico (gating)

Exemplos comuns:

__HAL_RCC_GPIOA_CLK_ENABLE();

__HAL_RCC_USART2_CLK_ENABLE();

__HAL_RCC_SPI1_CLK_ENABLE();

__HAL_RCC_I2C1_CLK_ENABLE();

__HAL_RCC_ADC_CLK_ENABLE();

__HAL_RCC_TIM3_CLK_ENABLE();

Se você não habilitar, o periférico não funciona.

5.2 Segundo nível: selecionar fonte (quando existe MUX)

Use HAL_RCCEx_PeriphCLKConfig() somente quando:

você quer operar em baixo consumo com clock específico

ou o periférico tem opção de fonte (USART/I2C/ADC/RTC etc.)

---

6) Mini-checklist "de bancada" (quando algo dá errado)

SYSCLK está mesmo no que você pensa? (HAL_RCC_GetSysClockFreq())

HCLK/PCLK batem com seus cálculos?

PLL está "ready"? (flags)

Periférico tem clock habilitado? (__HAL_RCC_xxx_CLK_ENABLE)

Periférico tem fonte selecionada correta? (ADC/USART/I2C)

Flash latency compatível? (se errar, pode travar/instabilizar)

Quadro rápido de clocks por periférico (STM32G070RB) — "o que ligar, onde está e de onde vem"

A ideia aqui é ser um guia de consulta: para cada periférico, você vê:

Clock gating que você precisa habilitar (__HAL_RCC_..._CLK_ENABLE())

Barramento (AHB/APB) onde ele "vive" (conceito)

Fonte de clock: se é só PCLK/HCLK ou se tem seletor (MUX) via HAL_RCCEx_PeriphCLKConfig()

Observações práticas (pega-ratão comum)

Nota: o RCC do STM32G0 permite “peripheral clock selection” para vários blocos (USART/I2C/ADC/RTC etc.). O detalhamento completo está no datasheet/RM; aqui fica o que você precisa no dia a dia. (st.com)

---

1) GPIO (PORTA/PORTB/PORTC…)

Gating: __HAL_RCC_GPIOA_CLK_ENABLE() (e B/C/D conforme usar)

Barramento: AHB (I/O port clock)

Fonte: vem do domínio AHB (HCLK)

Pegadinha: sem clock de GPIO, nem configuração de AF funciona.

---

2) DMA1

Gating: __HAL_RCC_DMA1_CLK_ENABLE()

Barramento: AHB

Fonte: HCLK (AHB)

Pegadinha: além do DMA, você precisa habilitar NVIC da IRQ do canal.

---

3) ADC (ADC1)

Gating: __HAL_RCC_ADC_CLK_ENABLE()

Barramento: APB (periférico), mas com clock de conversão próprio

Fonte (MUX): pode vir de PCLK ou PLLP (dependendo da config do G0) via:

HAL_RCCEx_PeriphCLKConfig() com RCC_PERIPHCLK_ADC e RCC_ADCCLKSOURCE_...

Pegadinha: mesmo com gating habilitado, se a fonte do ADC estiver errada, conversão fica fora do esperado (taxa/precisão). (st.com)

---

4) TIM (Timers)

4.1 TIM6/TIM7 (basic)

Gating: __HAL_RCC_TIM6_CLK_ENABLE(), __HAL_RCC_TIM7_CLK_ENABLE()

Barramento: APB

Fonte: PCLK (APB timer clock derivado de HCLK/PCLK)

Pegadinha: se APB prescaler ≠ 1, em várias famílias o “timer clock” pode ser multiplicado internamente; confirme no RM/datasheet para o seu modo.

4.2 TIM1/TIM3/TIM14/15/16/17 (general/advanced)

Gating: __HAL_RCC_TIMx_CLK_ENABLE()

Barramento: APB

Fonte: PCLK (derivada do clock do barramento)

Pegadinha: PWM só sai no pino se além do clock você configurar GPIO AF correto.

---

5) USART/UART (ex.: USART2)

Gating: __HAL_RCC_USART2_CLK_ENABLE()

Barramento: APB

Fonte (MUX): frequentemente pode ser:

PCLK (APB)

HSI16

LSE

SYSCLK

(conforme instância e família) via HAL_RCCEx_PeriphCLKConfig() (RCC_PERIPHCLK_USARTx). (st.com)

Pegadinha: se o baud estiver “estranho” após mudar SYSCLK, você provavelmente está usando fonte diferente do que imagina.

---

6) I²C (I2C1/I2C2)

Gating: __HAL_RCC_I2C1_CLK_ENABLE(), etc.

Barramento: APB

Fonte (MUX): normalmente selecionável (PCLK/HSI16/SYSCLK, depende da instância) via HAL_RCCEx_PeriphCLKConfig() (RCC_PERIPHCLK_I2C1). (st.com)

Pegadinha: I²C é sensível ao clock: se você trocar o SYSCLK/PCLK e não regenerar o timing, a comunicação pode ficar instável.

---

7) SPI (SPI1/SPI2)

Gating: __HAL_RCC_SPI1_CLK_ENABLE(), etc.

Barramento: APB

Fonte: em geral PCLK (sem MUX comum no dia a dia)

Pegadinha: máxima frequência de SCK depende de PCLK e prescaler do SPI.

---

8) RTC + TAMP + Backup Registers

Gating: __HAL_RCC_RTC_ENABLE() (e habilitar fonte no BDCR)

Barramento: domínio de backup (separado)

Fonte (MUX):

LSE (32.768k)

LSI (~32k)

HSE/RTCCLK (em algumas famílias)

Seleção via __HAL_RCC_RTC_CONFIG(RCC_RTCCLKSOURCE_...). (st.com)

Pegadinha: precisa liberar escrita no backup domain: HAL_PWR_EnableBkUpAccess().

---

9) IWDG (Independent Watchdog)

Gating: não depende do gating do barramento como periféricos comuns; é ativado/configurado via suas chaves.

Fonte: LSI (independente, ~32 kHz). (st.com)

Pegadinha: se habilitado por option bytes (hardware), começa a contar cedo.

---

10) WWDG (Window Watchdog) — referência rápida

Gating: __HAL_RCC_WWDG_CLK_ENABLE()

Barramento: APB

Fonte: PCLK

Pegadinha: diferente do IWDG, ele depende do clock do sistema/barramento.

---

11) SYSCFG / COMP / VREFBUF (analógico e sistema)

Gating: __HAL_RCC_SYSCFG_CLK_ENABLE() (para coisas como EXTI mapping, etc.)

Fonte: barramento (AHB/APB conforme bloco)

Pegadinha: muita gente esquece SYSCFG e EXTI não funciona como esperado.

---

12) Como usar isso na prática (receita curta)

Configure SystemClock (SYSCLK/HCLK/PCLK).

Para cada periférico:

habilite o gating clock __HAL_RCC_..._CLK_ENABLE()

configure pinos (GPIO AF, se aplicável)

se o periférico tiver MUX de clock, selecione com HAL_RCCEx_PeriphCLKConfig()

inicialize o periférico (HAL_xxx_Init)

Se algo ficar estranho após mudar o clock do sistema:

confirme fonte do periférico (MUX)

regenere timings (I^2C, UART baud, ADC clock, timers)

Mini-diagnóstico de clocks pelo debugger (STM32G070): como conferir fonte, prescalers e READY flags

A ideia é você conseguir abrir o watch window do debugger e responder, em minutos:

"SYSCLK está vindo de onde?"

"PLL está ligada e pronta?"

"HSE/LSE realmente estabilizaram?"

"AHB/APB estão dividindo como?"

"Qual clock foi selecionado para ADC/USART/I^2C?"

Abaixo vai um bloco independente (HAL + leitura direta de registradores) que você pode colar no projeto e inspecionar as variáveis.

---

1) Estrutura de snapshot de clocks (tudo em um lugar)

```
#include "stm32g0xx_hal.h"

typedef struct
{
    /* Frequências calculadas via HAL */
    uint32_t sysclk_hz;
    uint32_t hclk_hz;
    uint32_t pclk1_hz;

    /* RCC status flags */
    uint8_t hsi_ready;
    uint8_t hse_ready;
    uint8_t lsi_ready;
    uint8_t lse_ready;
    uint8_t pll_ready;

    /* SYSCLK source (raw + decodificado) */
    uint32_t cfgr_sws;
    uint8_t  sys_src;   // 0=HSI, 1=HSE, 2=PLL, 3=LSI/LSE (depende do chip)

    /* Prescalers (raw) */
    uint32_t ahb_presc_bits;
    uint32_t apb1_presc_bits;

    /* Periph clock selections (raw registers) */
    uint32_t ccipr;     // clock selection register para periféricos (USART/I2C/ADC etc.)
```

```
    uint32_t bdcr;      // backup domain control (RTC/LSE)
    uint32_t cr;        // RCC_CR (HSI/HSE/PLL enable/ready)
    uint32_t cfgr;      // RCC_CFGR (SYSCLK src + prescalers)

} clock_snapshot_t;

static clock_snapshot_t g_clk;

static uint8_t decode_sysclk_src(uint32_t sws_bits)
{
    /* No RCC_CFGR.SWS, valores típicos:
       0b00 HSI
       0b01 HSE
       0b10 PLL
       0b11 reservado/outro (depende da família)
    */
    switch (sws_bits)
    {
        case RCC_CFGR_SWS_HSI: return 0;
        case RCC_CFGR_SWS_HSE: return 1;
        case RCC_CFGR_SWS_PLL: return 2;
        default:               return 3;
    }
}

/* Chame isso depois do SystemClock_Config() */
void clocks_snapshot_update(void)
{
```

```
    g_clk.sysclk_hz = HAL_RCC_GetSysClockFreq();
    g_clk.hclk_hz   = HAL_RCC_GetHCLKFreq();
    g_clk.pclk1_hz  = HAL_RCC_GetPCLK1Freq();

    g_clk.hsi_ready = (__HAL_RCC_GET_FLAG(RCC_FLAG_HSIRDY) != RESET);
    g_clk.hse_ready = (__HAL_RCC_GET_FLAG(RCC_FLAG_HSERDY) != RESET);
    g_clk.lsi_ready = (__HAL_RCC_GET_FLAG(RCC_FLAG_LSIRDY) != RESET);
    g_clk.lse_ready = (__HAL_RCC_GET_FLAG(RCC_FLAG_LSERDY) != RESET);
    g_clk.pll_ready = (__HAL_RCC_GET_FLAG(RCC_FLAG_PLLRDY) != RESET);

    g_clk.cr   = RCC->CR;
    g_clk.cfgr = RCC->CFGR;
    g_clk.ccipr = RCC->CCIPR;
    g_clk.bdcr  = RCC->BDCR;

    g_clk.cfgr_sws = (g_clk.cfgr & RCC_CFGR_SWS);
    g_clk.sys_src = decode_sysclk_src(g_clk.cfgr_sws);

    g_clk.ahb_presc_bits  = (g_clk.cfgr & RCC_CFGR_HPRE);
    g_clk.apb1_presc_bits = (g_clk.cfgr & RCC_CFGR_PPRE);
}
```

Como usar

No main():

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
```

```
    clocks_snapshot_update();


    while (1)
    {
        // Coloque breakpoint aqui e veja g_clk no debugger.
    }
}
```

---

2) Como interpretar rapidamente (na prática)

2.1 Fonte do SYSCLK (SWS)

No debugger:

g_clk.sys_src:

0 = HSI

1 = HSE

2 = PLL

3 = “outro / reservado” (verifique g_clk.cfgr_sws)

Isso confirma “de onde vem o SYSCLK” sem achismo.

2.2 Prescalers AHB/APB

Você vê:

g_clk.ahb_presc_bits (HPRE)

g_clk.apb1_presc_bits (PPRE)

Se quiser decodificar para divisores (1/2/4/8…), o jeito mais prático é:

conferir o manual/datasheet (tabela HPRE/PPRE),

ou confiar nos valores já calculados: hclk_hz e pclk1_hz.

2.3 Flags READY

Você sabe se o oscilador realmente está estável:

g_clk.hse_ready / g_clk.lse_ready (os mais críticos quando usados)

g_clk.pll_ready confirma PLL travada

---

3) Diagnóstico de clocks de periféricos (CCIPR / BDCR)

Para periféricos com MUX de clock, o STM32G0 usa registradores como CCIPR para selecionar fonte de USART/I²C/ADC, e o BDCR para RTC/LSE.

No snapshot:

g_clk.ccipr → seleções de clock de periféricos (você confere os campos no RM/CubeMX)

g_clk.bdcr → LSE, RTC source e enable

Exemplo rápido: "RTC está realmente em LSE?"

g_clk.lse_ready == 1

g_clk.bdcr deve ter:

LSEON ligado

RTCSEL = LSE

RTCEN ligado

---

4) Função opcional: imprimir/registrar diagnóstico (sem printf obrigatório)

Para deixar o diagnóstico rastreável sem depender de `printf`, o snapshot pode ser armazenado em Flash, backup registers ou transmitido por UART quando disponível.

Exemplo mínimo: chame clocks_snapshot_update() e mantenha g_clk global; o debugger já resolve.

---

5) Checklists de problemas comuns (usando o snapshot)

"Meu clock não mudou para PLL"

g_clk.pll_ready está 0 → PLL não travou (config inválida ou fonte não ready)

g_clk.sys_src não é 2 → SYSCLK não foi setado para PLL (ClockConfig falhou)

"HSE não funciona"

g_clk.hse_ready 0 após tempo → cristal/clock externo falhou ou config errada

solução: conferir carga do cristal, drive, bypass, CSS, layout

"RTC não mantém hora"

g_clk.lse_ready 0 → LSE não oscilou

g_clk.bdcr não está com RTCSEL = LSE

backup domain sem acesso (BkUpAccess não habilitado) antes de configurar BDCR

"I²C/UART ficou estranho depois que mudei SYSCLK"

g_clk.sysclk_hz mudou, mas periférico está com fonte PCLK

ou CCIPR selecionou outra fonte (HSI16/SYSCLK/LSE)

I²C timing e UART baud precisam ser recalculados (CubeMX)

Capítulo 12 — Interrupções e eventos no STM32G070 (NVIC + EXTI): lista completa, como habilitar, desabilitar, pendente, flags e limpeza

Este capítulo explica todo o caminho de uma interrupção no STM32G070: do evento físico até a execução do ISR, e mostra como controlar isso via HAL e também via registradores (CMSIS). O objetivo é o leitor sair sabendo diagnosticar e dominar qualquer interrupção do chip.

---

1. Visão geral: quem faz o quê

1.1 CPU (Cortex-M0+) e "exceptions"

A CPU lida com exceptions, que incluem:

internas do core (SysTick, PendSV, SVC, HardFault, NMI),

externas (IRQ de periféricos, EXTI, DMA etc.).

Quando uma exception ocorre:

o hardware salva automaticamente contexto essencial na stack,

busca o endereço do handler na vector table,

executa o ISR e depois retorna.

1.2 NVIC (Nested Vectored Interrupt Controller)

O NVIC é o “gerenciador” das interrupções externas:

tem 32 linhas de interrupção mascaráveis no STM32G070,

possui 4 níveis de prioridade (2 bits),

faz preempção (prioridade maior interrompe menor) e tail-chaining.

1.3 EXTI (Extended Interrupt/Event Controller)

O EXTI é o bloco que recebe eventos de pinos e alguns sinais internos e pode:

gerar interrupção (IRQ) e/ou

gerar evento (para acordar do Stop, por exemplo),

“latchar” (registrar) eventos e capturar pulsos curtos.

---

2. Lista completa de interrupções do STM32G070 (IRQn_Type)

A lista abaixo é a lista oficial de IRQs para STM32G070xx conforme o header CMSIS do device (é o que o compilador usa para nomes como TIM3_IRQn, USART2_IRQn, etc.).

2.1 Exceptions do Cortex-M0+ (números negativos)

NonMaskableInt_IRQn (NMI)

HardFault_IRQn

SVCall_IRQn

PendSV_IRQn

SysTick_IRQn

No Cortex-M0+, não existem MemManage/BusFault/UsageFault como no M3/M4; “HardFault” cobre os casos principais.

2.2 IRQs específicos STM32G070xx (0 a 29)

WWDG_IRQn (Window Watchdog)

RTC_TAMP_IRQn (RTC/Tamper via EXTI 19 & 21)

FLASH_IRQn

RCC_IRQn

EXTI0_1_IRQn

EXTI2_3_IRQn

EXTI4_15_IRQn

DMA1_Channel1_IRQn

DMA1_Channel2_3_IRQn

DMA1_Ch4_7_DMAMUX1_OVR_IRQn

ADC1_IRQn

TIM1_BRK_UP_TRG_COM_IRQn

TIM1_CC_IRQn

TIM3_IRQn

TIM6_IRQn

TIM7_IRQn

TIM14_IRQn

TIM15_IRQn

TIM16_IRQn

TIM17_IRQn

I2C1_IRQn (combinado com EXTI 23)

I2C2_IRQn

SPI1_IRQn

SPI2_IRQn

USART1_IRQn

USART2_IRQn

USART3_4_IRQn

---

3. Como uma interrupção “nasce”: as 3 condições obrigatórias

Para uma IRQ realmente chamar o handler, três coisas precisam estar verdadeiras ao mesmo tempo:

A fonte gerou o pedido

Ex.: periférico setou sua flag (RXNE, TC, UIF, etc.) ou EXTI setou pending.

A fonte está habilitada localmente (no periférico/EXTI)

Ex.: USARTx->CR1 habilitando RXNEIE, TIMx->DIER habilitando UIE, EXTI->IMR1 liberando a linha, etc.

O NVIC está habilitado para aquela IRQ

Ex.: NVIC_EnableIRQ(USART2_IRQn).

Se qualquer um desses 3 estiver “off”, a interrupção não entra (mas pode ficar pendente aguardando).

---

4. NVIC na prática: habilitar, desabilitar, prioridade, pendência e ativo

4.1 Habilitar e desabilitar (HAL)

```
HAL_NVIC_SetPriority(USART2_IRQn, 1, 0);

HAL_NVIC_EnableIRQ(USART2_IRQn);


HAL_NVIC_DisableIRQ(USART2_IRQn);
```

No M0+ __NVIC_PRIO_BITS = 2, então as prioridades efetivas vão de 0 a 3 (0 = mais alta).

4.2 Forçar pendente, limpar pendente e checar pendente (HAL)

```
HAL_NVIC_SetPendingIRQ(USART2_IRQn);     // força "pendente" no NVIC

HAL_NVIC_ClearPendingIRQ(USART2_IRQn);   // limpa pendência no NVIC

uint32_t p = HAL_NVIC_GetPendingIRQ(USART2_IRQn);
```

Importante

“Pendência no NVIC” ≠ “flag do periférico”.

Se você setar pendente no NVIC mas o handler limpa nada no periférico, pode gerar loop ou comportamento estranho dependendo da fonte.

4.3 Saber se está “ativo” (em execução)

Via CMSIS:

```
uint32_t a = NVIC_GetActive(USART2_IRQn); // 0 ou 1
```

4.4 Habilitar/desabilitar globalmente (PRIMASK)

Isso não desabilita NMI/HardFault, mas mascara as IRQs normais:

__disable_irq();

__enable_irq();

Use com cuidado (trechos mínimos).

---

5. Flags x Pendência: como pensar corretamente

Em 99% dos periféricos STM32:

O periférico tem uma flag (ex.: RXNE, TC, UIF, ERR).

O periférico tem um enable local (ex.: RXNEIE, UIE, ERRIE).

Quando (flag AND enable) = 1, ele "pede" interrupção ao NVIC.

Limpar pendência do NVIC não limpa a flag do periférico.

Você precisa limpar a flag do periférico (ou ler um registrador que a limpa) para a fonte parar de pedir interrupção.

---

6. EXTI por dentro: linhas, máscara, bordas, pendência e software trigger

6.1 Linhas EXTI e mapeamento

Linhas EXTI0..EXTI15 correspondem ao número do pino (0..15).

Você escolhe qual porta (A/B/C/...) alimenta cada linha via EXTI->EXTICR[].

Exemplo: PA0 usa EXTI0. PC13 usa EXTI13 (via EXTI4_15_IRQn).

6.2 Borda de disparo

EXTI->RTSR1 habilita borda de subida

EXTI->FTSR1 habilita borda de descida

6.3 Mascarar interrupção e/ou evento

EXTI->IMR1 libera/bloqueia interrupção

EXTI->EMR1 libera/bloqueia evento

Evento é útil para acordar do low-power sem necessariamente executar ISR.

6.4 Pendência no EXTI: como verificar e limpar (isso é essencial)

No STM32G0, há pendência separada por borda:

RPR1 = Rising Pending

FPR1 = Falling Pending

Checar pendente (manual):

```
if (EXTI->RPR1 & (1u << 13)) { /* houve rising em EXTI13 */ }
if (EXTI->FPR1 & (1u << 13)) { /* houve falling em EXTI13 */ }
```

Limpar pendente (manual):

no EXTI, você limpa escrevendo 1 no bit correspondente:

```
EXTI->RPR1 = (1u << 13);   // limpa pending de rising
EXTI->FPR1 = (1u << 13);   // limpa pending de falling
```

6.5 Forçar EXTI por software (Software Interrupt/Event)

O EXTI tem SWIER1 para forçar evento/interrupção via software:

```
EXTI->SWIER1 = (1u << 13); // gera software interrupt/event na linha 13
```

---

7. EXTI com HAL: o caminho "padrão do livro"

7.1 ISR chama o handler HAL

Exemplo para linhas 4..15:

```
void EXTI4_15_IRQHandler(void)
{
   HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
}
```

7.2 Callback único do HAL (onde você trata)

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
   if (GPIO_Pin == GPIO_PIN_13)
   {
      // ação do botão/sinal
   }
}
```

7.3 Verificar e limpar flag EXTI via macros HAL

```
if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_13) != 0u)
{
  __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_13); // limpa a pending do EXTI
}
```

Isso limpa a pendência do EXTI (equivalente a escrever 1 em PR/RPR/FPR conforme o caso).

---

8. "Como limpar flag" (regra geral por tipo de fonte)

8.1 EXTI

limpar pendente no EXTI (RPR/FPR) — escrevendo 1 no bit, ou via macro HAL.

8.2 Periféricos (USART, TIM, ADC, DMA…)

Cada periférico tem sua própria regra:

alguns flags limpam ao ler status e depois ler dado (ex.: UART RXNE clássico),

outros limpam escrevendo 0/1 em um registro de clear (ex.: I2C tem ICR),

outros limpam ao escrever em SR/IFCR etc. (ex.: DMA IFCR).

A regra prática de especialista é:

Identifique qual flag está gerando a IRQ

Limpe a flag do periférico do jeito correto

Só então a interrupção deixa de reentrar continuamente

---

9. Exercícios "de bancada": forçar, observar e provar que entendeu

9.1 Forçar pendência no NVIC (sem periférico)

```
HAL_NVIC_EnableIRQ(TIM3_IRQn);
HAL_NVIC_SetPendingIRQ(TIM3_IRQn); // o handler deve executar imediatamente
```

Se seu TIM3_IRQHandler() estiver vazio e não chamar HAL_TIM_IRQHandler(), você verá a interrupção entrar, mas não terá "fonte" para limpar — é só teste de NVIC.

9.2 Forçar EXTI por software

configure EXTI13 como interrupção

habilite NVIC de EXTI4_15_IRQn

faça:

EXTI->SWIER1 = (1u << 13);

O handler deve disparar mesmo sem mudar o pino.

---

10. O que você sempre deve checar quando “interrupção não funciona”

Checklist rápido:

IRQ correta foi habilitada no NVIC?

(HAL_NVIC_EnableIRQ(...))

Prioridade foi configurada (e não está mascarada globalmente)?

(__enable_irq() e PRIMASK)

A flag da fonte está realmente sendo gerada?

A flag está sendo limpa corretamente no ISR?

No caso de EXTI:

linha mapeada corretamente (EXTICR)

borda correta (RTSR/FTSR)

IMR liberado

pending foi limpa (RPR/FPR)

Fechamento

Com este capítulo o leitor deve conseguir:

listar e reconhecer todas as IRQs do STM32G070,

entender NVIC (habilitar/desabilitar, prioridade, pending, active),

entender EXTI (máscaras, bordas, pending, software trigger),

diferenciar claramente flag do periférico vs pending do NVIC/EXTI,

e, principalmente, parar interrupções “em loop” limpando a flag correta no lugar correto.

Desafios da Parte 3

Desafio 1 — GPIO + EXTI: botão com debounce e LED

Objetivo

Implementar um botão com interrupção por EXTI e debounce por software, alternando um LED a cada pressionamento válido.

Requisitos

1. Configurar um LED como saída GPIO.

2. Configurar um botão com pull-up e EXTI em borda de descida.

3. Implementar debounce por tempo usando `HAL_GetTick()`.

4. Alternar o LED somente quando o evento for validado fora da ISR.

Critérios de validação

- O LED não deve piscar várias vezes por bouncing.
- Cada pressionamento válido deve gerar apenas um toggle.
- A ISR deve permanecer curta.

Desafio 2 — ADC: média e conversão para milivolts

Objetivo

Ler um canal ADC, calcular a média de 16 amostras e converter o resultado para milivolts.

Requisitos

1. Fazer a leitura por polling usando HAL.

2. Acumular 16 conversões consecutivas.

3. Calcular a média inteira.

4. Converter o valor médio para milivolts considerando ADC de 12 bits e `Vref = 3300 mV`.

Critérios de validação

- O valor médio deve variar de forma mais estável do que uma leitura única.
- A conversão para milivolts deve respeitar a faixa de 0 a 3300 mV.
- O código deve separar claramente aquisição, média e conversão.

Desafio 3 — UART: recepção de linha com buffer

Objetivo

Montar uma recepção UART simples por interrupção, acumulando bytes em um buffer até receber `\\n`.

Requisitos

1. Receber 1 byte por vez com `HAL_UART_Receive_IT()`.

2. Armazenar os bytes em um buffer circular ou linear protegido por limite.

3. Detectar fim de linha ao receber `\\n`.

4. Sinalizar para o `while(1)` que uma mensagem completa está pronta para processamento.

Critérios de validação

- O buffer não deve estourar ao receber mensagens longas.
- A recepção deve ser rearmada ao final de cada byte.
- O processamento principal deve ocorrer fora do callback.